

Formation Python avancé et scientifique

Matthieu Falce

Mars 2020

Syllabus I

Matthieu Falce

Vue d'ensemble

Vue d'ensemble

Langage Python

Langage Python

Programmation Orientée objet (POO)

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bonnes pratiques

Bibliothèque standard

Bibliothèque
standard

Création de modules

Création de
modules

Programmation concurrente

Programmation
concurrente

Python scientifique

Syllabus II

Python scientifique

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

A propos de moi – Qui suis-je ?

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ Qui suis-je ?
 - ▶ Matthieu Falce
 - ▶ habite à Lille
 - ▶ ingénieur en bioinformatique (INSA Lyon)

A propos de moi – Qui suis-je ?

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ Qu'est ce que j'ai fait ?
 - ▶ ingénieur R&D en Interaction Homme-Machine (IHM), Inria Lille, équipe [Mint](#) puis [Mjolnir](#)
 - ▶ développeur *fullstack* / *backend* à [FUN-MOOC](#) (France Université Numérique)

A propos de moi – Actuellement

Matthieu Falce

- ▶ entrepreneur salarié dans une SCOP (Société COOPérative) : MFconsulting
 - ▶ conseil en python
 - ▶ rédaction de dossier de financement de l'innovation
 - ▶ formations

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

A propos de moi – Actuellement

Matthieu Falce

- ▶ entrepreneur salarié dans une SCOP (Société COOPérative) : MFconsulting
 - ▶ conseil en python
 - ▶ rédaction de dossier de financement de l'innovation
 - ▶ formations
- ▶ créateur de *Oh Ce Cours Formation*

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

A propos de moi – Actuellement

Matthieu Falce

- ▶ entrepreneur salarié dans une SCOP (Société COOPérative) : MFconsulting
 - ▶ conseil en python
 - ▶ rédaction de dossier de financement de l'innovation
 - ▶ formations
- ▶ créateur de *Oh Ce Cours Formation*
- ▶ cofondateur de l'agence de communication **méninges à trois**
 - ▶ stratégie
 - ▶ développement back-end

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

A propos de moi – Actuellement

Matthieu Falce

- ▶ entrepreneur salarié dans une SCOP (Société COOPérative) : MFconsulting
 - ▶ conseil en python
 - ▶ rédaction de dossier de financement de l'innovation
 - ▶ formations
- ▶ créateur de *Oh Ce Cours Formation*
- ▶ cofondateur de l'agence de communication **méninges à trois**
 - ▶ stratégie
 - ▶ développement back-end
- ▶ cofondateur / CTO de **ExcellencePriority** (site de partage exclusif de petites annonces orienté luxe)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

A propos de moi – Actuellement

Matthieu Falce

- ▶ entrepreneur salarié dans une SCOP (Société COOPérative) : MFconsulting
 - ▶ conseil en python
 - ▶ rédaction de dossier de financement de l'innovation
 - ▶ formations
- ▶ créateur de *Oh Ce Cours Formation*
- ▶ cofondateur de l'agence de communication **méninges à trois**
 - ▶ stratégie
 - ▶ développement back-end
- ▶ cofondateur / CTO de **ExcellencePriority** (site de partage exclusif de petites annonces orienté luxe)
- ▶ coorganisateur de meetups à Lille
 - ▶ python
 - ▶ big data et machine learning

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Où me trouver ?

- ▶ mail: matthieu@falce.net
- ▶ github : [ice3](#)
- ▶ twitter : [@matthieufalce](#)
- ▶ site: [falce.net](#)

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Vue d'ensemble

- Historique
- Philosophie
- Open Source
- Python, CPython, ...
- Calcul scientifique
- Cas d'utilisations de python
- Installation
- Environnement de développement

Langage Python

- Programmation
Orientée objet
(POO)

- Bonnes pratiques

- Bibliothèque
standard

- Création de
modules

- Programmation
concurrente

- Python scientifique

Vue d'ensemble

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de
développement

Langage Python

Programmation Orientée objet (POO)

Bonnes pratiques

Bibliothèque standard

Création de modules

Programmation concurrente

Python scientifique

1- Vue d'ensemble

1.1. Historique

Un vieux langage ?

Matthieu Falce

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation Orientée objet (POO)

Bonnes pratiques

Bibliothèque standard

Création de modules

Programmation concurrente

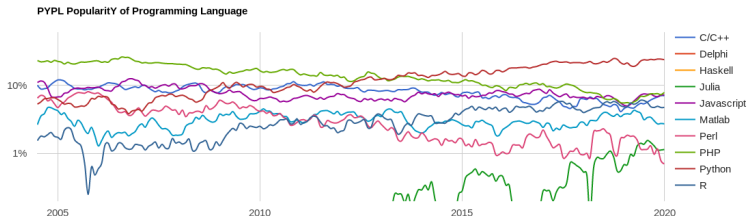
Python scientifique

- ▶ Créateur (et bdf) : Guido van Rossum
- ▶ 1ère version : 20 février 1991
- ▶ dernière version stable sortie : 3.9 (5 octobre 2020)

Un vieux langage ?

Matthieu Falce

- ▶ Créateur (et bdf) : Guido van Rossum
- ▶ 1ère version : 20 février 1991
- ▶ dernière version stable sortie : 3.9 (5 octobre 2020)



Source: <http://pypl.github.io/PYPL.html>

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation Orientée objet (POO)

Bonnes pratiques

Bibliothèque standard

Création de modules

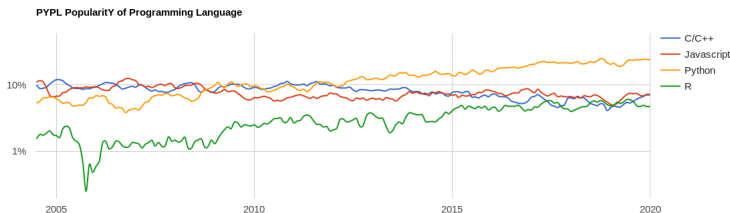
Programmation concurrente

Python scientifique

Un vieux langage ?

Matthieu Falce

- ▶ Créateur (et bdf) : Guido van Rossum
- ▶ 1ère version : 20 février 1991
- ▶ dernière version stable sortie : 3.9 (5 octobre 2020)



Source: <http://pypl.github.io/PYPL.html>

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation Orientée objet (POO)

Bonnes pratiques

Bibliothèque standard

Création de modules

Programmation concurrente

Python scientifique

Origine du nom

Matthieu Falce

Le nom n'est pas inspiré du serpent...

Over six years ago, in December 1989, I was looking for a 'hobby' programming project that would keep me occupied during the week around Christmas. My office ... would be closed, but I had a home computer, and not much else on my hands. I decided to write an interpreter for the new scripting language I had been thinking about lately: a descendant of ABC that would appeal to Unix/C hackers. I chose Python as a working title for the project, being in a slightly irreverent mood (and a big fan of Monty Python's Flying Circus)."

Guido Van Rossum

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Origine du nom

Matthieu Falce

- ▶ Il y a de nombreuses références aux Monty Python dans la communauté, la documentation officielle.
- ▶ Listing d'autres exemples sur Quora
- ▶ Le plus connu est l'utilisation de spam et egg au lieu de foo et bar.

```
def spam():  
    eggs = 12  
    return eggs  
  
print(spam())
```

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation Orientée objet (POO)

Bonnes pratiques

Bibliothèque standard

Création de modules

Programmation concurrente

Python scientifique

Rétrocompatibilité

- ▶ Python est un langage plutôt stable.
- ▶ La syntaxe a globalement peu changé depuis le début.

Un exemple de code de démo de la version 1.0.0



```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

"""
    Simple script demonstrating the Python 1.0.0 syntax.
    This script prints the Python version and the current date.
"""

import sys
import datetime

def main():
    """Main function of the script"""
    print("Python version: %s" % sys.version)
    print("Current date: %s" % datetime.datetime.now())

if __name__ == '__main__':
    main()
```

Matthieu Falce

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation Orientée objet (POO)

Bonnes pratiques

Bibliothèque standard

Création de modules

Programmation concurrente

Python scientifique

Rétrocompatibilité

Matthieu Falce

- ▶ Python est un langage plutôt stable.
- ▶ La syntaxe a globalement peu changé depuis le début.

Un exemple de code de démo de la version 1.0.0

```
from math import sqrt

class complex:

    def __init__(self, re, im):
        self.re = float(re)
        self.im = float(im)

    def __repr__(self):
        return 'complex' + `self.re, self.im`

    def __cmp__(a, b):
        a = a.__abs__()
        b = b.__abs__()
        return (a > b) - (a < b)

    def __float__(self):
        if self.im:
            raise ValueError, 'cannot convert complex to float'
        return float(self.re)

    ...
```

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de
développement

Langage Python

Programmation Orientée objet (POO)

Bonnes pratiques

Bibliothèque standard

Création de modules

Programmation concurrente

Python scientifique

Python 2 vs Python 3

Matthieu Falce

Cependant la compatibilité ascendante a été cassée en passant de python 2 à python 3.

- ▶ réduire les redondances dans le fonctionnement de Python
- ▶ suppression des méthodes obsolètes
- ▶ modification de la grammaire
- ▶ modification des opérations mathématiques
- ▶ beaucoup d'opérations deviennent paresseuses
- ▶ ...

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation Orientée objet (POO)

Bonnes pratiques

Bibliothèque standard

Création de modules

Programmation concurrente

Python scientifique

Python 2 vs Python 3

Matthieu Falce

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation Orientée objet (POO)

Bonnes pratiques

Bibliothèque standard

Création de modules

Programmation concurrente

Python scientifique

Transition plutôt compliquée :

- ▶ certains développements continuent en python 2
- ▶ nouvelles habitudes
- ▶ grosses bases de code à modifier
- ▶ manque de certaines bibliothèques "essentielles" (non portées)

De nos jours, python 3 est complètement utilisable pour un nouveau projet.

Python 2 End Of Life

Matthieu Falce

Fin du support de Python le 1er janvier 2020



Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Python 2 End Of Life

Matthieu Falce

Fin du support de Python le 1er janvier 2020

If people find catastrophic security problems in Python 2, or in software written in Python 2, then most volunteers will not help fix them. If you need help with Python 2 software, then many volunteers will not help you, and over time fewer and fewer volunteers will be able to help you. You will lose chances to use good tools because they will only run on Python 3, and you will slow down people who depend on you and work with you. Some of these problems will start on January 1. Other problems will grow over time.

<https://www.python.org/doc/sunset-python-2/>

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation

Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de
développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

1- Vue d'ensemble

1.2. Philosophie

Zen of Python

Matthieu Falce

Le langage (et ses utilisateurs) ont des idées plutôt précises de ce qui fait un "bon code".

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de
développement

Langage Python

Programmation

Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Zen of Python (PEP 20¹)²

Matthieu Falce

```
import this
```

The Zen of Python, by Tim Peters

```
Beautiful is better than ugly.  
Explicit is better than implicit.  
Simple is better than complex.  
Complex is better than complicated.  
Flat is better than nested.  
Sparse is better than dense.  
Readability counts.  
Special cases aren't special enough to break the rules.  
Although practicality beats purity.  
Errors should never pass silently.  
Unless explicitly silenced.  
In the face of ambiguity, refuse the temptation to guess.  
There should be one-- and preferably only one --obvious way to do it.  
Although that way may not be obvious at first unless you're Dutch.  
Now is better than never.  
Although never is often better than *right* now.  
If the implementation is hard to explain, it's a bad idea.  
If the implementation is easy to explain, it may be a good idea.  
Namespaces are one honking great idea -- let's do more of those!
```

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

1.<https://www.python.org/dev/peps/pep-0020/>

2.<https://inventwithpython.com/blog/2018/08/17/the-zen-of-python-explained/>

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de
développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

1- Vue d'ensemble

1.3. Open Source

Open Source – Définition

Matthieu Falce

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de
développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ logiciels dont la licence respecte les critères de l'*Open Source Initiative*
 - ▶ s'oppose au logiciel *fermé, propriétaire, privé*
 - ▶ proche du concept de *free software* (gratuit et libre de droit)

Open Source – Business Models

Matthieu Falce

Le logiciel étant accessible librement, on peut l'utiliser gratuitement si on le souhaite (dans les conditions de la licence). Comment faire quelque chose de viable économiquement ?

- ▶ support et services (conseils, meilleure connaissance)
- ▶ partenariat publicité (firefox propose google comme navigateur par défaut)
- ▶ offre SaaS / PaaS (elastic cloud par exemple)
- ▶ vente de produits (le logiciel est Open Source mais le prix de vente compense)
- ▶ fonctionnalités payantes en plus (NGINX, MySQL, ...)
- ▶ double licence (Qt, Neo4j)
- ▶ projets open sourcés
- ▶ financement d'une fondation

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Open Source – Intérêt

- ▶ gratuité (dans la plupart des cas)
- ▶ qualité du code, tout le monde peut corriger (discutable en pratique)
- ▶ plus de choix de solutions

Matthieu Falce

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Licences les plus connues ³ :

- ▶ Non copyleft
 - ▶ BSD 3-Clause "New" or "Revised" license
 - ▶ BSD 2-Clause "Simplified" or "FreeBSD" license
 - ▶ MIT license
 - ▶ Apache License 2.0
- ▶ Copyleft (la licence se propage à ceux qui l'utilisent)
 - ▶ GNU General Public License (GPL)
 - ▶ GNU Library or "Lesser" General Public License (LGPL)
 - ▶ CeCILL

Il peut y avoir des problèmes avec le droit français, le mieux est d'en parler avec un juriste / avocat compétent ⁴

3. <https://opensource.org/licenses>

4. <https://www.journaldunet.com/solutions/dsi/1141398-comment-se-reperer-dans-la-jungle-des-licences-open-source/>

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Open Source – Exemples

- ▶ Python
- ▶ GNU/Linux
- ▶ WordPress
- ▶ Apache
- ▶ ...

Et plein d'autres :

- ▶ github : 96M projets ⁵
- ▶ Ubuntu 19.10 : 30565⁶ logiciels dans les dépôts
- ▶ AUR : 47331⁷ logiciels dans les dépôts

5.<https://octoverse.github.com/>

6.<https://repology.org/>

7.<https://repology.org/>

Matthieu Falce

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de
développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de
développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

1- Vue d'ensemble

1.4. Python, CPython, ...

C'est quoi python au final ?

Matthieu Falce

Python peut désigner plusieurs choses quand on n'est pas précis.

- ▶ un langage (la syntaxe et des règles de grammaire)
- ▶ un interpréteur officiel (CPython)
- ▶ des interpréteurs tiers (Jython, IronPython, PyPy, ...)
- ▶ des compilateurs (Cython, Nuitka, ...)

La plupart des gens parlent de CPython avec la grammaire standard quand ils parlent de python.

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de
développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

1- Vue d'ensemble

1.5. Calcul scientifique

Python c'est lent ?

Oui

Matthieu Falce

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de
développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Python c'est lent ?

Matthieu Falce

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de
développement

Langage Python

Programmation Orientée objet (POO)

Bonnes pratiques

Bibliothèque standard

Création de modules

Programmation concurrente

Python scientifique

- ▶ langage dynamiquement typé
- ▶ interprété
- ▶ pas de gestion fine de la mémoire (garbage collection)
- ▶ probablement moins rapide que le programme équivalent en C/C++/Go/Rust/(Java)/insérer langage préféré ⁸

8. <https://benchmarksgame.alioth.debian.org/u64q/compare.php?lang=python3&lang2=gpp>

Python c'est lent ?

Matthieu Falce

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de
développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Oui
Mais...⁸ ⁹

8.<https://www.quora.com/Why-is-Python-so-popular-despite-being-so-slow>

9.<https://www.quora.com/Why-is-Python-used-for-deep-learning-if-it-is-so-slow>

Python c'est lent ?

Matthieu Falce

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation Orientée objet (POO)

Bonnes pratiques

Bibliothèque standard

Création de modules

Programmation concurrente

Python scientifique

En fait, c'est plus subtil :

- ▶ moins d'optimisations inutiles
- ▶ temps de développement réduits
- ▶ langage de haut niveau $\Rightarrow$ moins de code $\Rightarrow$ moins de bugs
- ▶ code proche de l'algorithme
- ▶ VM probablement mieux écrite que la solution naïve ⁸
- ▶ possibilité d'optimiser les parties critiques

⁸<https://stackoverflow.com/questions/9371238/why-is-reading-lines-from-stdin-much-slower-in-c-than-python?rq=1>

- ▶ interface entre langages
- ▶ facilité de développement

Python a été pensé pour servir d'interface entre les langages : ⁹

- ▶ manipulation d'autres programmes (à la bash)
- ▶ échange de données binaires directement entre langages

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation

Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

9.<https://www.python.org/doc/essays/omg-darpa-mcc-position/>

10.<https://www.datacamp.com/community/blog/python-scientific-computing-case>

- ▶ interface entre langages
- ▶ facilité de développement

Python a été pensé pour servir d'interface entre les langages : ⁹

- ▶ manipulation d'autres programmes (à la bash)
- ▶ échange de données binaires directement entre langages

Exemple : lancer un calcul puis l'analyser en utilisant des méthodes en C ou Fortran.

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation

Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

9.<https://www.python.org/doc/essays/omg-darpa-mcc-position/>

10.<https://www.datacamp.com/community/blog/python-scientific-computing-case>

Pour les scientifiques ⁹

Matthieu Falce

- ▶ interface entre langages
- ▶ facilité de développement

Langage utilisé dans les milieux scientifiques. Quelques raisons :

- ▶ Prototypage
- ▶ Programmation accessible aux experts scientifiques, pas qu'aux développeurs
- ▶ "piles incluses" + écosystème riche

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation

Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Scripting

Exemples personnels

Installation

Environnement de
développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

1- Vue d'ensemble

1.6. Cas d'utilisations de python

Interpréteur embarqué dans des logiciels

Matthieu Falce

Python sert de langage de script dans de nombreux logiciels :

- ▶ blender ¹⁰
- ▶ qgis ¹¹
- ▶ autodesk ¹²
- ▶ Vim ¹³
- ▶ Minecraft ¹⁴
- ▶ ...

10.<https://blender.org>

11.<https://qgis.org/en/site/>

12.<https://autodesk.com/>

13.<https://www.vim.org/>

14.<https://minecraft.net/fr-ca/>

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Scripting

Exemples personnels

Installation

Environnement de
développement

Langage Python

Programmation Orientée objet (POO)

Bonnes pratiques

Bibliothèque standard

Création de modules

Programmation concurrente

Python scientifique

Exemples personnels

Matthieu Falce

- ▶ électronique / projets *makers*
 - ▶ Artefact (un jeu d'énigmes tangible) ¹⁵ ¹⁶
 - ▶ *Real Full Stack Python* (du microcontrôleur à la page web en python) ¹⁷
 - ▶ Réalisation de souris / claviers / *joysticks* / *touchpad* USB HID
- ▶ Web
 - ▶ EdX ¹⁸ / OpenFUN ¹⁹
- ▶ Analyse de données
 - ▶ analyse de séries temporelles
 - ▶ analyse géospatiale

15.<https://bidouilleurslibristes.github.io/Artefact/>

16.http://falce.net/presentation/Artefact-LillePy/prez_artefact.slides.html

17.http://falce.net/presentation/IoT_Dashboard/index.html

18.<https://github.com/edx>

19.<https://github.com/openfun>

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Scripting

Exemples personnels

Installation

Environnement de
développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

Environnement de
développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

1- Vue d'ensemble

1.7. Installation

Vue d'ensemble

Historique
Philosophie
Open Source
Python, CPython, ...
Calcul scientifique
Cas d'utilisations de python

Installation

Environnement de développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Il existe plusieurs distributions python.

Les plus connues :

- ▶ l'officielle
- ▶ anaconda
- ▶ compilation par Intel
- ▶ ...

Pour commencer et sous Windows, je conseille anaconda

Vue d'ensemble

Historique

Philosophie

Open Source

Python, CPython, ...

Calcul scientifique

Cas d'utilisations de python

Installation

**Environnement de
développement**

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

1- Vue d'ensemble

1.8. Environnement de développement

Pas forcément besoin d'outils spécifiques pour développer (à part un éditeur de texte)...

- ▶ éditeurs de texte + extensions
 - ▶ Microsoft Studio Code
 - ▶ sublime text 3 + anaconda (plugin ST, pas la distribution du dessus...)
 - ▶ ViM / Emacs + plugins
- ▶ IDE
 - ▶ eclipse + mode python
 - ▶ PyCharm

Toujours une faiblesse des outils par rapport à Java par exemple... Mais ça s'améliore.

Vue d'ensemble

Historique
Philosophie
Open Source
Python, CPython, ...
Calcul scientifique
Cas d'utilisations de python
Installation
Environnement de développement

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Langage Python

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2- Langage Python

2.1. Syntaxe

Votre premier programme Python

Matthieu Falce



A partir de maintenant, toutes les commandes se tapent dans un terminal.

Comment lancer un programme python ?

```
## En codant directement
## depuis l'interpréteur

python
# Python 3.6.3 (default,
# Oct 3 2017, 21:45:48)
# [GCC 7.2.0] on linux
# Type "help", "copyright",
# "credits" or "license"
# for more information.

print("Bonjour le monde")
```

```
## En lançant un fichier.py

# on écrit dans un fichier
echo \
    "print('Bonjour le monde')" \
    > hello.py

# on lance le fichier
python hello.py
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Votre premier programme Python



A partir de maintenant, toutes les commandes se tapent
dans un terminal.

Comment lancer un programme python ?

Essayez aussi : jupyter notebook et ouvrez votre navigateur sur le lien marqué dans la console

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Analyse de la syntaxe

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Factorielle en Python

```
def factorielle(n):  
    if n < 2:  
        return 1  
    else:  
        return n * factorielle(n - 1)
```

// factorielle en C

```
int factorielle(int n) {  
    if (n < 2) {  
        return 1;  
    } else {  
        return n * factorielle(n - 1);  
    }  
}
```

Analyse de la syntaxe

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Factorielle en Python

```
def factorielle(n):  
    """Doc de la fonction.  
    Prend un nombre et renvoie n!  
  
    Args:  
        n (int): le nombre  
        dont on veut la factorielle.  
  
    Returns:  
        int. la factorielle  
    """  
    if n < 2:  
        # condition d'arrêt  
        return 1  
    else:  
        return n * factorielle(n - 1)
```

// factorielle en C

```
int factorielle(int n) {  
    /* doc de la fonction :  
    Prend un nombre et renvoie n!  
  
    Args:  
        n (int): le nombre  
        dont on veut la factorielle.  
  
    Returns:  
        int. la factorielle  
    */  
    if (n < 2) {  
        // condition d'arrêt  
        return 1;  
    } else {  
        return n * factorielle(n - 1);  
    }  
}
```


Analyse de la syntaxe

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ séparation par l'indentation
- ▶ pas de séparateur de fin de ligne (juste retour chariot)
- ▶ typage dynamique (pas de déclaration des types)
- ▶ mots clefs réservés par le langage

Analyse de la syntaxe

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

Syntaxe

[Types standards](#)

[Gestion des variables](#)

[Structures de données](#)

[Duck typing](#)

[Slicing](#)

[Gestion des fichiers](#)

[Encodage des caractères](#)

[Contrôle de flux](#)

[Fonctions](#)

[Générateurs / Itérateurs](#)

[Exceptions](#)

[Introspection](#)

[Bibliographie](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

- ▶ séparation par l'indentation
- ▶ pas de séparateur de fin de ligne (juste retour chariot)
- ▶ typage dynamique (pas de déclaration des types)
- ▶ mots clefs réservés par le langage



Ne jamais mélanger espaces et tabulation dans un fichier.

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2- Langage Python

2.2. Types standards

Types numériques

- ▶ entier (aussi grand que la RAM le peut)
- ▶ flottants
- ▶ type décimal
- ▶ type complexe

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Types numériques

Matthieu Falce

- ▶ entier (aussi grand que la RAM le peut)
- ▶ flottants
- ▶ type décimal
- ▶ type complexe

```
a = 2 * 2 + 3  
print(a)
```

```
# http://mortada.net/can-integer-operations-overflow-in-python.html  
# https://stackoverflow.com/questions/4581842/python-integer-ranges  
a = 2 ** 32 ** 2  
print(a) # pas d'overflow sur les grands ints
```

```
a = 23134/2  
print(a, type(a))
```

```
a = 2**3 + 1  
print(bin(a)) # avoir la représentation sous forme binaire
```

```
c = complex(0, -1)  
print(c)
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ divisions flottantes par défaut
- ▶ ordre des opérateurs mathématiques

- ▶ divisions flottantes par défaut
- ▶ ordre des opérateurs mathématiques

```
import math
import cmath

print("Priorité des opérations")
un = (2 * (3 + 1) - 1) / 7
print(un)

print("calcul sur les nombres réels")
pi_sur_deux = math.pi / 2
print(math.cos(pi_sur_deux))

print("calcul sur les complexes")
c = complex(0, -1)
print(cmath.exp(c * math.pi))
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

On peut manipuler facilement les chaînes :

```
print("Concaténation : ")
debut = "il était"
fin = "une fois"
print(debut + fin)

try:
    print("Attention au typage : ")
    print(debut + 1)
except Exception as e:
    print(e)

print("Fonctions de formatage")
i = 10
print("il y a {} éléments".format(i))
print(f"il y a {i} éléments") # fstring ; python >= 3.6
```

[Vue d'ensemble](#)[Langage Python](#)[Syntaxe](#)[Types standards](#)[Gestion des variables](#)[Structures de données](#)[Duck typing](#)[Slicing](#)[Gestion des fichiers](#)[Encodage des caractères](#)[Contrôle de flux](#)[Fonctions](#)[Générateurs / Itérateurs](#)[Exceptions](#)[Introspection](#)[Bibliographie](#)[Programmation
Orientée objet
\(POO\)](#)[Bonnes pratiques](#)[Bibliothèque
standard](#)[Création de
modules](#)[Programmation
concurrente](#)[Python scientifique](#)

Chaînes – performances

Matthieu Falce

Concaténation des chaînes $\neq$ rapide :

```
print("Attention pour les performances")
print("Les chaines sont immutables")
a = ""
print(id(a))
a += "Autre chose"
print(id(a))
a += "Encore autre chose"
print(id(a))
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Chaînes – contenu spécial

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Syntaxe](#)

[Types standards](#)

[Gestion des variables](#)

[Structures de données](#)

[Duck typing](#)

[Slicing](#)

[Gestion des fichiers](#)

[Encodage des caractères](#)

[Contrôle de flux](#)

[Fonctions](#)

[Générateurs / Itérateurs](#)

[Exceptions](#)

[Introspection](#)

[Bibliographie](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Problème d'échappement

"\" pour échapper un caractère spécial

Chemin de fichier windows => C:\Foo\Bar\Baz

```
print("C:\\F00\\Bar\\Baz")
```

raw strings (un seul \)

```
print(r"C:\Foo\Bar\Baz\ ")
```

Les chaînes en Python 3 sont unicodes

```
print("éàùμ")
```

Les conteneurs permettent de regrouper plusieurs valeurs

```
# différents types de conteneurs
```

```
# ajout d'un élément
```

```
liste = [1, 2, 3]  
liste.append(4)
```

```
humanize = {  
    0: "zero",  
    1: "un",  
}  
humanize[2] = "deux"
```

```
# un tuple bloque la modification
```

```
# du conteneur après sa création
```

```
immutable = tuple(liste)
```

```
# il ne peut pas y avoir de
```

```
# duplication dans les set
```

```
pas_elements_double = set([1, 2, 3])  
pas_elements_double.add(1)
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Les conteneurs permettent de regrouper plusieurs valeurs

- ▶ les conteneurs n'ont pas de contraintes de type des objets contenus
- ▶ les conteneurs peuvent avoir une taille infinie
- ▶ chaque type a des propriétés et des complexités (algorithmique) spécifiques
- ▶ les conteneurs sont itérables

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Les conteneurs permettent de regrouper plusieurs valeurs

```
# récupération d'un élément
liste = [1, 2, 3]
print(liste[0])
print(len(liste))

try:
    print(liste[10])
except Exception as e:
    # les conteneurs sont protégés contre
    # les dépassements mémoire
    print(e)

humanize = {
    0: "zero",
    1: "un",
}
print(humanize[0])

# il ne peut pas y avoir de duplication dans les set
ensemble = set([1, 2, 3])
print(1 in ensemble)
print("non" in ensemble)
print(len(ensemble))
```

[Vue d'ensemble](#)[Langage Python](#)[Syntaxe](#)[Types standards](#)[Gestion des variables](#)[Structures de données](#)[Duck typing](#)[Slicing](#)[Gestion des fichiers](#)[Encodage des caractères](#)[Contrôle de flux](#)[Fonctions](#)[Générateurs / Itérateurs](#)[Exceptions](#)[Introspection](#)[Bibliographie](#)[Programmation
Orientée objet
\(POO\)](#)[Bonnes pratiques](#)[Bibliothèque
standard](#)[Création de
modules](#)[Programmation
concurrente](#)[Python scientifique](#)

Les conteneurs permettent de regrouper plusieurs valeurs

```
ensemble = set([1, 2, 3])

try:
    ensemble[2]
except Exception as e:
    # les ensembles ne sont pas ordonnés
    print(e)

humanize = {
    0: "zero",
    1: "un",
}

# on peut récupérer les éléments d'un dictionnaire
print(list(humanize.items()))
print(list(humanize.keys()))
print(len(humanize))

# on peut avoir des valeurs par défaut pour les dico
print(humanize.get("absent", "valeur par défaut"))

# les tests d'inclusions sont rapides
print(0 in humanize)
print("absent" in humanize)
```

[Vue d'ensemble](#)[Langage Python](#)[Syntaxe](#)[Types standards](#)[Gestion des variables](#)[Structures de données](#)[Duck typing](#)[Slicing](#)[Gestion des fichiers](#)[Encodage des caractères](#)[Contrôle de flux](#)[Fonctions](#)[Générateurs / Itérateurs](#)[Exceptions](#)[Introspection](#)[Bibliographie](#)[Programmation
Orientée objet
\(POO\)](#)[Bonnes pratiques](#)[Bibliothèque
standard](#)[Création de
modules](#)[Programmation
concurrente](#)[Python scientifique](#)

Chaînes comme conteneurs

Matthieu Falce

```
print("Transformer un iterable en chaine :")
elements = (1, 2, 3)
print("-".join([str(i) for i in elements]))

print("Transformer une chaine en itérable :")
chaine = "Il était \n une fois"
print(chaine.split("\n"))

print("Les chaines sont des conteneurs que l'on peut slicer :")
ma_chaine = "Il était une fois"
print(ma_chaine[5:10])
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Trouver le type d'une variable

```
a = "une variable"  
print(a, type(a))  
# une variable <class 'str'>
```

```
a = 1  
print(a, type(a))  
# 1 <class 'int'>
```

```
b = 1.1  
print(b, type(b))  
# 1.1 <class 'float'>  
  
print(a == b, type(a == b))  
# False <class 'bool'>
```

```
c = complex(1, 1)  
print(c)  
# (1+4j)
```

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2- Langage Python

2.3. Gestion des variables

Passage par référence

Matthieu Falce



Python fait le maximum pour abstraire la gestion de mémoire.

Tous les passages se font par référence. Mais certains types sont mutables et pas d'autres.

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Mutabilité

*# un entier est un type primitif
on a le vrai objet*

```
a = 2  
b = a  
print(a, b)  
# 2, 2
```

```
a = 3  
print(a, b)  
# 3, 2
```

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Mutabilité

*# Quand on utilise des conteneurs, on manipule
une référence vers l'objet (+/- un pointeur)*

```
a = [1]
b = a
print(a, b)
# [1] [1]
```

```
a[0] = 3
print(a, b)
# [3] [3]
```

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Mutabilité

- ▶ types immutables
 - ▶ tuple
 - ▶ string
 - ▶ int / float
 - ▶ None
- ▶ types mutables
 - ▶ list
 - ▶ dict
 - ▶ set
 - ▶ types personnels
 - ▶ ...

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Construction des conteneurs

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

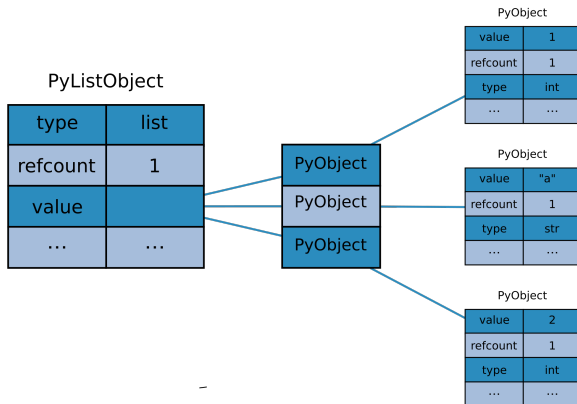
Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Structure mémoire d'une liste



Pour les classes

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

```
class Exemple():
    a = [1, 2]

exemple1 = Exemple()
exemple2 = Exemple()

print(exemple1.a, exemple2.a) # [1, 2] [1, 2]
print(exemple1.a is exemple2.a) # True

exemple1.a.pop()
print(exemple1.a, exemple2.a) # [1] [1]
print(exemple1.a is exemple2.a) # True

exemple1.a = [10]
print(exemple1.a, exemple2.a) # [10] [1]
print(exemple1.a is exemple2.a) # False
# a est devenu un attribut et non plus une variable de classe
```

Copier une variable

```
import copy

a = [1, 2]
b = a[:]
print(a is b) # False

a = [1, 2]
b = copy.copy(a)
print(a is b) # False

a = [[1, 2], [3, 4]]
b = copy.copy(a)
print(a[0] is b[0]) # True

c = copy.deepcopy(a)
print(a[0] is c[0]) # False
```

[Vue d'ensemble](#)

[Langage Python](#)

[Syntaxe](#)

[Types standards](#)

[Gestion des variables](#)

[Structures de données](#)

[Duck typing](#)

[Slicing](#)

[Gestion des fichiers](#)

[Encodage des caractères](#)

[Contrôle de flux](#)

[Fonctions](#)

[Générateurs / Itérateurs](#)

[Exceptions](#)

[Introspection](#)

[Bibliographie](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Il y a un *garbage collector* qui s'occupe de supprimer les variables inutilisées.

Il compte les références vers une variable.
Quand il n'y en a plus, il la supprime.

Voilà comment supprimer une référence.

```
a = [1, 2]  
b = a
```

```
del a  
print(b) # [1, 2]  
del b # plus de références
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2- Langage Python

2.4. Structures de données

Structures de données

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Python permet de faire beaucoup avec les structures de données de sa bibliothèque standard.

- ▶ list
- ▶ set
- ▶ dict
- ▶ tuple

Piles et files

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ comment implémenter une pile (*stack*)
- ▶ comment implémenter un file (*queue*)

- ▶ comment implémenter une pile (*stack*)
 - ▶ `list.pop`
 - ▶ `list.append`
- ▶ comment implémenter un file (*queue*)
 - ▶ `list.pop(0)`
 - ▶ `list.append`

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation Orientée objet (POO)

Bonnes pratiques

Bibliothèque standard

Création de modules

Programmation concurrente

Python scientifique

- ▶ comment implémenter une pile (*stack*)
 - ▶ `list.pop`
 - ▶ `list.append`
- ▶ comment implémenter un file (*queue*)
 - ▶ `list.pop(0)`
 - ▶ `list.append`
 - ▶ ou bien utiliser `collection.dequeue`

Les arbres peuvent se construire (entre autres) avec des dictionnaires et des listes

```
noise_ontology = {  
    "Mammalia": {  
        "Carnivora": {  
            "Canidae": {  
                "Canis": {  
                    "dog": "waf"  
                }  
            },  
            "Felidae": {  
                "Felis": {  
                    "cat": "miaou"  
                }  
            }  
        }  
    }  
}  
  
print(taxonomy['Mammalia']['Carnivora']['Felidae']['Felis']['cat'])
```

[Vue d'ensemble](#)

[Langage Python](#)

[Syntaxe](#)

[Types standards](#)

[Gestion des variables](#)

Structures de données

[Duck typing](#)

[Slicing](#)

[Gestion des fichiers](#)

[Encodage des caractères](#)

[Contrôle de flux](#)

[Fonctions](#)

[Générateurs / Itérateurs](#)

[Exceptions](#)

[Introspection](#)

[Bibliographie](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Les arbres peuvent se construire (entre autres) avec des dictionnaires et des listes

source <https://gist.github.com/hrldcpr/2012250>

```
import pprint
from collections import defaultdict

def tree():
    return defaultdict(tree)

def tree_to_dicts(t):
    return {k: tree_to_dicts(t[k]) if isinstance(t[k], defaultdict) else t[k]
            for k in t}

# exemple d'utilisation
taxonomy = tree()
taxonomy['Chordata']['Mammalia']['Carnivora']['Felidae']['Felis']['cat'] = "miaou"
taxonomy['Chordata']['Mammalia']['Carnivora']['Canidae']['Canis']['dog'] = "waf"

pprint.pprint(tree_to_dicts(taxonomy))
# {'Chordata': {'Mammalia': {'Carnivora': {'Canidae': {'Canis': {'dog': 'waf'}},
#                                           'Felidae': {'Felis': {'cat': 'miaou'}}}}}
#
```

[Vue d'ensemble](#)[Langage Python](#)[Syntaxe](#)[Types standards](#)[Gestion des variables](#)[Structures de données](#)[Duck typing](#)[Slicing](#)[Gestion des fichiers](#)[Encodage des caractères](#)[Contrôle de flux](#)[Fonctions](#)[Générateurs / Itérateurs](#)[Exceptions](#)[Introspection](#)[Bibliographie](#)[Programmation
Orientée objet
\(POO\)](#)[Bonnes pratiques](#)[Bibliothèque
standard](#)[Création de
modules](#)[Programmation
concurrente](#)[Python scientifique](#)

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2- Langage Python

2.5. Duck typing

Le *duck typing* ?

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Si ça ressemble à un canard, si ça nage comme un canard et si ça cancanne comme un canard, c'est qu'il s'agit sans doute d'un canard.

Le test du canard

Le duck typing ?

Matthieu Falce

A pythonic programming style which determines an object's type by inspection of its method or attribute signature rather than by explicit relationship to some type object ("If it looks like a duck and quacks like a duck, it must be a duck.").

By textitazing interfaces rather than specific types, well-designed code improves its flexibility by allowing polymorphic substitution. Duck-typing avoids tests using `type()` or `isinstance()`. (Note, however, that duck-typing can be complemented with abstract base classes.) Instead, it typically employs `hasattr()` tests or EAFP programming.

<https://docs.python.org/3.0/glossary.html>

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Le duck typing ?

Matthieu Falce

Les objets sont contraints selon leur comportement et pas leur type.

- ▶ déterminé à l'exécution plutôt qu'à la compilation
- ▶ l'objet doit posséder une certaine méthode
- ▶ cela rend les paramètres plus génériques
- ▶ on s'intéresse à ce que l'objet peut faire plutôt qu'à ce qu'il est

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Exemple

Matthieu Falce

```
def prend_premier(conteneur):  
    return conteneur[0]
```

```
def prend_premier_2(iterable):  
    for element in iterable:  
        return element
```

```
print(prend_premier([1, 2]))  
print(prend_premier((1, 2)))  
print(prend_premier(open("/etc/hosts"))) # TypeError
```

```
print(prend_premier_2([1, 2]))  
print(prend_premier_2((1, 2)))  
print(prend_premier_2(open("/etc/hosts")))
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Il est classique en python d'utiliser le *duck typing* pour définir des paramètres.

- ▶ `iterable` : on peut appliquer une boucle `for`
- ▶ `callable` : on peut utiliser `x()` dessus
- ▶ `hashable` : peut être passé à la fonction `hash`
- ▶ `indexable` : on peut récupérer un élément précis
- ▶ `slicable` : on peut appliquer une slice
- ▶ ...

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2- Langage Python

2.6. Slicing

Slicing

```
a = [x for x in range(100)]
print(a[30:50])
print(a[30:])
print(a[:30])
print(a[1000:2200])

# extended slices
print(a[30:50:10])
print(a[30:50:-1])
print(a[:50:-1])
print(a[30::-1])
print(a[::-1])

# remplacement
a[2:5] = [0, 0, 0, 0]
a[::10] = [0, 0, 0, 0, 0, 0, 0, 0, 0] # ValueError
```

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Explication des slices

Slicing avec un seul bord

index	0	1	2	3	4	5	6	7
valeur	12	18	15	20	28	10	6	5
	liste[:5]					liste[5:]		

Slicing avec index négatif

index	0	1	2	3	4	5	6	7
valeur	12	18	15	20	28	10	6	5
	liste[2:-3]							

Slicing avec pas

index	0	1	2	3	4	5	6	7
valeur	12	18	15	20	28	10	6	5
	liste[:2]			liste[1:2]				

Objet slice

Matthieu Falce

```
a = [x for x in range(10)]

dix_premiers = slice(0, 10)
print(type(dix_premiers))

print(a[dix_premiers])
print(dix_premiers.start)
print(dix_premiers.stop)
print(dix_premiers.step)

index_pairs = slice(0, None, 2)
print(a[index_pairs])

slice_inverse = slice(None, None, -1)
print(a[slice_inverse])
```

Utile pour conserver les valeurs de début, fin et pas dans un objet précis.

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2- Langage Python

2.7. Gestion des fichiers

Lecture de fichiers

Matthieu Falce

```
# lecture fichier texte
# par défaut "lecture en mode texte"

## chemin absolu
f_text = open("/tmp/text.txt")

## chemin relatif
f_text = open("../text.txt")

## qu'est-ce que c'est que f_text
# f_text
# <_io.TextIOWrapper name='/tmp/text.txt' mode='r' encoding='UTF-8'>
# c'est une sorte de générateur

text = f_text.read()
text = f_text.read() # texte est vide

# pour lire ligne par ligne
lines = f_text.readlines()
## ou bien
for line in f_text: # équivalent à "in f_text.readline()"
    print(line)
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Lecture de fichiers

Matthieu Falce

```
# lecture binaire
```

```
f_data = open("/tmp/image.png", "rb")
```

```
## si on lit en mode texte
```

```
# f_data = open("/tmp/image.png")
```

```
# f_data.read()
```

```
# UnicodeDecodeError: 'utf-8' codec can't decode byte 0x89
```

```
# in position 0: invalid start byte
```

```
# en binaire les fichiers contiennent des bytes strings
```

```
magic_number = b'\x89\x50\x4E\x47\x0D\x0A'
```

```
(magic_number in f_data) is True
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Ecriture de fichiers

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

```
# ATTENTION : l'écriture d'un fichier l'efface
```

```
# on peut écrire toute une chaîne de caractères
```

```
f = open("/tmp/text.txt", "w")  
f.write("Oh le joli\nmoustique")  
f.close()
```

```
# ou donner une liste de lignes
```

```
f = open("/tmp/text2.txt", "w")  
f.writelines(["Oh le joli\n", "moustique.\n\n"])  
f.close()
```

```
# on peut rajouter des éléments à la suite d'un
```

```
# fichier en l'ouvrant différemment
```

```
f = open("/tmp/text2.txt", "a")  
f.writelines(["Il fait du bruit près de mon oreille\n"])  
f.close()
```

```
# attention le fichier n'est écrit qu'après l'appel de "flush" ou "close"
```

Context Manager – gestionnaire de contexte

Matthieu Falce

```
# plutot que de fermer explicitement les fichiers,  
# on peut dire qu'ils appartiennent à une partie du code particulière
```

```
with open("/tmp/texte.txt") as f_text:  
    for line in f_text:  
        print(line)  
assert f_text.closed is True
```

```
# on peut aussi ouvrir plusieurs fichiers  
with open("./text.txt") as f_rel, open("/tmp/texte.txt") as f_abs:  
    print(f_rel.readlines())  
    print(f_abs.readlines())
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Context Manager – gestionnaire de contexte

Matthieu Falce

```
# plutôt que de fermer explicitement les fichiers,  
# on peut dire qu'ils appartiennent à une partie du code particulière
```

```
with open("/tmp/texte.txt") as f_text:  
    for line in f_text:  
        print(line)  
assert f_text.closed is True
```

```
# on peut aussi ouvrir plusieurs fichiers  
with open("./text.txt") as f_rel, open("/tmp/texte.txt") as f_abs:  
    print(f_rel.readlines())  
    print(f_abs.readlines())
```

Les gestionnaires de contexte sont bien plus génériques que ça. Ils facilitent la gestion de ressources et plus encore.

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2- Langage Python

2.8. Encodage des caractères

Encodage des caractères

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vérifiez toujours l'encodage de vos entrées / sorties.
Spécifiez les si besoin.

```
import sys, locale

# essai réalisé sous windows
print(locale.getpreferredencoding(), sys.getdefaultencoding())
# cp1252, utf-8
print(sys.stdout.encoding, sys.stdin.encoding)
# utf-8, utf-8

# phrases_magic_8_ball est un fichier texte, encodé en UTF8
# il contient des guillemets anglais « » qui ne sont pas
# ascii

# on lit le fichier en mode binaire, nous renvoie un bytearray
a = open("./phrases_magic_8_ball.txt", "rb").read()
print(a.decode("utf8"))
# « Essaye plus tard »
# « Pas d'avis »
# ...

# on lit le fichier en précisant l'encodage, nous renvoie de l'unicode
print(open("phrases_magic_8_ball.txt", encoding="utf8").read())
# ...
# « C'est non »
# « Peu probable »
# ...
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2- Langage Python

2.9. Contrôle de flux

Boucles

Matthieu Falce

on peut itérer sur un conteneur

```
ages = [5, 19, 30]
```

```
for age in ages:
    print(age)
```

```
noms = {"tuple": (), "liste": []}
```

```
for nom in noms:
    print(nom, noms[nom])
```

on peut créer des "listes" de nombre

```
for i in range(10):
    print(i)
```

il y a aussi while

```
i = 0
```

```
while i != 10:
    i += 1
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Boucles – contrôles

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

On peut contrôler une boucle avec :

- ▶ `break` : sortir de la boucle
- ▶ `continue` : passer à l'élément suivant

Boucles – “pythonique et non pythonique”

Matthieu Falce



Python a une approche particulière des itérations.
Il *faut* itérer sur les conteneurs et pas les index.

```
# OUI :o)
```

```
elements = [3, 2, 40, 10]
```

```
for element in elements:  
    print(element)
```

```
# NON :'(
```

```
elements = [3, 2, 40, 10]
```

```
for index in range(len(elements)):  
    print(elements[index])
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Tuple unpacking

Matthieu Falce

On peut déconstruire des tuples à la volée.

```
premier, deuxieme, *autres, avant_dernier, dernier = range(10)
print("premier", premier)
print("deuxieme", deuxieme)
print("autres", autres)
print("avant_dernier", avant_dernier)
print("dernier", dernier)
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

* en compréhension

Matthieu Falce

On peut construire / manipuler des itérables à la volée

On appelle ça les listes en compréhension ('list comprehension') ou 'dictionary comprehension' selon ce que l'on fait.

```
pts = [1, 2, 10, 103]
carres = [p**2 for p in pts]

nbs = range(100)
somme_des_carres_paires = sum(nb**2 for nb in nbs if nb % 2 == 0)

# marche aussi avec les dictionnaires
noms = ["un", "deux", "trois"]
elements = [1, 2, 3]
humanize = {e: n for e, n in zip(elements, noms)}
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Tests et conditions – syntaxe

Matthieu Falce

On utilise if, elif, else pour tester une variable

```
a = 3
```

```
if a == 1:
    print("ah")
elif a == 2:
    print("je le savais")
else:
    print(":'(")
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Tests et conditions – booléens

Matthieu Falce

On peut convertir (*caster*) quasiment tous les types en booléens :

```
# les variables ont des évaluations booléennes logiques
a_evaluer = ["salut", [], {}, [], "", 0, (), [[]], None, 50]
bools = [bool(element) for element in a_evaluer]

# les évaluations booléennes (et, ou...) sont paresseuses
et = False and 1 / 0
ou = True or 1 / 0
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Paresse et générateurs

Matthieu Falce

```
# instantannée (évaluation paresseuse)
gen = (i for i in range(100000) if i % 2 == 0)

# plus "long" + utilisation mémoire car provoque l'évaluation
b = list(gen)
b = list(gen) # vide car le générateur est déjà parcouru
print(b)

# on peut chaîner les générateurs :
elements = range(100000)
divisible_par_1000 = (e for e in elements if e % 1000 == 0)
multiple_de_43 = (e for e in divisible_par_1000 if e % 43 == 0)
carre = (x ** 2 for x in multiple_de_43)
somme = sum(carre)

# range ne crée pas de liste
# et est plus malin que ce que l'on croit
gros_range = range(20000, int(2e10), 10)
23000 in gros_range
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Le besoin

```
def f1():
    foo = open("/tmp/foo", "w")
    try:
        foo.write('Salut !')
    finally:
        foo.close()

#####

import threading

def f2():
    lock = threading.Lock()
    lock.acquire()
    my_list = []
    try:
        my_list.append(1)
    finally:
        lock.release()
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Le besoin

```
def f1():
    with open("/tmp/foo", "w") as f:
        f.write("Salut !")

#####

import threading

def f2():
    lock = threading.Lock()
    my_list = []
    with lock:
        my_list.append(1)
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Gestionnaires de contexte

Matthieu Falce

La solution

```
class MonGestionnaire():  
    def __enter__(self):  
        print("entrée dans le bloc")  
  
    def __exit__(self, type, value, traceback):  
        print("sortie du bloc")  
  
with MonGestionnaire():  
    print("dans le block")
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Le mot clé as

```
class ListeVide():
    def __enter__(self):
        self.ma_liste = []
        return self.ma_liste

    def __exit__(self, type, value, traceback):
        print("Nb éléments : {}".format(len(self.ma_liste)))

with ListeVide() as l:
    l.append(1)
    l.append(2)
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Utiliser plusieurs gestionnaires en même temps

```
with open("/tmp/t1.txt", "w") as f, open("/tmp/t2.txt", "w") as g:  
    f.write("f - t1")  
    g.write("g - t2")
```


Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

2- Langage Python

2.10. Fonctions

Déclaration d'une fonction

Matthieu Falce

```
def ma_fonction(param1):  
    param1 * 2  
  
def autre_fonction(param1):  
    return param1 * 2  
  
# Les fonctions renvoient toujours quelque chose.  
# Si pas de return, elles renvoient "None"  
a = ma_fonction(1)  
print(a)  
  
b = autre_fonction(2)  
print(b)  
  
# Une fonction peut renvoyer plusieurs valeurs,  
# de plusieurs types différents  
def exemple_return():  
    return None, [1, 2, 3]  
  
a = exemple_return()  
print(a)
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Déclaration d'une fonction

Matthieu Falce

```
def exemple_defaults(param1, param2=None):  
    """Une fonction peut accepter des paramètres  
    nommés et des paramètres par défaut"""  
    print(param1, param2)
```

```
exemple_defaults(1) # 1, None  
exemple_defaults(1, 2) # 1, 2  
exemple_defaults(1, param2=32) # 1, 32
```

```
def exemple_arg_kwargs(param1, *args, **kwargs):  
    """Une fonction peut accepter un nombre dynamique  
    de paramètres anonymes et nommés.  
    Souvent utilisés par les API de bibliothèques.  
    Ou quand on ne connaît pas le nombre d'éléments à priori  
    """  
    print("obligatoire", param1)  
    print("liste d'autres arguments anonymes", args)  
    print("dict des autres arguments nommés", kwargs)
```

```
exemple_arg_kwargs(1)  
exemple_arg_kwargs(1, 2)  
exemple_arg_kwargs(1, 2, param3=3)
```

Vue d'ensemble

Langage Python

- Syntaxe
- Types standards
- Gestion des variables
- Structures de données
- Duck typing
- Slicing
- Gestion des fichiers
- Encodage des caractères
- Contrôle de flux

Fonctions

- Gestion des arguments
- Gotchas
- Higher order functions
- Closures
- Décorateurs
- Générateurs / Itérateurs
- Exceptions
- Introspection
- Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Déclaration d'une fonction

Matthieu Falce

Ces trois codes sont globalement équivalents

```
# fonction classique
```

```
def addition(x, y):
```

```
    return x+y
```

```
addition(2, 3)
```

```
# lambda
```

```
addition = lambda x, y: x+y
```

```
addition(2, 3)
```

```
# fonction anonyme
```

```
(lambda x, y: x+y)(2, 3)
```

[Vue d'ensemble](#)

[Langage Python](#)

[Syntaxe](#)

[Types standards](#)

[Gestion des variables](#)

[Structures de données](#)

[Duck typing](#)

[Slicing](#)

[Gestion des fichiers](#)

[Encodage des caractères](#)

[Contrôle de flux](#)

Fonctions

[Gestion des arguments](#)

[Gotchas](#)

[Higher order functions](#)

[Closures](#)

[Décorateurs](#)

[Générateurs / Itérateurs](#)

[Exceptions](#)

[Introspection](#)

[Bibliographie](#)

[Programmation](#)

[Orientée objet](#)

[\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

Arguments des fonctions

Matthieu Falce

A votre avis, que donnent les fonctions suivantes ?

```
def f(a, b="default"):  
    print("a", a)  
    print("b", b)  
    print("-----")
```

```
f(1)  
f(1, 2)  
f(1, 2, 3)  
f([1, 2], (3, 4))
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation

Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Arguments des fonctions

Matthieu Falce

A votre avis, que donnent les fonctions suivantes ?

```
def g(a, b, *args):  
    print("a", a)  
    print("b", b)  
    print("args", args)  
    print("-----")
```

```
g(1, 2)
```

```
g(1, 2, 3)
```

```
## opérateur splat
```

```
liste_exemple = [1, 2, 3, 4, 5]
```

```
g(liste_exemple)
```

```
g(*liste_exemple)
```

[Vue d'ensemble](#)

[Langage Python](#)

[Syntaxe](#)

[Types standards](#)

[Gestion des variables](#)

[Structures de données](#)

[Duck typing](#)

[Slicing](#)

[Gestion des fichiers](#)

[Encodage des caractères](#)

[Contrôle de flux](#)

[Fonctions](#)

[Gestion des arguments](#)

[Gotchas](#)

[Higher order functions](#)

[Closures](#)

[Décorateurs](#)

[Générateurs / Itérateurs](#)

[Exceptions](#)

[Introspection](#)

[Bibliographie](#)

[Programmation](#)

[Orientée objet](#)

[\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

Arguments des fonctions

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Syntaxe](#)

[Types standards](#)

[Gestion des variables](#)

[Structures de données](#)

[Duck typing](#)

[Slicing](#)

[Gestion des fichiers](#)

[Encodage des caractères](#)

[Contrôle de flux](#)

[Fonctions](#)

[Gestion des arguments](#)

[Gotchas](#)

[Higher order functions](#)

[Closures](#)

[Décorateurs](#)

[Générateurs / Itérateurs](#)

[Exceptions](#)

[Introspection](#)

[Bibliographie](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

A votre avis, que donnent les fonctions suivantes ?

```
def f(a, b="default"):  
    print("a", a)  
    print("b", b)  
    print("-----")
```

f(1)

f(1, 2)

f(1, b=2)

f(1, c=2)

Arguments des fonctions

Matthieu Falce

A votre avis, que donnent les fonctions suivantes ?

```
def g(a, b, **kwargs):  
    print("a", a)  
    print("b", b)  
    print("kwargs", kwargs)  
    print("-----")  
  
g(1, 2)  
g(1, 2, c=(3, 4))  
g(1, c=3)  
  
## opérateur double splat  
dico_exemple = {"a": 1, "b": 2, "c": 3, "d": 4}  
g(dico_exemple)  
g(**dico_exemple)
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Arguments des fonctions

Matthieu Falce

A votre avis, que donnent les fonctions suivantes ?

```
def f(a, b="default", *args, **kwargs):  
    print("a", a)  
    print("b", b)  
    print("args", args)  
    print("kwargs", kwargs)  
    print("-----")
```

f(1)

f(1, 2)

f(1, b=2)

f(1, 2, 3, b=4, c=5)

f(1, *["c", 3, 4], **{"d": 5, "e": 6})

[Vue d'ensemble](#)

[Langage Python](#)

[Syntaxe](#)

[Types standards](#)

[Gestion des variables](#)

[Structures de données](#)

[Duck typing](#)

[Slicing](#)

[Gestion des fichiers](#)

[Encodage des caractères](#)

[Contrôle de flux](#)

[Fonctions](#)

[Gestion des arguments](#)

[Gotchas](#)

[Higher order functions](#)

[Closures](#)

[Décorateurs](#)

[Générateurs / Itérateurs](#)

[Exceptions](#)

[Introspection](#)

[Bibliographie](#)

[Programmation](#)

[Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

Arguments des fonctions

Matthieu Falce

A votre avis, que donnent les fonctions suivantes ?

```
def f(a, *, b="default"):  
    print("a", a)  
    print("b", b)  
    print("-----")
```

```
f(1)  
f(1, 2)  
f(1, b=2)  
f(1, c=2)
```

[Vue d'ensemble](#)

[Langage Python](#)

[Syntaxe](#)

[Types standards](#)

[Gestion des variables](#)

[Structures de données](#)

[Duck typing](#)

[Slicing](#)

[Gestion des fichiers](#)

[Encodage des caractères](#)

[Contrôle de flux](#)

[Fonctions](#)

[Gestion des arguments](#)

[Gotchas](#)

[Higher order functions](#)

[Closures](#)

[Décorateurs](#)

[Générateurs / Itérateurs](#)

[Exceptions](#)

[Introspection](#)

[Bibliographie](#)

[Programmation
Orientée objet
\(POO\)](#)

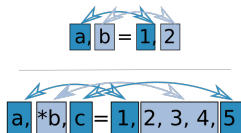
[Bonnes pratiques](#)

[Bibliothèque
standard](#)

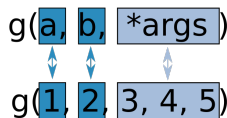
[Création de
modules](#)

Unpacking

Pour les variables



Pour les arguments



Arguments des fonctions – résumé

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

- ▶ args et kwargs sont des conventions
- ▶ * permet de *pack* / *unpack* les listes
- ▶ ** permet de *pack* / *unpack* les dictionnaires
- ▶ * / ** sont appelés opérateurs splat

Intérêts :

- ▶ `kwargs.pop` permet de gérer les valeurs de paramètres par défaut
- ▶ intérêt pour les API
 - ▶ manipulation de fonction sans connaître ses paramètres (décorateurs)
 - ▶ fonctions plus ou moins spécialisées (`matplotlib`)
 - ▶ faible couplage entre les fonctions

Limites :

- ▶ complexifie la documentation / utilisation

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Problèmes classiques – éléments mutables ^{21 22}

Matthieu Falce

```
# Attention voilà ce qu'il ne faut pas faire.  
# Ne pas mettre d'éléments mutables dans les  
# arguments par défaut
```

```
def append_wrong(value, li=[]):  
    """On s'attend à toujours avoir une liste d'un élément."""  
    li.append(value)  
    return li
```

```
a = append_wrong(1)  
b = append_wrong(2)  
print(a, b)  
# [1, 2], [1, 2]
```

```
# on peut également tester en mettant arg=time.time() pour comprendre  
# le moment de l'évaluation des paramètres
```

```
def append_correct(value, li=None):  
    """On met une valeur nulle par défaut et on regarde  
si elle est renseignée ou pas."""  
    if li is None:  
        li = []  
    li.append(value)  
    return li
```

```
a = append_correct(1)  
b = append_correct(2)
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation

Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

21.<http://docs.python-guide.org/en/latest/writing/gotchas/>

22.<http://blog.notdot.net/2009/11/Python-Gotchas>

Problèmes classiques – portée des variables

Matthieu Falce

```
variable = 1
```

```
def print_variable():  
    print(variable)
```

```
def modifie_variable():  
    variable += 1
```

```
def local_variable():  
    variable = 2  
    return variable
```

```
def modifie_variable_ok():  
    global variable  
    variable += 1
```

```
def outer():  
    variable = 1  
    def inner():  
        nonlocal variable  
        variable = 2  
  
    print("avant appel inner", variable)  
    inner()  
    print("apres appel inner", variable)
```

```
##### late binding des variables dans les fonctions
```

```
variable = 10  
print_variable()  
variable = 11  
print_variable()
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

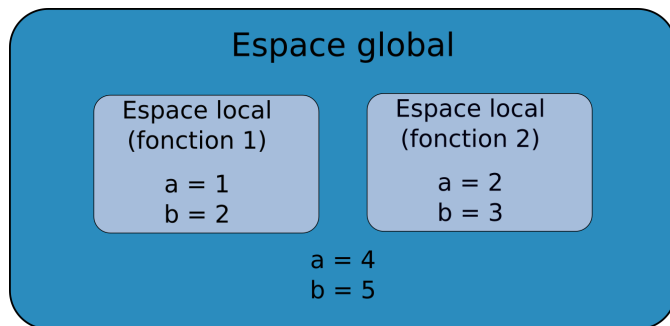
Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Espaces de noms



Fonctions d'ordre supérieur

Matthieu Falce

- ▶ les fonctions sont des variables comme les autres
- ▶ on peut les passer comme argument à d'autres fonctions
- ▶ on dit que les fonctions sont des *first class citizen*

Les fonctions d'ordre supérieur manipulent d'autres fonctions

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Fonctions d'ordre supérieur

Matthieu Falce

- ▶ les fonctions sont des variables comme les autres
- ▶ on peut les passer comme argument à d'autres fonctions
- ▶ on dit que les fonctions sont des *first class citizen*

Les fonctions d'ordre supérieur manipulent d'autres fonctions

```
# on veut trier selon la lettre
```

```
a = [(1, "d"), (2, "c"), (3, "b"), (4, "a")]
```

```
b = sorted(a, key=lambda x: x[1])
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation

Orientée objet

(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Fonctions comme variables

Matthieu Falce

```
def plus(a, b):  
    return a + b  
  
print(ma_fonction, type(ma_fonction))  
# <function ma_fonction at 0x7f97716e5620> <class 'function'>  
  
calcul = {  
    "plus": plus,  
    "moins": lambda x, y: x - y,  
    "fois": lambda x, y: x * y,  
    "divise": lambda x, y: x / y,  
}  
  
calcul["moins"](2, 1)
```

Vue d'ensemble

L langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Fonctions comme variables

Matthieu Falce

```
def plus(a, b):  
    return a + b  
  
print(ma_fonction, type(ma_fonction))  
# <function ma_fonction at 0x7f97716e5620> <class 'function'>
```

```
calcul = {  
    "plus": plus,  
    "moins": lambda x, y: x - y,  
    "fois": lambda x, y: x * y,  
    "divise": lambda x, y: x / y,  
}
```

```
calcul["moins"](2, 1)
```



Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Dans un langage de programmation, une fermeture ou clôture (en anglais : closure) est une fonction accompagnée de l'ensemble des variables non locales qu'elle a capturé.

[https://fr.wikipedia.org/wiki/Fermeture_\(informatique\)](https://fr.wikipedia.org/wiki/Fermeture_(informatique))

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Closures – Exemples

Matthieu Falce

```
# on peut déclarer des fonctions locales à d'autres fonctions.
```

```
def parler():
```

```
    # On peut définir une fonction à la volée dans "parler" ...
```

```
    def chuchoter(mot="yes"):  
        return mot.lower() + "..."
```

```
    # ... et l'utiliser immédiatement !
```

```
    print(chuchoter())
```

```
parler()
```

```
# chuchoter n'existe pas dans l'espace global
```

```
try:
```

```
    print(chuchoter())
```

```
except NameError as e:
```

```
    print(e)
```

```
    # output : "name 'chuchoter' is not defined"
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Closures – Exemples

Matthieu Falce

```
def ajoute_avec(nombre):  
    def ajouter(autre_nombre):  
        return nombre + autre_nombre  
    return ajouter
```

```
ajoute_avec_10 = ajoute_avec(10)  
print(ajoute_avec_10(5))  # 15
```

```
ajoute_avec_20 = ajoute_avec(20)  
print(ajoute_avec_20(2))  # 22
```

[Vue d'ensemble](#)

[Langage Python](#)

[Syntaxe](#)

[Types standards](#)

[Gestion des variables](#)

[Structures de données](#)

[Duck typing](#)

[Slicing](#)

[Gestion des fichiers](#)

[Encodage des caractères](#)

[Contrôle de flux](#)

[Fonctions](#)

[Gestion des arguments](#)

[Gotchas](#)

[Higher order functions](#)

[Closures](#)

[Décorateurs](#)

[Générateurs / Itérateurs](#)

[Exceptions](#)

[Introspection](#)

[Bibliographie](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Les décorateurs permettent de modifier ou d'injecter un comportement à des fonctions.

Décorateurs – Syntaxe

Matthieu Falce

La syntaxe avec le @ est un raccourci syntaxique.
Ces deux façons de faire sont identiques.

```
@decorateur  
def fonction():  
    pass
```

```
fonction = decorateur(fonction)
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation

Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Décorateurs – Syntaxe

Matthieu Falce

```
def ecrit_avant_apres(fonction_a_decorer):  
    """Cette fonction prend une fonction qu'elle va  
    decorer.  
    """  
  
    def wrapper():  
        """Cette fonction entoure l'appel de la fonction  
        d'origine."""  
        print("avant")  
        res = fonction_a_decorer()  
        print("apres")  
        return res  
  
    # on retourne la **fonction** wrapper  
    return wrapper  
  
@ecrit_avant_apres  
def test_deco_syntaxe():  
    print("dans test deco syntaxe")  
  
print(test_deco_syntaxe())
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Décorateurs – Syntaxe

Matthieu Falce

```
# comment accepter des paramètres
```

```
def ecrit_avant_apres(fonction_a_decorer):  
    """Cette fonction prend une fonction qu'elle va  
    decorer.  
    """  
  
    def wrapper(*args, **kwargs):  
        """Cette fonction entoure l'appel de la fonction  
        d'origine."""  
        print("avant")  
        res = fonction_a_decorer(*args, **kwargs)  
        print("pendant", res)  
        print("apres")  
        return res  
  
    # on retourne la **fonction** wrapper  
    return wrapper  
  
@ecrit_avant_apres  
def test_deco_syntaxe(a, b, c=0):  
    return "resultat test 2", a, b, c  
  
print(test_deco_syntaxe(1, b=2, c=3))
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Décorateurs paramétrés

Matthieu Falce

comment passer des paramètres au décorateur

```
def ecrit_avant_apres_plein_de_fois(nb_avant, nb_apres):  
    # on rajoute un niveau, une fabrique de décorateur  
    # permet d'avoir les paramètres par closure  
    def ecrit_avant_apres(fonction_a_decorer):  
        def wrapper(*args, **kwargs):  
            print("avant " * nb_avant)  
            res = fonction_a_decorer(*args, **kwargs)  
            print("après " * nb_apres)  
            return res  
        return wrapper  
    return ecrit_avant_apres
```

```
@ecrit_avant_apres_plein_de_fois(nb_avant=2, nb_apres=4)  
def f(a, b):  
    print("dans f ", a, b)
```

```
f(1, 3)  
# avant avant  
# dans f 1 3  
# après après après après
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation

Orientée objet

(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Décorateurs paramétrés

Matthieu Falce

La syntaxe avec le @ est un raccourci syntaxique.
Ces deux façons de faire sont identiques.

```
@decorateur(a, b)
def fonction():
    pass
```

```
fonction = decorateur(a, b)(fonction)
```

[Vue d'ensemble](#)

[Langage Python](#)

[Syntaxe](#)

[Types standards](#)

[Gestion des variables](#)

[Structures de données](#)

[Duck typing](#)

[Slicing](#)

[Gestion des fichiers](#)

[Encodage des caractères](#)

[Contrôle de flux](#)

[Fonctions](#)

[Gestion des arguments](#)

[Gotchas](#)

[Higher order functions](#)

[Closures](#)

Décorateurs

[Générateurs / Itérateurs](#)

[Exceptions](#)

[Introspection](#)

[Bibliographie](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

Introspection ?

Matthieu Falce

```
def inutile(fonction_a_decorer):
    """Décorateur inutile."""

    def wrapper(*args, **kwargs):
        """Fonction wrapper."""
        return fonction_a_decorer(*args, **kwargs)

    return wrapper

@inutile
def f(a):
    """Une super fonction !"""
    return "dans f"

help(f)
# Help on function wrapper in module __main__:
# wrapper(*args, **kwargs)
#     Fonction wrapper.
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Introspection ?

Matthieu Falce

```
from functools import wraps

def inutile(fonction_a_decorer):
    """Décorateur inutile."""

    @wraps(fonction_a_decorer) # on indique que wrapper entoure une fonction
    def wrapper(*args, **kwargs):
        """Fonction wrapper."""
        return fonction_a_decorer(*args, **kwargs)

    return wrapper

@inutile
def f(a):
    """Une super fonction !"""
    return "dans f"

help(f)
# Help on function f in module __main__:
# f(a)
#     Une super fonction !
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Introspection ?

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

implémentée à travers des variables magiques

- ▶ `__qualname__` : nom qualifié (chemin depuis le module) ²³
- ▶ `__name__` : nom de la fonction (pas de la variable qui la contient)
- ▶ `__doc__` : docstring de la fonction

Comment fonctionne `functools.wraps` d'après vous ?

23.<https://docs.python.org/3/glossary.html#term-qualified-name>

- ▶ étendre une fonction qu'on ne peut pas modifier
- ▶ gérer des permissions
- ▶ analyse de performances (mesure du temps passé / mémoire utilisée)
- ▶ mise en cache
- ▶ casting du résultat d'une fonction dans un type
- ▶ ...

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Gestion des arguments

Gotchas

Higher order functions

Closures

Décorateurs

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2- Langage Python

2.11. Générateurs / Itérateurs

Protocole d'itération

Matthieu Falce

```
a = [1, 2, 3]
print(a, type(a))  # [1, 2, 3] <class 'list'>

it = iter(a)
print(it)  # <list_iterator object at 0x7fa8359057b8>
print(type(it))  # <class 'list_iterator'>

print(next(it))
print(next(it))
print(next(it))
print(next(it))
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Les différents types

Matthieu Falce

```
nombre_pairs = (i for i in range(100_000_000) if i % 2 == 0)
```

```
for pair in nombre_pairs:
    print(pair)
    if pair > 10:
        break

print("fin du premier for")
```

```
# à partir de quel nombre est-ce que ça reprend ?
```

```
for pair in nombre_pairs:
    print(pair)
    if pair > 20:
        break
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Les différents types

Matthieu Falce

```
def generateur(nb_elements):
    print("début générateur")
    for i in range(nb_elements):
        yield i * 50
    print("fin générateur")

def generateur_2(nb_elements):
    """On peut utiliser de la délégation de générateurs avec yield from."""
    yield from (i*10 for i in range(nb_elements))

gen = generateur(4)

# première lecture
for element in gen:
    print(element)

print("- - - - -")

# deuxième lecture
for element in gen: # ne rentre pas
    print(element)

next(gen) # raise StopIteration
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Les différents types

Matthieu Falce

```
class Iterateur():
    def __init__(self, nb_elements):
        self.nb_elements = nb_elements
        self.counter = 0

    def __iter__(self):
        print("dans iter")
        return self

    def __next__(self):
        print("dans next")
        if self.counter > self.nb_elements:
            raise StopIteration

        self.counter += 1
        return self.counter

it = Iterateur(5)
for i in it:
    print(i)

next(it) # StopIteration
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

A quoi ça sert ?

Matthieu Falce

- ▶ évaluation paresseuse
 - ▶ flux de données infini
 - ▶ meilleure gestion de la mémoire
- ▶ traitement asynchrone sans callbacks
 - ▶ le yield bloque l'exécution jusqu'au prochain next

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

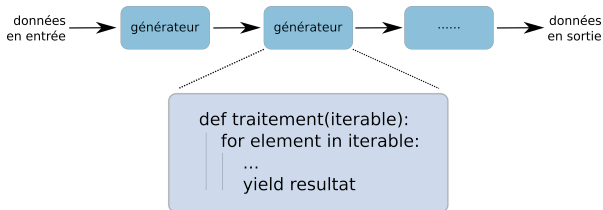
Création de
modules

Programmation
concurrente

Python scientifique

Traitement de grandes quantités de données en minimisant l'empreinte mémoire

- ▶ on empile des générateurs à la chaîne
- ▶ similaire aux pipelines Unix



Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Pipeline

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

```
def is_palindrome(nb):  
    return str(nb) == str(nb)[::-1]  
  
nombres_pairs = (i for i in range(100_000_000) if i % 2 == 0)  
palindromes = (i for i in nombres_pairs if is_palindrome(i))  
fois_cinquante = (i * 50 for i in palindromes)  
trente_premiers = (i for i, j in zip(fois_cinquante, range(30)))  
  
print(sum(trente_premiers))
```

Pipeline

Matthieu Falce

```
def palindromes(iterable):
    for element in iterable:
        s_element = str(element)
        if s_element == s_element[::-1]:
            yield element

def nombres_paires(iterable):
    for element in iterable:
        if element % 2 == 0:
            yield element

def fois_cinquante(iterable):
    for element in iterable:
        yield element * 50

def trente_premiers(iterable):
    for index, element in enumerate(iterable):
        if index > 30:
            break
        yield element

elements = range(100_000_000)
s = sum(trente_premiers(fois_cinquante(palindromes(nombres_paires(elements)))))
print(s)
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Un module de la bibliothèque standard pour manipuler des itérables.

- ▶ générateurs infinis (count, cycle, repeat)
- ▶ chaînage d'itérateurs
- ▶ prendre un certain nombre d'éléments (dropwhile, takeWhile)
- ▶ prendre certains éléments (compress, filterfalse)
- ▶ ...

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Les générateurs peuvent aussi recevoir des valeurs.



Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

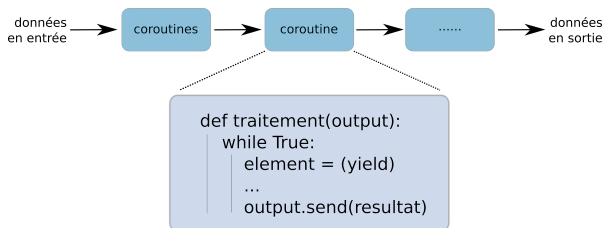
Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Les générateurs peuvent aussi recevoir des valeurs.



Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Consommateurs / Coroutines

Matthieu Falce

```
def coroutine(func):  
    """Permet d'appeler next la première fois automatiquement."""  
    def wrapper(*arg, **kwargs):  
        generator = func(*arg, **kwargs)  
        next(generator)  
        return generator  
    return wrapper
```

```
@coroutine  
def nombres_paires(output):  
    while True:  
        element = (yield)  
        if element % 2 == 0:  
            output.send(element)
```

```
@coroutine  
def fois_cinquante(output):  
    while True:  
        element = (yield)  
        output.send(element * 50)
```

```
@coroutine  
def afficher():  
    while True:  
        x = (yield)  
        print(x)  
  
aff = afficher()  
multiplier = fois_cinquante(aff)  
nb_paires = nombres_paires(multiplier)
```

```
for nombre in range(100):  
    nb_paires.send(nombre)
```

Vue d'ensemble

L langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ permet de tirer les données plutôt que de les pousser
- ▶ permet d'attendre des données / réagir à des événements
- ▶ fonctionnement inverse des consommateurs précédents
- ▶ premiers pas dans l'asynchrone

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2- Langage Python

2.12. Exceptions

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Toujours utiliser une exception précise et bien logger les erreurs.

Sinon des erreurs peuvent en cacher d'autres.

Exceptions

Matthieu Falce

```
try:
    print("peut lever une exception")
    raise AssertionError()
except AssertionError as e:
    print("    gère l'exception AssertionError")
except (IndexError, ArithmeticError) as e:
    print("    gère d'autres exceptions")
except Exception as e:
    print("    gère le reste des exceptions")
else:
    print("suite logique du code qui peut lever une exception")
    print("mais qui n'en lève pas lui-même")
finally:
    print("appelé quel que soit le parcours d'exception")
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Exceptions

Matthieu Falce

```
# Philosophie en python  
# Mieux vaut demander pardon que la permission
```

```
def utile(tableau):  
    try :  
        clef, valeur = tableau[0]  
    except IndexError as e:  
        clef, valeur = None, None  
    else:  
        valeur *= 3  
    finally:  
        return clef, valeur  
  
print(utile([]))  
print(utile([1, 2]))  
print(utile([(3, 4)]))
```

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Exceptions

```
def test1():
    try:
        return 1 + "1"
    except TypeError:
        return "exception"

def test2():
    try:
        return 1 + "1"
    except TypeError:
        return "exception"
    finally:
        return "finally"

print(test1())
print(test2())
```

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Demander pardon plutôt que la permission

Matthieu Falce

Point pythonique : capturer l'exception plutôt que tester si l'action est possible

Easier to ask for forgiveness than permission. This common Python coding style assumes the existence of valid keys or attributes and catches exceptions if the assumption proves false. This clean and fast style is characterized by the presence of many try and except statements. The technique contrasts with the **LBYL** style common to many other languages such as C.

Documentation Python

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2- Langage Python

2.13. Introspection

Introspection ?

Matthieu Falce

In everyday life, introspection is the act of self-examination. Introspection refers to the examination of one's own thoughts, feelings, motivations, and actions. The great philosopher Socrates spent much of his life in self-examination, encouraging his fellow Athenians to do the same. He even claimed that, for him, "the unexamined life is not worth living."

<https://www.ibm.com/developerworks/library/l-pyint/index.html>

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Introspection ?

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

In computer programming, introspection is the ability to determine the type of an object at runtime.

It is one of Python's strengths.

Everything in Python is an object and we can examine those objects. Python ships with a few built-in functions and modules to help us.

http://book.pythontips.com/en/latest/object_introspection.html

Comment faire ?

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ `sys` : informations sur l'interpréteur
- ▶ `dir` : méthodes / fonctions d'une classe / module
- ▶ `id` : zone mémoire d'un objet
- ▶ `type` : type d'un objet
- ▶ `hasattr` / `getattr` : modification d'une instance
- ▶ `isinstance` / `issubclass` : savoir si un objet est d'un certain type
- ▶ méthode magique (`__name__`) : quel est le nom d'un objet
- ▶ ...

Comment faire ?

Matthieu Falce

Module `inspect`²⁵ de la bibliothèque standard

- ▶ avoir des informations sur le code source (fichier / module / ligne / ...)
- ▶ inspecter les signatures des *callables*
- ▶ analyser les classes et fonctions
- ▶ déterminer l'état de l'interpréteur (*function stack*, ...)
- ▶ ...

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

²⁵<https://docs.python.org/3/library/inspect.html>

A quoi ça sert ?

- ▶ analyse des exceptions / *stacktraces*
- ▶ code dépendant du type d'un objet
- ▶ familiarisation avec un nouveau code (autocomplétion dans le shell / analyse des attributs en direct...)
- ▶ ...

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2- Langage Python

2.14. Bibliographie

► Générateurs

- <http://sametmax.com/parcourir-un-iterable-par-morceaux-en-python/>
- <https://www.pythonsheets.com/notes/python-generator.html>
- <https://brett.is/writing/about/generator-pipelines-in-python/>
- <http://xion.io/post/code/python-generator-args.html>
- <https://stackoverflow.com/questions/19302530/python-generator-send-function-purpose>
- <http://sametmax.com/quest-ce-quune-coroutine-en-python-et-a-quoi-ca-sert/>
- <http://treyhunner.com/2018/06/how-to-make-an-iterator-in-python/>
- <https://docs.python.org/3/library/itertools.html#itertools.cycle>

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Bibliographie II

Matthieu Falce

- ▶ <http://www.dabeaz.com/coroutines/Coroutines.pdf>
- ▶ <http://www.dabeaz.com/finalgenerator/FinalGenerator.pdf>
- ▶ Décorateurs
 - ▶ <http://sametmax.com/comprendre-les-decorateur-python-pas-a-pas-partie-2/>
 - ▶ <http://sametmax.com/le-pattern-observer-en-utilisant-des-decorateurs/>
 - ▶ <https://python-3-patterns-idioms-test.readthedocs.io/en/latest/PythonDecorators.html>
- ▶ Utilisation des astérisques
 - ▶ <http://treyhunner.com/2018/10/asterisks-in-python-what-they-are-and-how-to-use-them/>
- ▶ Types de données :

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Bibliographie III

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ <https://docs.python.org/fr/3/tutorial/datastructures.html>
- ▶ http://python-prepa.github.io/information_theory.html
- ▶ <https://deptinfo-ensip.univ-poitiers.fr/ENS/doku/doku.php/stu:algo2>
- ▶ <https://www.apprendre-en-ligne.net/info/structures/structures.pdf>
- ▶ Introspection :
 - ▶ http://book.pythontips.com/en/latest/object_introspection.html
 - ▶ <https://www.ibm.com/developerworks/library/l-pyint/index.html>
 - ▶ https://python.developpez.com/cours/DiveIntoPython/php/frdiveintopython/power_of_introspection/index.php
 - ▶ <https://docs.python.org/3/library/inspect.html>

Bibliographie IV

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ <http://sametmax.com/comprendre-les-decorateur-python-pas-a-pas-partie-2/>

▶ Variables :

- ▶ <http://sametmax.com/valeurs-et-references-en-python/>
- ▶ <http://sametmax.com/id-none-et-bidouilleries-memoire-en-python/>
- ▶ <https://medium.com/@tyastropheus/tricky-python-i-memory-management-for-mutable-immutable-objects-21507d1e5b95>

▶ Exceptions :

- ▶ <http://sametmax.com/gestion-des-erreurs-en-python/>
- ▶ <http://sametmax.com/comment-recruter-un-developpeur-python/>
- ▶ <http://sametmax.com/pourquoi-utiliser-un-mecanisme-dexceptions/>

Bibliographie V

Matthieu Falce

► *Context managers*

- <http://sametmax.com/les-context-managers-et-le-mot-cle-with-en-python/>
- <https://alysivji.github.io/managing-resources-with-context-managers-pythonic.html>
- <http://eigenhombre.com/introduction-to-context-managers-in-python.html>

► *Duck Typing*

- <https://stackoverflow.com/questions/4205130/what-is-duck-typing>
- <https://hackernoon.com/python-duck-typing-or-automatic-interfaces-73988ec9037f>
- https://en.wikipedia.org/wiki/Duck_typing
- <http://sametmax.com/quest-ce-que-le-duck-typing-et-a-quoi-ca-sert/>
- <http://sametmax.com/les-trucmunchables-en-python/>

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Bibliographie VI

Matthieu Falce

Vue d'ensemble

Langage Python

Syntaxe

Types standards

Gestion des variables

Structures de données

Duck typing

Slicing

Gestion des fichiers

Encodage des caractères

Contrôle de flux

Fonctions

Générateurs / Itérateurs

Exceptions

Introspection

Bibliographie

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ <https://stackoverflow.com/questions/1952464/in-python-how-do-i-determine-if-an-object-is-iterable>
- ▶ <https://stackoverflow.com/questions/6589967/how-to-handle-duck-typing-in-python>

Programmation Orientée objet (POO)

3- Programmation Orientée objet (POO)

3.1. Concepts

Programmation orientée objet (POO)

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

La POO consiste en la définition et l'interaction de briques logicielles appelées objets ; un objet représente un concept, une idée ou toute entité du monde physique, comme une voiture, une personne ou encore une page d'un livre. Il possède une structure interne et un comportement, et il sait interagir avec ses pairs.

https://fr.wikipedia.org/wiki/Programmation_orient%C3%A9e_objet

Programmation orientée objet (POO)

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Il s'agit donc de représenter ces objets et leurs relations ; l'interaction entre les objets via leurs relations permet de concevoir et réaliser les fonctionnalités attendues, de mieux résoudre le ou les problèmes. Dès lors, l'étape de modélisation revêt une importance majeure et nécessaire pour la POO. C'est elle qui permet de transcrire les éléments du réel sous forme virtuelle.

https://fr.wikipedia.org/wiki/Programmation_orient%C3%A9e_objet

Constitution d'une classe

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Une classe est constituée de 2 entités (en gros) :

- ▶ les méthodes : des "fonctions" qui s'appliquent sur un objet
- ▶ les attributs : des "variables" qui s'appliquent sur un objet

Constitution d'une classe

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Une classe est constituée de 2 entités (en gros) :

- ▶ les méthodes : des "fonctions" qui s'appliquent sur un objet
- ▶ les attributs : des "variables" qui s'appliquent sur un objet

Cela permet de conserver le *comportement* et *l'état* à l'intérieur de l'instance.

Des appels à des méthodes vont modifier l'état interne en changeant les attributs.

Constitution d'une classe

Matthieu Falce

Une classe est constituée de 2 entités (en gros) :

- ▶ les méthodes : des "fonctions" qui s'appliquent sur un objet
- ▶ les attributs : des "variables" qui s'appliquent sur un objet

Cela permet de conserver le *comportement* et *l'état* à l'intérieur de l'instance.

Des appels à des méthodes vont modifier l'état interne en changeant les attributs.

Une classe est une *boîte noire*. On interagit avec elle à l'aide de quelques leviers et boutons sans savoir ce qui se passe à l'intérieur.

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ une classe définit un nouveau *type* (comme `int`)
- ▶ un *objet* est une *instance* d'une classe (comme 2 est une instance de *int*)

3- Programmation Orientée objet (POO)

3.2. Association

Association entre classes

Matthieu Falce

2 grandes techniques pour associer des classes entre elles :

- ▶ *héritage* (*inheritance* en anglais): on étend une classe mère en faisant un nouveau type qui le restreint
 - ▶ modélise la relation "*est un*"
 - ▶ le type fille peut être utilisé à la place du type mère (*polymorphisme*)
 - ▶ on peut redéfinir ou *surcharger* certains comportements (méthodes, attributs)
 - ▶ les relations classe mère / classe fille définissent un *arbre d'héritage*

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Association entre classes

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

2 grandes techniques pour associer des classes entre elles :

- ▶ *composition* : on étend une classe en l'utilisant comme attribut d'une classe
 - ▶ modélise la relation "*possède un*"
 - ▶ assouplit la relation de dépendance

3- Programmation Orientée objet (POO)

3.3. Modélisation

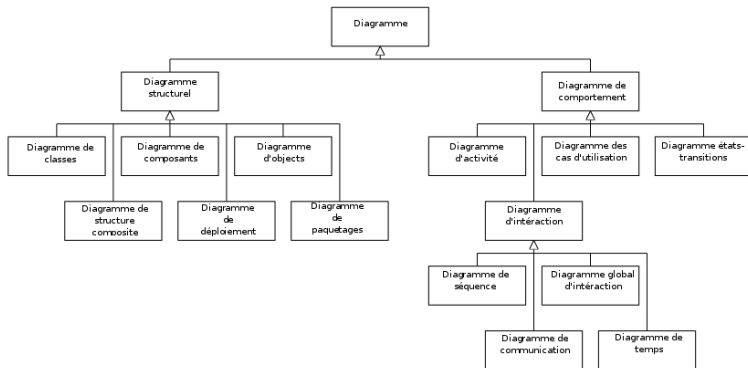
Le Langage de Modélisation Unifié, de l'anglais Unified Modeling Language (UML), est un langage de modélisation graphique à base de pictogrammes conçu pour fournir une méthode normalisée pour visualiser la conception d'un système. Il est couramment utilisé en développement logiciel et en conception orientée objet.

[https://fr.wikipedia.org/wiki/UML_\(informatique\)](https://fr.wikipedia.org/wiki/UML_(informatique))

Différents types de diagrammes

- ▶ *diagramme de classes* : représente les classes intervenant dans le système
- ▶ diagramme d'objets : représente les instances de classes
- ▶ diagramme d'activité : représente la suite des actions à effectuer dans le programme
- ▶ ...

Diagramme montrant la hiérarchie de types de diagrammes UML



source: [https://fr.wikipedia.org/wiki/UML_\(informatique\)#/media/File:Uml_diagram-fr.png](https://fr.wikipedia.org/wiki/UML_(informatique)#/media/File:Uml_diagram-fr.png)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

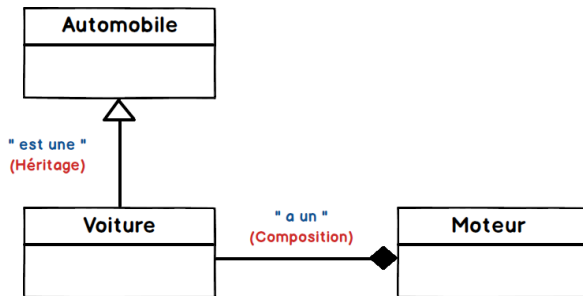
Programmation
concurrente

Python scientifique

Diagrammes de classe

Matthieu Falce

Diagramme de classes montrant composition et héritage



source: <https://waytolearnx.com/2018/08/difference-entre-heritage-et-composition.html>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

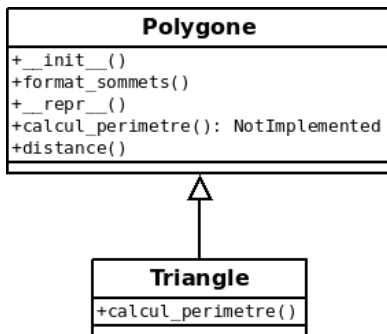
Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Diagramme de classes montrant un exemple d'héritage



3- Programmation Orientée objet (POO)

3.4. POO en python

Créer une classe

```
class MonObjet():  
    pass
```

```
o = MonObjet()  
print(o)
```

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Créer une classe

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

```
# Constructeur, méthodes et attributs
```

```
class MonAutreObjet:
    def __init__(self, nom):
        self.nom = nom

    def dis_ton_nom(self):
        print("Bonjour, je suis {}".format(self.nom))
```

```
o1 = MonAutreObjet(1)
o2 = MonAutreObjet(2)
```

```
print(o1.nom)
print(o2.nom)
```

```
o1.dis_ton_nom()
o2.dis_ton_nom()
```

Créer une classe

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

```
# Les attributs sont dynamiques et ajoutable  
# TOUT EST PUBLIC (en première approximation)
```

```
class DisBonjour():  
    def dis_bonjour(self):  
        print("Bonjour : {}".format(self.nom))
```

```
d = DisBonjour()
```

```
try:  
    # ne fonctionne pas ici, self.nom n'est pas défini  
    d.dis_bonjour()  
except NameError:  
    pass
```

```
d.nom = "Toto" # on définit un nom à qui dire bonjour  
d.dis_bonjour()  
d.nom = "Tata"  
d.dis_bonjour()
```

Méthodes magiques

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Certaines méthodes (les `__*__`) sont utilisées par l'interpréteur pour modifier le comportement des objets.

La plus connue est `__init__` qui permet d'initialiser l'objet.

Mais il y en a d'autres.

Méthodes magiques

Matthieu Falce

```
class Point():
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        """Appelée lors de print(Point(1,1))."""
        return "{}, {}".format(self.x, self.y)

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __lt__(self, other):
        print(self, other)
        return self.x < other.x # bah

    def __gt__(self, other):
        return not self.__lt__(other)

p1 = Point(1, 1)
p2 = Point(2, 1)
assert (p1 < p2) is True
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Héritage

Matthieu Falce

```
class Bonjour():
    """Classe "abstraite"
    """
    def __init__(self, nom):
        self.nom = nom

    def dis_ton_nom(self):
        # Méthode "abstraite"
        raise NotImplementedError

class BonjourFrancais(Bonjour):
    def dis_ton_nom(self):
        print("Bonjour, je suis {}".format(self.nom))

class BonjourItalien(Bonjour):
    def dis_ton_nom(self):
        print("Ciao, sono {}".format(self.nom))

# le __init__ et le nom sont gérés dans la classe mère
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Héritage

Matthieu Falce

```
import math
```

```
class Polygone():
    def __init__(self, sommets):
        self.sommets = [tuple(p) for p in sommets]
        self.name = "Polygone"

    def format_sommets(self):
        return " - ".join([str(point) for point in self.sommets])

    def __repr__(self):
        return "{}: {}".format(self.name, self.format_sommets())

    def calcule_perimetre(self):
        raise NotImplementedError

    def distance(self, a, b):
        return math.sqrt((a[0]-b[0]) ** 2 + (a[1] - b[1]) ** 2)

class Triangle(Polygone):
    def __init__(self, sommets):
        super().__init__(sommets) # !\
        self.name = "triangle"

    def calcule_perimetre(self):
        cotes = [
            (self.sommets[0], self.sommets[1]),
            (self.sommets[1], self.sommets[2]),
            (self.sommets[2], self.sommets[0])
        ]
        ds = [self.distance(p1, p2) for p1, p2 in cotes]
        return sum(ds)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Les attributs sont publics par défaut. Comment protéger certaines contraintes dans ce cas ?

- ▶ contrat avec les autres développeurs : variables "privées", préfixées par `_` : (`_temperature`)
- ▶ on peut préfixer avec un double underscore (`__temperature`) pour les rendre inaccessible hors de l'instance (l'attribut est renommé automatiquement par l'interpréteur)
- ▶ getters / setters : utiliser les propriétés

3- Programmation Orientée objet (POO)

3.5. Gestion des exceptions

Capter une exception

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

on peut capturer une exception

```
try:
    a = 1 / 0
except Exception as e:
    print(e)
else:
    print("Si pas d'exception")
finally:
    print("Dans tous les cas")
```

il faut essayer d'être plus précis dans son exception

```
try:
    a = 1 / 0
    print(a)
except ZeroDivisionError as e:
    print(e)
```

on peut capturer plusieurs exceptions

```
li = [0]
try:
    calcul = 1 / li[0]
    print(a)
except (IndexError, ZeroDivisionError) as e:
    print(e)
```

Lever une exception – Personnalisation

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

On peut lever des exceptions dans certains cas

```
def notation(note):  
    if 0 < note < 20:  
        raise ValueError(  
            "une note est entre 0 et 20, pas {}".format(note)  
        )  
    # faire des choses avec la note correcte
```

```
# =====
```

On peut créer ses propres exceptions

*# Les exceptions héritent toutes de Exception,
c'est pour ça que 'except Exception' fonctionne*

```
class MaBelleException(Exception):  
    pass
```

Taxonomie d'exceptions de la DB API

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Taxonomie des exceptions d'après la PEP 249

StandardError

 |__Warning

 |__Error

 |__InterfaceError

 |__DatabaseError

 |__DataError

 |__OperationalError

 |__IntegrityError

 |__InternalError

 |__ProgrammingError

 |__NotSupportedError

3- Programmation Orientée objet (POO)

3.6. Classe ou pas ?

Quand utiliser une classe ?

```
class Bonjour():  
    def __init__(self, nom):  
        self.nom = nom  
  
    def parle(self):  
        return "Bonjour {}".format(self.nom)
```

```
bonjour = Bonjour("Matthieu")  
print(bonjour.parle())
```

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Concepts](#)

[Association](#)

[Modélisation](#)

[POO en python](#)

[Gestion des exceptions](#)

[Classe ou pas ?](#)

[Accès attributs](#)

[Méthodes](#)

[Classes abstraites](#)

[Héritage multiple](#)

[Design Patterns](#)

[Bibliographie](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Quand utiliser une classe ?

Matthieu Falce

```
class Bonjour():  
    def __init__(self, nom):  
        self.nom = nom  
  
    def parle(self):  
        return "Bonjour {}".format(self.nom)
```

```
bonjour = Bonjour("Matthieu")  
print(bonjour.parle())
```

```
def bonjour(nom):  
    return "Bonjour {}".format(nom)  
  
print(bonjour("Matthieu"))
```

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Concepts](#)

[Association](#)

[Modélisation](#)

[POO en python](#)

[Gestion des exceptions](#)

[Classe ou pas ?](#)

[Accès attributs](#)

[Méthodes](#)

[Classes abstraites](#)

[Héritage multiple](#)

[Design Patterns](#)

[Bibliographie](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Quand utiliser une classe ?

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ Ne pas utiliser
 - ▶ quand moins de 2 méthodes...
 - ▶ seulement conteneurs, pas de méthodes (utiliser plutôt dict, namedtuple, ...)
 - ▶ gestion des ressources (plutôt context manager)

Quand utiliser une classe ?

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ Ne pas utiliser
 - ▶ quand moins de 2 méthodes...
 - ▶ seulement conteneurs, pas de méthodes (utiliser plutôt dict, namedtuple, ...)
 - ▶ gestion des ressources (plutôt context manager)
- ▶ Utiliser une classe
 - ▶ organisation (boîte noire)
 - ▶ conserver un état
 - ▶ profiter de l'OOP (héritage, ...)
 - ▶ surcharge d'opérateurs / méthodes magiques
 - ▶ produire une API définie

Conteneurs

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

```
Point2d = collections.namedtuple('Point2d', ['x', 'y'])
p1 = Point2d(3, 2)
p2 = Point2d(10, 1)

dist = math.sqrt(
    (p2.x - p1.x)**2 + (p2.y - p1.y)**2
)
```



Version python ≥ 3.7

```
@dataclass
class InventoryItem:
    '''Class for keeping track of an item in inventory.'''
    name: str
    unit_price: float
    quantity_on_hand: int = 0

    def total_cost(self) -> float:
        return self.unit_price * self.quantity_on_hand
```

3- Programmation Orientée objet (POO)

3.7. Accès attributs

Le problème

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ tous les attributs sont publics par défaut
- ▶ on ne peut pas les encapsuler facilement
- ▶ les getters et setters sont souvent redondants

Le problème

Matthieu Falce

```
class TemperatureConverter:
    def __init__(self, kelvin):
        self.kelvin = kelvin
        self.celsius = self.kelvin - 273.15

    def get_celsius(self):
        return self.kelvin - 273.15

    def set_celsius(self, value_celsius):
        self.celsius = value_celsius
        self.kelvin = value_celsius + 273.15
```

```
tc = TemperatureConverter(0)
print(tc.get_celsius())
```

```
tc.set_celsius(0)
print(tc.kelvin)
print(tc.get_celsius())
```

```
tc.celsius = 45 # mouahah
print(tc.kelvin)
print(tc.get_celsius())
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

__getattr__ / __getattribute__

- ▶ permettent de gérer les autres cas

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

__getattr__ / __getattribute__

- ▶ permettent de gérer les autres cas
- ▶ permettent de créer des attributs à la volée



Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

__getattr__ / __getattribute__

```
class Inutile():  
    def __init__(self):  
        self.a = 1  
  
    def __getattr__(self, attr_name):  
        print("getattr", attr_name)  
        return 2
```

```
i = Inutile()  
print(i.a)  
print(i.b)
```

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

__getattr__ / __getattribute__

Matthieu Falce

```
class Inutile():
    def __init__(self):
        self.a = 1

    def __getattribute__(self, attr_name):
        print("getattribute", attr_name)
        return 3

i = Inutile()
print(i.a)
print(i.b)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

__getattr__ / __getattribute__

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

```
class Inutile():  
    def __init__(self):  
        self.a = 1  
  
    def __getattribute__(self, attr_name):  
        print("getattribute", attr_name)  
        if hasattr(self, attr_name):  
            print("deja présent")  
        return 3
```

```
i = Inutile()  
print(i.a)  # RecursionError
```

Descriptors

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ utilisés pour les accès aux attributs
- ▶ définissent un protocole de lecture / écriture / suppression
- ▶ accès supplémentaire rajouté par Python

Descriptors

Matthieu Falce

```
class Celsius:
    def __init__(self, value=0.0):
        self.value = value

    def __get__(self, instance, owner):
        return instance.kelvin - 273.15

    def __set__(self, instance, value):
        self.value = value
        instance.kelvin = value + 273.15

class TemperatureConverter:
    # un descripteur modifie le comportement d'une classe
    # PAS d'une instance.
    celsius = Celsius()

    def __init__(self, kelvin):
        self.kelvin = kelvin

tc = TemperatureConverter(0)
print(tc.celsius)
print(tc.kelvin)

tc.kelvin = 0
print(tc.kelvin)
print(tc.celsius)

tc.celsius = 0
print(tc.kelvin)
print(tc.celsius)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Descriptors

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

pourquoi ça marche ?

en gros, dans le code de `objet`

```
def __getattribute__(self, attr):  
    obj = object.__getattribute__(self, attr)  
    if hasattr(obj, "__get__"):  
        return obj.__get__(self, type(self)) # là où la magie opère  
    return obj
```

Propriétés

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ gestion des accès en lecture / écriture / suppression
- ▶ décorateur avec une syntaxe un peu spéciale
- ▶ avantage : on ne sait pas que l'on ne modifie pas l'attribut directement
- ▶ les propriétés sont une sorte de descripteur

Properties

Matthieu Falce

```
class TemperatureConverter:
    def __init__(self, kelvin):
        self.kelvin = kelvin

    @property
    def celsius(self):
        return self.kelvin - 273.15

    @celsius.setter
    def celsius(self, value_celsius):
        self.kelvin = value_celsius + 273.15

tc = TemperatureConverter(0)
print(tc.celsius)
print(tc.kelvin)

tc.kelvin = 0
print(tc.kelvin)
print(tc.celsius)

tc.celsius = 0
print(tc.kelvin)
print(tc.celsius)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Conclusion

Essayer par priorité :

- ▶ attribut classique
- ▶ property
- ▶ descriptor
- ▶ `__getattr__`
- ▶ `__getattribute__`

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

3- Programmation Orientée objet (POO)

3.8. Méthodes

Différents types de méthodes

Matthieu Falce

```
class Exemple():
    def __init__(self, attribut):
        self.attribut = attribut

    def methode(self, param):
        print(self, type(self))
        return self.attribut + param
```

```
e = Exemple(10)
print(e.methode(2))
```

method

- ▶ classique
- ▶ s'applique à une instance
- ▶ accès aux variables de classe et d'instance
- ▶ `self` est injecté automatiquement (bound method)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Différents types de méthodes

```
class Exemple():  
    variable_de_classe = 1  
  
    @classmethod  
    def methode_de_classe(cls, param):  
        print(cls, type(cls))  
        return cls.variable_de_classe + param
```

```
print(Exemple.methode_de_classe(5))
```

classmethod

- ▶ s'applique sur une classe et pas une instance
- ▶ accès aux variables de classe
- ▶ cls est injecté automatiquement

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Concepts](#)

[Association](#)

[Modélisation](#)

[POO en python](#)

[Gestion des exceptions](#)

[Classe ou pas ?](#)

[Accès attributs](#)

Méthodes

[Classes abstraites](#)

[Héritage multiple](#)

[Design Patterns](#)

[Bibliographie](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Différents types de méthodes

Matthieu Falce

```
class Galette():
    def __init__(self, ingredients):
        self.ingredients = ingredients

    @classmethod
    def complete(cls):
        return cls(["jambon", "fromage", "oeuf"])

    @classmethod
    def nature(cls):
        return cls(["beurre salé"])

print(Galette.complete().ingredients)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Différents types de méthodes

Matthieu Falce

```
class Exemple():  
    @staticmethod  
    def methode_statique(param):  
        return param  
  
print(Exemple.methode_statique(5))
```

staticmethod

- ▶ permet de regrouper des fonctions dans l'objet
- ▶ n'a accès à aucune information classe ou instance
- ▶ ne va pas modifier l'état de la classe ou de l'instance

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Quel est le résultat ?

```
class MyClass:
    def method(self):
        return "méthode d'instance", self

    @classmethod
    def _classmethod(cls):
        return 'méthode de classe', cls

    @staticmethod
    def _staticmethod():
        return 'méthode statique'

print(MyClass._staticmethod())
print(MyClass._classmethod())
print(MyClass.method())

m = MyClass()
print(m._staticmethod())
print(m._classmethod())
print(m.method())
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

3- Programmation Orientée objet (POO)

3.9. Classes abstraites

Classe abstraite ?

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ classe à *trous* : il manque des méthodes
- ▶ possède des méthodes abstraites
- ▶ on doit en hériter pour l'instancier

Exemple

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

```
class FausseClasseAbstraite():
    def fausse_methode_abstraite(self):
        raise NotImplementedError

class ClasseFille(FausseClasseAbstraite):
    def fausse_methode_abstraite(self):
        return "surchargée"

fca = FausseClasseAbstraite()
fca.fausse_methode_abstraite() # NotImplementedError

cf = ClasseFille()
cf.fausse_methode_abstraite()
```

Exemple

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Concepts](#)

[Association](#)

[Modélisation](#)

[POO en python](#)

[Gestion des exceptions](#)

[Classe ou pas ?](#)

[Accès attributs](#)

[Méthodes](#)

[Classes abstraites](#)

[Héritage multiple](#)

[Design Patterns](#)

[Bibliographie](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

```
from abc import ABC, abstractmethod
```

```
class Abstraite(ABC):
```

```
    @abstractmethod
```

```
    def methode_abstraite(self):
```

```
        pass
```

```
class Fille(Abstraite):
```

```
    def methode_abstraite(self):
```

```
        print("methode abstraite de Fille")
```

```
a = Abstraite()
```

```
# TypeError: Can't instantiate abstract class Abstraite
```

```
# with abstract methods methode_abstraite
```

```
b = Fille()
```

```
b.methode_abstraite()
```

A quoi ça sert

Matthieu Falce

```
from abc import ABC, abstractmethod
import sys

class Button(ABC):
    @abstractmethod
    def paint(self):
        print("Code multiplateforme commun")

class LinuxButton(Button):
    def paint(self):
        super().paint()
        print("code pour X11")

class WindowsButton(Button):
    def paint(self):
        super().paint()
        print("code pour dwm")

if "linux" in sys.platform.lower():
    button = LinuxButton()
elif "windows" in sys.platform.lower():
    button = WindowsButton()

button.paint()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

A quoi ça sert

- ▶ permet de fixer un contrat
- ▶ assez rarement utilisé en pratique

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

3- Programmation Orientée objet (POO)

3.10. Héritage multiple

Syntaxe

```
class A:  
    pass
```

```
class B:  
    pass
```

```
class C(A, B):  
    pass
```

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

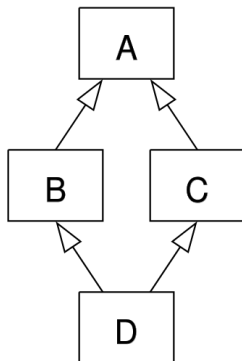
Création de
modules

Programmation
concurrente

Python scientifique

Héritage en diamant

Quelle méthode va être appelée ?



Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Method Resolution Order (MRO)

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ déterminer l'ordre des méthodes à appeler (super)
- ▶ linéarisation l'arbre d'héritage
- ▶ utilise l'algorithme du C3 ²⁶

²⁶https://en.wikipedia.org/wiki/C3_linearization

Method Resolution Order (MRO)

Exemple de résolution

```
class A1: pass
```

```
class A2: pass
```

```
class B(A1): pass
```

```
class C(B): pass
```

```
class D1(A1, A2): pass
```

```
class D2(A2, A1): pass
```

```
print('mro A', A1.__mro__) # A1 / objet
```

```
print('mro B', B.__mro__) # B / A1 / objet
```

```
print('mro C', C.__mro__) # C / B / A1 / objet
```

```
print('mro D1', D1.__mro__) # D1 / A1 / A2 / objet
```

```
print('mro D2', D2.__mro__) # D2 / A2 / A1 / objet
```

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Method Résolution Order (MRO)

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Héritage en diamant

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass

print("MRO diamant : ", D.__mro__)
```

Method Resolution Order (MRO)

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

L'algorithme de linéarisation a cependant des limites...

```
class A: pass
class B(A): pass

class C(B, A): pass
class D(A, B): pass

# TypeError: Cannot create a consistent method resolution
# order (MRO) for bases A, B
```


Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ classe destinée à être composée par héritage
- ▶ service que l'on peut rajouter aux classes filles
- ▶ pas destinée à être utilisée seule

Mixins

Matthieu Falce

```
class Bonjour:
    def __init__(self, name):
        self.name = name

    def parler(self):
        print("bonjour {} !".format(self.name))

class DisEnCouleur:
    def parler(self):
        print("\033[1;31m", end="") # code terminal pour le rouge
        print("bonjour {} !".format(self.name), end="")
        print("\033[0;0m") # code terminal pour remettre à zéro

class DisPas:
    def parler(self):
        print("*** censuré ***")

class BonjourEnCouleur(DisEnCouleur, Bonjour): pass
class DisPasBonjour(DisPas, Bonjour): pass

b = Bonjour("Matthieu")
bec = BonjourEnCouleur("Matthieu")
dpb = DisPasBonjour("Matthieu")

b.parler()
bec.parler()
dpb.parler()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

3- Programmation Orientée objet (POO)

3.11. Design Patterns

Patrons de conception ?

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Introduits par Gamma, Helm, Johnson et Vlissides (le Gang of Four) en 1994 par le livre *Design Patterns: Elements of Reusable Software*

Patrons de conception ?

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

En informatique, et plus particulièrement en développement logiciel, un patron de conception (souvent appelé design pattern) est un arrangement caractéristique de modules, reconnu comme bonne pratique en réponse à un problème de conception d'un logiciel. Il décrit une solution standard, utilisable dans la conception de différents logiciels.

https://fr.wikipedia.org/wiki/Patron_de_conception

Patrons de conception ?

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

« Chaque patron décrit un problème qui se manifeste constamment dans notre environnement, et donc décrit le cœur de la solution à ce problème, d'une façon telle que l'on puisse réutiliser cette solution des millions de fois, sans jamais le faire deux fois de la même manière »

Christopher Alexander, 1977.

Patrons de conception ?

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

3 familles de patrons d'après le GoF

- ▶ *créateurs* : ils définissent comment faire l'instanciation et la configuration des classes et des objets ;
- ▶ *structuraux* : ils définissent comment organiser les classes d'un programme dans une structure plus large (séparant l'interface de l'implémentation) ;
- ▶ *comportementaux* : ils définissent comment organiser les objets pour que ceux-ci collaborent (distribution des responsabilités) et expliquent le fonctionnement des algorithmes impliqués ;

Patrons de conception ?

Matthieu Falce

Quelques exemples :

- ▶ factory
- ▶ adapter
- ▶ chain of responsibility
- ▶ decorator
- ▶ facade
- ▶ iterator
- ▶ observer
- ▶ strategy
- ▶ Model View Controller (MVC)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Patrons de conception ?

Matthieu Falce

2 principes généraux :

- ▶ Tenir compte de l'interface et pas de l'implémentation.
- ▶ Préférer la composition à l'héritage.

```
class ConteneurComposition():  
    def __init__(self):  
        self._conteneur = []  
  
    def append(self, valeur):  
        print("avant append")  
        self._conteneur.append(valeur)  
        print("après append")
```

```
class ConteneurHéritage(list):  
    def append(self, valeur):  
        print("avant append")  
        super().append(valeur)  
        print("après append")
```

```
cc = ConteneurComposition()  
ch = ConteneurHéritage()
```

les 2 objets ont la même interface mais pas le même type (duck typing)

```
cc.append(1)  
ch.append(2)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Accès aux éléments d'un conteneur séquentiellement sans avoir besoin d'exposer la structure interne du conteneur

Inclus de base dans le langage

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Comportementaux – Chain of responsibility

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Comment assurer une série de traitement sur des objets ?

```
class ContentFilter(object):
    def __init__(self, filters=None):
        self._filters = filters or []

    def filter(self, content):
        for filter in self._filters:
            content = filter(content)
        return content

content_filter = ContentFilter(
    [
        lambda c: (e for e in c if e % 2 == 0),
        lambda c: (e for e in c if str(e) == str(e)[::-1]),
    ]
)
content = range(1000)
filtered_content = content_filter.filter(content)
print(list(filtered_content)[10:20])
```

Comportementaux – Observer

Matthieu Falce

Comment distribuer des notifications à plusieurs objets qui doivent être avertis d'une notification ?

```
class Observer():
    observers = []
    def __init__(self):
        self.observers.append(self)
        self._observables = {}
    def observe(self, event_name, callback):
        self._observables[event_name] = callback

class Event():
    def __init__(self, name, data):
        self.name = name
        self.data = data
    def fire(self):
        for observer in Observer.observers:
            if self.name in observer._observables:
                observer._observables[self.name](self.data)

class Salle(Observer):
    def vient_d_arriver(self, who):
        print("nouvel événement : ", who, "est arrivé !")

salle = Salle()
salle.observe('arrive', salle.vient_d_arriver)
Event('arrive', 'Matthieu').fire()
Event('part', 'Matthieu').fire()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Createur – Singleton

Matthieu Falce

Comment s'assurer qu'une seule instance d'une classe ne peut exister à un moment donné et fournir un point d'accès vers cette instance ?

```
class Singleton(object):
    _instances = {}

    def __new__(cls, *args, **kw):
        if not cls in cls._instances:
            instance = super().__new__(cls)
            cls._instances[cls] = instance
        return cls._instances[cls]
```

```
class Logger(Singleton):
    pass
```

```
class Logger2(Singleton):
    pass
```

```
l1 = Logger()
l1_bis = Logger()
print(l1 is l1_bis)
```

```
l2 = Logger2()
print(l1 is l2)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Comment s'assurer qu'une seule instance d'une classe ne peut exister à un moment donné et fournir un point d'accès vers cette instance ?

On peut le faire autrement (à la main) :

- ▶ définir dans un module
- ▶ définir dans un fichier de conf chargé une seule fois
- ▶ passer l'instance à chaque objet

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Comment ajouter de nouvelles fonctions à un objet
dynamiquement lors de l'exécution ?

Inclus de base dans le langage (les fonctions sont des *first
class citizen*)

Comment ajouter de nouvelles fonctions à un objet
dynamiquement lors de l'exécution ?

Inclus de base dans le langage (les fonctions sont des *first
class citizen*)

Pas forcément !

Les décorations avec @ sont statiques et pas dynamiques.
Mais même concept d'enveloppement.

Comment déguiser une classe en une autre ?

```
from datetime import datetime

def log(message, destination):
    destination.write("{} - {}".format(datetime.now(), message))

class ConsoleDestination:
    def write(self, message):
        print(message)

# le duck typing facilite ce pattern car on n'a pas
# besoin d'avoir le bon type, juste la bonne interface
log("dans un fichier", open("/tmp/log.log", "w"))
log("dans la console", ConsoleDestination())
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Conclusion

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ faciles à mettre en place en python avec le *duck typing*
- ▶ permettent d'exprimer et de formaliser une approche
- ▶ permettent de structurer des projets en ayant des abstractions connues (changement des équipes, longs développements)
- ▶ permettent de prévoir de bonnes API à ces classes
- ▶ ne pas chercher à en abuser

3- Programmation Orientée objet (POO)

3.12. Bibliographie

Bibliographie I

Matthieu Falce

- ▶ Tous les sujets :
 - ▶ <http://www.dabeaz.com/py3meta/Py3Meta.pdf>
- ▶ Classe ou pas
 - ▶ <https://eev.ee/blog/2013/03/03/the-controller-pattern-is-awful-and-other-oo-heresy/>
 - ▶ <https://www.youtube.com/watch?v=o9pEzgHorH0>
 - ▶ <http://lucumr.pocoo.org/2013/2/13/moar-classes/>
- ▶ Méthodes de classe / statiques / méthode :
 - ▶ <https://realpython.com/instance-class-and-static-methods-demystified/>
 - ▶ commentaire de l'article
<http://sametmax.com/comprendre-les-decorateur-python-pas-a-pas-partie-2/>
 - ▶ <https://rushter.com/blog/python-class-internals/>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Bibliographie II

Matthieu Falce

► Descriptors :

- <https://docs.python.org/3/howto/descriptor.html>
- <https://docs.python.org/3/reference/datamodel.html#customizing-attribute-access>
- <https://eev.ee/blog/2012/05/23/python-faq-descriptors/>
- <https://stackoverflow.com/questions/22616559/use-cases-for-property-vs-descriptor-vs-get-attribute>
- <https://www.smallsurething.com/python-descriptors-made-simple/>
- <https://stackoverflow.com/questions/12846116/python-descriptor-vs-property>
- <https://stackoverflow.com/questions/17330160/how-does-the-property-decorator-work>
- <http://sametmax.com/les-descripteurs-en-python/>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Bibliographie III

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ <https://stackoverflow.com/questions/3798835/understanding-get-and-set-and-python-descriptors>
- ▶ <https://stackoverflow.com/questions/23309698/why-is-the-descriptor-not-getting-called-when-defined-as-instance-attribute>
- ▶ <https://stackoverflow.com/questions/4877290/what-is-the-dict-dict-attribute-of-a-python-class#4877655>
- ▶ Design patterns :
 - ▶ https://github.com/ActiveState/code/blob/master/recipes/Python/102187_Singleton_as_a_metaclass/recipe-102187.py
 - ▶ <https://github.com/faif/python-patterns>
 - ▶ <https://www.toptal.com/python/python-design-patterns>
 - ▶ <https://www.youtube.com/watch?v=0vJJlVBTFg>

Bibliographie IV

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ <http://sametmax.com/objets-proxy-et-pattern-a-dapter-en-python/>
- ▶ <http://www.e-naxos.com/Blog/post/Design-Patterns-ou-quand-comment-et-pourquoi-.aspx>
- ▶ <http://sdz.tdct.org/sdz/le-pattern-decorator-en-python.html>
- ▶ Loi de Demeter :
 - ▶ <https://www2.ccs.neu.edu/research/demeter/demeter-method/LawOfDemeter/paper-boy/demeter.pdf>
- ▶ Héritage multiple / MRO :
 - ▶ https://en.wikipedia.org/wiki/Multiple_inheritance
 - ▶ https://en.wikipedia.org/wiki/C3_linearization

Bibliographie V

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ <https://stackoverflow.com/questions/3277367/how-does-pythons-super-work-with-multiple-inheritance>
- ▶ <https://easyaspython.com/mixins-for-fun-and-profit-cb9962760556>
- ▶ <https://stackoverflow.com/questions/14088294/multithreaded-web-server-in-python>
- ▶ Métaclasses:
 - ▶ <https://stackoverflow.com/questions/392160/what-are-some-concrete-use-cases-for-metaclasses>
 - ▶ <http://www.dabeaz.com/py3meta/Py3Meta.pdf>
 - ▶ <https://stackoverflow.com/questions/6760685/creating-a-singleton-in-python>

Bibliographie VI

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Concepts

Association

Modélisation

POO en python

Gestion des exceptions

Classe ou pas ?

Accès attributs

Méthodes

Classes abstraites

Héritage multiple

Design Patterns

Bibliographie

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ <http://sametmax.com/le-guide-ultime-et-definitif-sur-la-programmation-orientee-objet-en-python-a-lusage-des-debutants-qui-sont-rassures-par-les-textes-detaillies-qui-prennent-le-temps-de-tout-expliquer-partie-8/>
- ▶ <https://stackoverflow.com/questions/5730211/how-does-get-field-display-in-django-work>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Bonnes pratiques

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

4- Bonnes pratiques

4.1. Pourquoi ?

Qu'est-ce que c'est ?

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

La QA (*Quality Assurance*)

- ▶ monitore le développement logiciel et les méthodes utilisées
- ▶ doit être suivie et contrôlée
- ▶ doit s'adapter aux nécessités métier (ne pas être trop contraignante)

Pourquoi ?

- ▶ le code est plus souvent lu que écrit
 - ▶ règle de nommage des fichiers / modules / fonctions / variables
 - ▶ *linter*
 - ▶ documentation (qui évolue avec le code)

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Pourquoi ?

Matthieu Falce

- ▶ le code est plus souvent lu que écrit
 - ▶ règle de nommage des fichiers / modules / fonctions / variables
 - ▶ *linter*
 - ▶ documentation (qui évolue avec le code)
- ▶ le code doit fonctionner
 - ▶ vérifier le code avec des tests unitaires
 - ▶ utiliser des vérificateurs de typage statique

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Pourquoi ?

Matthieu Falce

- ▶ le code est plus souvent lu que écrit
 - ▶ règle de nommage des fichiers / modules / fonctions / variables
 - ▶ *linter*
 - ▶ documentation (qui évolue avec le code)
- ▶ le code doit fonctionner
 - ▶ vérifier le code avec des tests unitaires
 - ▶ utiliser des vérificateurs de typage statique
- ▶ le code doit pouvoir être déployé facilement
 - ▶ utiliser des système de build automatiques (qui évoluent avec le code)
 - ▶ utiliser un système d'intégration continue (*CI*)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Pourquoi ?

Matthieu Falce

- ▶ le code est plus souvent lu que écrit
 - ▶ règle de nommage des fichiers / modules / fonctions / variables
 - ▶ *linter*
 - ▶ documentation (qui évolue avec le code)
- ▶ le code doit fonctionner
 - ▶ vérifier le code avec des tests unitaires
 - ▶ utiliser des vérificateurs de typage statique
- ▶ le code doit pouvoir être déployé facilement
 - ▶ utiliser des système de build automatiques (qui évoluent avec le code)
 - ▶ utiliser un système d'intégration continue (*CI*)
- ▶ on peut revenir à une version antérieure du projet / savoir qui a fait quoi / quand
 - ▶ utiliser un système de contrôle de version (Git, ...)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Avant propos

Écosystème

pip

Environnements virtuels

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

4- Bonnes pratiques

4.2. Installation de paquets

Avant Propos

Matthieu Falce

Le packaging en python est relativement mal connu et compris.

- ▶ plusieurs outils concurrents (distutils, setuptools, pip, pipenv, virtuelenv...)
- ▶ difficulté à installer des packages (compilation à l'installation)
- ▶ peu de considération des "core dev"

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Avant propos

Écosystème

pip

Environnements virtuels

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Avant Propos

Matthieu Falce

Le packaging en python est relativement mal connu et compris.

- ▶ plusieurs outils concurrents (distutils, setuptools, pip, pipenv, virtualenv...)
- ▶ difficulté à installer des packages (compilation à l'installation)
- ▶ peu de considération des “core dev”

Ce n'est plus trop le cas aujourd'hui.

A présent : outils matures, inclus par défaut et utilisés.

Merci au **PyPA** <3

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Avant propos

Écosystème

pip

Environnements virtuels

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

- ▶ Gestion environnement :
 - ▶ virtualenv (+ wrappers)
 - ▶ pip
 - ▶ easy_install
 - ▶ pipenv
 - ▶ (conda)
- ▶ PyPI
- ▶ wheels
- ▶ eggs
- ▶ ...

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Avant propos

Écosystème

pip

Environnements virtuels

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Installer

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Pourquoi ?](#)

[Installation de paquets](#)

[Avant propos](#)

[Écosystème](#)

[pip](#)

[Environnements virtuels](#)

[Débug](#)

[Linter](#)

[Analyse des performances](#)

[Tests](#)

[Documentation](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Si on lui donne un chemin, pip cherche un setup.py

Si on lui donne un nom, il va chercher sur pypi.

On peut aussi lui donner un chemin distant en http / git /
hg / ...

```
# installation depuis Pypi  
pip install numpy
```

Installation

```
# installer depuis PyPi
pip install unModule

# installer depuis un wheel local
pip install unModule-1.0-py2.py3-none-any.whl

# installer une version "précise"
pip install unModule==0.10.1
pip install unModule>=0.9,<0.11

# installation depuis un chemin
pip install .

# installation depuis git
## url d'un dépôt git
## git@github.com:pypa/sampleproject.git
## on doit rajouter git+ssh:// et changer le :pypa en /pypa
pip install git+ssh://git@github.com/pypa/sampleproject.git

# installer des paquets avec des options
pip install "project[extra]"
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Avant propos

Écosystème

pip

Environnements virtuels

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Installation (cas particuliers)

```
# installation depuis un chemin  
pip install .
```

```
# installation depuis git  
## url d'un dépôt git  
## git@github.com:pypa/sampleproject.git  
## on doit rajouter git+ssh:// et changer le :pypa en /pypa  
pip install git+ssh://git@github.com/pypa/sampleproject.git
```

```
# installer des paquets avec des options  
pip install "project[extra]"
```

Commandes classiques

Matthieu Falce

Cycle de vie des paquets installés

lister les modules non à jour

```
pip list --outdated
```

mettre à jour un module

```
pip install --upgrade unModule
```

```
pip install -U unModule
```

supprimer un module

```
pip uninstall SomePackage
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Avant propos

Écosystème

pip

Environnements virtuels

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Commandes classiques

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Avant propos

Écosystème

pip

Environnements virtuels

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Fichier requirements.txt

freeze des dépendances

```
pip freeze > requirements.txt
```

installer depuis un fichier de requirements

```
pip install -r requirements.txt
```

Autres commandes

- ▶ `pip download` (télécharge sans installer)
- ▶ `pip list` (liste les paquets installés)
- ▶ `pip show` (liste les informations sur les paquets installés)
- ▶ `pip search` (cherche les paquets avec un nom compatible)
- ▶ `pip check` (vérifie si les dépendances sont compatibles)
- ▶ `pip wheel` (construit un wheel)
- ▶ `pip hash` (calcule le *hash* d'un module)

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Pourquoi ?](#)

[Installation de paquets](#)

[Avant propos](#)

[Écosystème](#)

[pip](#)

[Environnements virtuels](#)

[Débug](#)

[Linter](#)

[Analyse des performances](#)

[Tests](#)

[Documentation](#)

[Bibliothèque
standard](#)

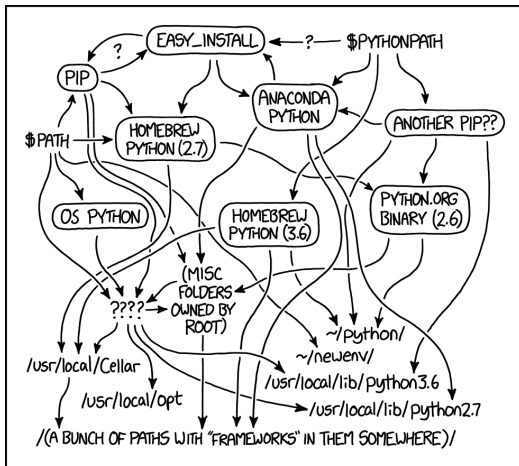
[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Environnement d'installation sain

Matthieu Falce



<https://xkcd.com/1987/>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Avant propos

Écosystème

pip

Environnements virtuels

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Environnement d'installation sain

- ▶ savoir ce que l'on installe ;
- ▶ savoir comment on l'installe ;
- ▶ savoir où on l'installe ;

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Avant propos

Écosystème

pip

Environnements virtuels

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Installer des modules externes

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Pourquoi ?](#)

[Installation de paquets](#)

[Avant propos](#)

[Écosystème](#)

[pip](#)

[Environnements virtuels](#)

[Débug](#)

[Linter](#)

[Analyse des performances](#)

[Tests](#)

[Documentation](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

On ne veut pas forcément installer des dépendances de façon globale :

- ▶ `virtualenv` (solution standard)
- ▶ `conda env` (développé par Continuum Analytics, ceux qui font Anaconda, utilisé en calcul scientifique, gère les bibliothèques C...)

- ▶ s'abstraire du python système
- ▶ changer de projet facilement
- ▶ avoir des versions différentes de bibliothèques installées en parallèle
- ▶ être "iso" avec l'environnement de production (plus subtil que ça)

virtualenv

```
#installation (avec le Python système)
pip install virtualenv

# aller dans le dossier où l'on veut créer le venv
# dossier du projet ou dossier commun à tous les venvs
cd my_project_folder

# on crée le venv
virtualenv venv

# on l'active (modifie les variables d'environnement pour Python)
source venv/bin/activate

# on vérifie que ça a marché
which python

### c'est ici qu'on travaille...

# on désactive pour quitter (restore les variables d'environnement)
deactivate
```

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Avant propos

Écosystème

pip

Environnements virtuels

Débug

Linter

Analyse des performances

Tests

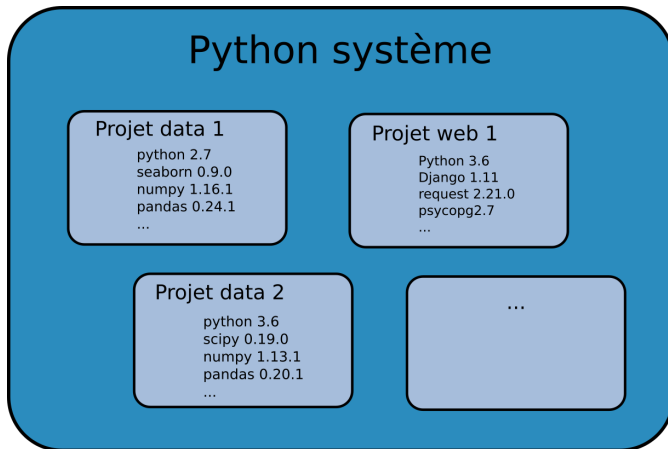
Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique



Coexistence de plusieurs versions de Python

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Avant propos

Écosystème

pip

Environnements virtuels

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

virtualenv

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Avant propos

Écosystème

pip

Environnements virtuels

Débug

Linter

Analyse des performances

Tests

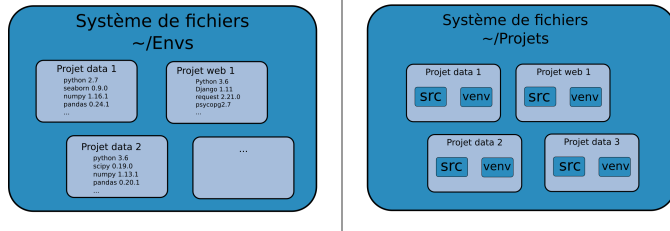
Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique



Organisation des environnements virtuels

virtualenv

Matthieu Falce

- ▶ on peut préciser la version de python (`virtualenv -p /usr/bin/python2.7 venv`)
- ▶ s'utilise souvent avec des *wrappers*
 - ▶ `pew`
 - ▶ `virtualenvwrapper`
 - ▶ ...
- ▶ ne permet pas l'isolation parfaite, juste Python
 - ▶ les dépendances externes (installer un paquet système) peuvent être gérées (`wheel`)
 - ▶ utiliser Vagrant ou Docker dans les cas complexes

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Avant propos

Écosystème

pip

Environnements virtuels

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

4- Bonnes pratiques

4.3. Débug

Outils de débogage

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Python contient des outils permettant de déboguer et d'analyser le bytecode généré pour une fonction

```
import pdb, dis

for i in range(-10, 11):
    try:
        print(100 / i)
    except Exception:
        import pdb; pdb.set_trace()

#####

def rapide():
    return 1

def lente():
    a = 5
    return a

print("decompilation de rapide : ")
dis.dis(rapide)
print("decompilation de lente : ")
dis.dis(lente)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

4- Bonnes pratiques

4.4. *Linter*

Qualité du code – pep8 / linters

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Python propose sa vision d'un "code propre" : la **PEP8**

- ▶ indentation avec 4 espaces
- ▶ lignes de 80 caractères
- ▶ respect d'une aération du code
- ▶ espace dans les expressions
- ▶ ...

Il existe des “linters” pour vous assister dans l'écriture. ²⁶ ²⁷
Ils peuvent lister les erreurs, variables non déclarées, typos, mauvais import...

- ▶ flake8
- ▶ pylint
- ▶ ...

Intégration avec les éditeurs de texte.

²⁶<https://stackoverflow.com/questions/5611776/what-are-the-comprehensive-lint-checkers-for-python>

²⁷<https://stackoverflow.com/questions/1428872/pylint-pychecker-or-pyflakes?noredirect=1&lq=1>

Qualité du code – pep8 / linters

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Certains outils reformatent automatiquement le code que vous leur donnez (concentration sur le code plutôt que la présentation).^{26 27}

- ▶ black
- ▶ yapf
- ▶ autopep8
- ▶ ...

Intégration avec les éditeurs de texte.

26.<https://medium.com/3yourmind/auto-formatters-for-python-8925065f9505>

27.<https://news.ycombinator.com/item?id=17155048>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

4- Bonnes pratiques

4.5. Analyse des performances

Timing et profilage

Matthieu Falce

```
import time, timeit, cProfile

def fonction_1():
    sum([i for i in range(int(1e5))])

def fonction_2():
    sum(i for i in range(int(1e5)))

tic = time.time()
fonction_1()
print("fonction 1 : {}".format(time.time() - tic))

print("100x fonction2 : {}".format(
    timeit.timeit("fonction_2()", number=100, globals=globals())
))

cProfile.run('fonction_1()')
cProfile.run('fonction_2()')
```

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Pourquoi ?](#)

[Installation de paquets](#)

[Débug](#)

[Linter](#)

[Analyse des performances](#)

[Tests](#)

[Documentation](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Résultat

6 function calls in 0.004 seconds

Ordered by: standard name

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
1	0.001	0.001	0.004	0.004	<ipython-input-8-ac539deb9692>:4(fonction_1)
1	0.002	0.002	0.002	0.002	<ipython-input-8-ac539deb9692>:5(<listcomp>)
1	0.000	0.000	0.004	0.004	<string>:1(<module>)
1	0.000	0.000	0.004	0.004	{built-in method builtins.exec}
1	0.001	0.001	0.001	0.001	{built-in method builtins.sum}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}

=====

100006 function calls in 0.012 seconds

Ordered by: standard name

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	0.012	0.012	<ipython-input-8-ac539deb9692>:8(fonction_2)
100001	0.006	0.000	0.006	0.000	<ipython-input-8-ac539deb9692>:9(<genexpr>)
1	0.000	0.000	0.012	0.012	<string>:1(<module>)
1	0.000	0.000	0.012	0.012	{built-in method builtins.exec}
1	0.006	0.006	0.012	0.012	{built-in method builtins.sum}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

4- Bonnes pratiques

4.6. Tests

En programmation informatique, le test unitaire ou test de composants est une procédure permettant de vérifier le bon fonctionnement d'une partie précise d'un logiciel ou d'une portion d'un programme (appelée « unité » ou « module »). Dans les applications non critiques, l'écriture des tests unitaires a longtemps été considérée comme une tâche secondaire. Cependant, les méthodes Extreme programming (XP) ou Test Driven Development (TDD) ont remis les tests unitaires, appelés 'tests du programmeur', au centre de l'activité de programmation. À noter que le test unitaire peut ne pas être automatique.

https://fr.wikipedia.org/wiki/Test_unitaire

Nous allons utiliser la bibliothèque unittest ²⁸

²⁸<https://docs.python.org/3/library/unittest.html>

Tests unitaires – tests verts

Matthieu Falce

```
import unittest
```

```
class TestThings(unittest.TestCase):
```

```
    def test_upper(self):
        self.assertEqual('foo'.upper(), 'F00')
```

```
    def test_isupper(self):
        self.assertTrue('F00'.isupper())
        self.assertFalse('Foo'.isupper())
```

```
    def test_split(self):
        s = 'hello world'
        self.assertEqual(s.split(), ['hello', 'world'])
        # check that s.split fails when the separator is not a string
        with self.assertRaises(TypeError):
            s.split(2)
```

```
    def test_almostEqual(self):
        self.assertAlmostEqual(1/3, 0.33333333333)
```

```
if __name__ == '__main__':
    unittest.main()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Tests unitaires – tests verts

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Résultat :

```
python test_unittest.py
```

```
....
```

```
-----
```

```
Ran 4 tests in 0.001s
```

```
OK
```

Tests unitaires – tests rouges

Matthieu Falce

```
import unittest

class TestErrors(unittest.TestCase):
    def test_error(self):
        computation = 2+2
        should_be = 3
        self.assertEqual(computation, should_be)

    def test_exception(self):
        computation = 1/0
        should_not_be = 1
        self.assertNotEqual(computation, should_be)

if __name__ == '__main__':
    unittest.main()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Tests unitaires – tests rouges

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Pourquoi ?](#)

[Installation de paquets](#)

[Débug](#)

[Linter](#)

[Analyse des performances](#)

Tests

[Documentation](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Résultat :

```
FE.
=====
ERROR: test_exception (__main__.TestMath)
-----
Traceback (most recent call last):
  File "../codes/modules/test_unittest2.py", line 13, in test_exception
    computation = 1/0
ZeroDivisionError: division by zero

=====
FAIL: test_error (__main__.TestMath)
-----
Traceback (most recent call last):
  File "../codes/modules/test_unittest2.py", line 10, in test_error
    self.assertEqual(computation, should_be)
AssertionError: 4 != 3

-----
Ran 3 tests in 0.001s

FAILED (failures=1, errors=1)
```

Tests unitaires – fixtures

Matthieu Falce

```
import unittest
```

```
class FixturesTest(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        print('In setUpClass()'); cls.set_for_class = 10

    @classmethod
    def tearDownClass(cls):
        print('\nIn tearDownClass()'); print(cls.set_for_class)
        del cls.set_for_class

    def setUp(self):
        super().setUp(); print('\n    In setUp()')
        self.set_for_function = 5

    def tearDown(self):
        print('    In tearDown()', '\n        ', 'set_for_function:', self.set_for_function)
        del self.set_for_function; super().tearDown()

    def test1(self):
        print('        In test1()');
        print('        ', FixturesTest.set_for_class, '\n        ', self.set_for_function);
        FixturesTest.set_for_class = 1; self.set_for_function = 2

    def test2(self):
        print('        In test2()');
        print('        ', FixturesTest.set_for_class, '\n        ', self.set_for_function);
        FixturesTest.set_for_class = 3; self.set_for_function = 4

if __name__ == '__main__':
    unittest.main()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Tests unitaires – fixtures

Matthieu Falce

Voilà le résultat :

```
In setUpClass()

In setUp()
  In test1()
    10
    5
In tearDown()
  set_for_function: 2
.
In setUp()
  In test2()
    1
    5
In tearDown()
  set_for_function: 4
.
In tearDownClass()
3
```

Ran 2 tests in 0.000s

OK

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Pourquoi ?](#)

[Installation de paquets](#)

[Débug](#)

[Linter](#)

[Analyse des performances](#)

Tests

[Documentation](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Bonne explication du module unittest :

<https://pymotw.com/3/unittest/>

Pour aller plus loin:

- ▶ découverte automatique de tests
- ▶ tearDown plus fiables
- ▶ code coverage et rapports
- ▶ ...

Aller plus loin

Bonne explication du module unittest :

<https://pymotw.com/3/unittest/>

Pour aller plus loin:

- ▶ découverte automatique de tests
- ▶ tearDown plus fiables
- ▶ code coverage et rapports
- ▶ ...

Cycle TDD (*Test Driven Development*)

1. écriture du test
2. erreur
3. écriture du code minimal pour passer le test
4. le test passe
5. retour à 1.

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Il existe d'autres modules pour lancer les tests ('testrunners')
29.

- ▶ (doctest ³⁰)
- ▶ nose ³¹
- ▶ pytest (allège la syntaxe des tests) ³²

Les tests sont souvent utilisés avec des 'mocks' ³³ pour modifier le comportement des modules externes.

29.<https://stackoverflow.com/questions/28408750/unittest-vs-pytest-vs-nose>

30.<https://docs.python.org/3.6/library/doctest.html>

31.<https://nose.readthedocs.io/en/latest/>

32.<https://docs.pytest.org/en/latest/>

33.<https://docs.python.org/3.6/library/unittest.mock.html>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

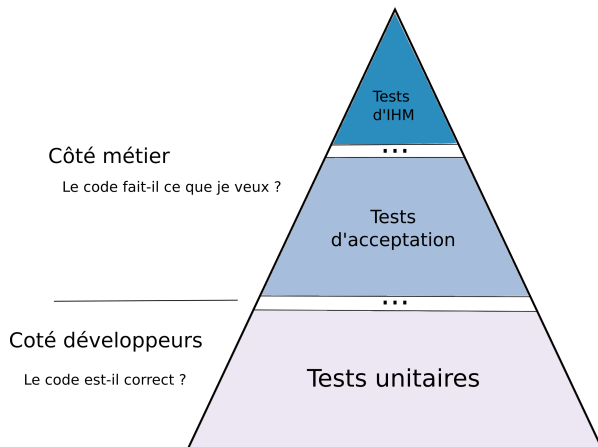
Création de
modules

Programmation
concurrente

Python scientifique

Aller plus loin – autres types de tests

Matthieu Falce



Inspiration :

<https://www.slideshare.net/RajIndugula/agile-testing-practices-38015016>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

4- Bonnes pratiques

4.7. Documentation

Documentation ?

- ▶ commentaires : donner des informations aux autres développeurs
- ▶ docstring : pour tout le monde

```
"""  
Une docstring pour le module / fichier ...  
Ici on décrit ce que doit faire le module  
"""
```

```
def spam(arg):  
    """  
    Une docstring pour la fonction  
  
    Params:  
        arg: int  
            Retourné par la fonction  
    """  
    # Attention : magique, ne pas toucher  
    return arg
```

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Pourquoi ?](#)

[Installation de paquets](#)

[Débug](#)

[Linter](#)

[Analyse des performances](#)

[Tests](#)

[Documentation](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Documentation ?

Matthieu Falce

- ▶ commentaires : donner des informations aux autres développeurs
- ▶ docstring : pour tout le monde

```
"""  
Une docstring pour le module / fichier ...  
Ici on décrit ce que doit faire le module  
"""
```

```
def spam(arg):  
    """  
    Une docstring pour la fonction  
  
    Params:  
        arg: int  
            Retourné par la fonction  
    """  
    # Attention : magique, ne pas toucher  
    return arg
```

Les docstrings sont traitées comme des objets python par l'interpréteur.

```
""" Show how to display docstrings in python."""  
# help(int)  
# print(int.__doc__)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Comment écrire sa documentation ?

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Pourquoi ?](#)

[Installation de paquets](#)

[Débug](#)

[Linter](#)

[Analyse des performances](#)

[Tests](#)

[Documentation](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Exemple minimal

```
def add(a, b):  
    """Addition for floats."""  
    return float(a + b)
```

Comment écrire sa documentation ?

Matthieu Falce

Exemple complet

```
"""
This module defines some operations on floating point numbers.
"""

def add_float(a, b):
    """
    Adds two numbers and casts them to float.

    Implements the binary function performing internal
    law of composition on floats.

    See:
    * https://en.wikipedia.org/wiki/Binary\_function
    * https://fr.wikipedia.org/wiki/Loi\_de\_composition\_interne

    Args:
        arg1(float): First number to sum
        arg2(float): Second number to sum

    Returns:
        float: Sum of the 2 arguments

    """
    return float(a + b)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Outils d'extraction de documentation

Matthieu Falce

- ▶ sphinx (semi automatique) avec :
 - ▶ autosummary³⁴
 - ▶ autodoc³⁵

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

34.<http://www.sphinx-doc.org/en/master/usage/extensions/autosummary.html>

35.<http://www.sphinx-doc.org/en/master/usage/extensions/autodoc.html>

Outils d'extraction de documentation

Matthieu Falce

- ▶ sphinx (semi automatique) avec :
 - ▶ autosummary³⁴
 - ▶ autodoc³⁵
- ▶ sphinx (automatique) avec :
 - ▶ autoapi³⁶
 - ▶ sphinx-autoapi³⁷

34.<http://www.sphinx-doc.org/en/master/usage/extensions/autosummary.html>

35.<http://www.sphinx-doc.org/en/master/usage/extensions/autodoc.html>

36.<http://autoapi.readthedocs.io/>

37.<http://sphinx-autoapi.readthedocs.io/en/latest/index.html>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Outils d'extraction de documentation

Matthieu Falce

- ▶ sphinx (semi automatique) avec :

- ▶ autosummary³⁴
- ▶ autodoc³⁵

- ▶ sphinx (automatique) avec :

- ▶ autoapi³⁶
- ▶ sphinx-autoapi³⁷

- ▶ pdoc³⁸

- ▶ pydoc³⁹

- ▶ doxygen⁴⁰

34.<http://www.sphinx-doc.org/en/master/usage/extensions/autosummary.html>

35.<http://www.sphinx-doc.org/en/master/usage/extensions/autodoc.html>

36.<http://autoapi.readthedocs.io/>

37.<http://sphinx-autoapi.readthedocs.io/en/latest/index.html>

38.<https://github.com/mitmproxy/pdoc>

39.<https://docs.python.org/3.6/library/pydoc.html>

40.<http://www.stack.nl/~dimitri/doxygen/>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Syntaxe pour extraction automatique

- ▶ PEP 8 : <https://www.python.org/dev/peps/pep-0008/#documentation-strings>
 - ▶ PEP 257 :
<https://www.python.org/dev/peps/pep-0257/>
-

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Syntaxe pour extraction automatique

Matthieu Falce

- ▶ PEP 8 : <https://www.python.org/dev/peps/pep-0008/#documentation-strings>
- ▶ PEP 257 : <https://www.python.org/dev/peps/pep-0257/>
- ▶ pdoc : markdown ⁴¹
- ▶ doxygen : markdown + syntaxe spécifique ⁴²
- ▶ sphinx : RestructuredText ⁴³
- ▶ sphinx avec extension Napoleon ⁴⁴
 - ▶ Google ⁴⁵
 - ▶ Numpy ⁴⁶

41.<https://help.github.com/articles/basic-writing-and-formatting-syntax/>

42.<https://www.stack.nl/~dimitri/doxygen/manual/docblocks.html>

43.https://thomas-cokelaer.info/tutorials/sphinx/rest_syntax.html

44.<http://www.sphinx-doc.org/en/master/usage/extensions/napoleon.html>

45.<https://github.com/google/styleguide/blob/gh-pages/pyguide.md>

46.<https://numpydoc.readthedocs.io/en/latest/format.html>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Formatage des docstrings – Doxygen

Matthieu Falce

```
## @package pyexample
# Documentation for this module.
#
# More details.

## Documentation for a function.
#
# More details.
def func():
    pass

## Documentation for a class.
#
# More details.
class PyClass:

    ## The constructor.
    def __init__(self):
        self._memVar = 0;

    ## Documentation for a method.
    # @param self The object pointer.
    def PyMethod(self):
        pass

    ## A class variable.
    classVar = 0;
    ## @var _memVar
    # a member variable
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Formatage des docstrings – Doxygen

Matthieu Falce

Python

[Main Page](#) | [Packages ▾](#) | [Classes ▾](#) | [pyexample Namespace Reference](#) Classes | Functions

Documentation for this module. [More...](#)

Classes

class `PyClass`
Documentation for a class. [More...](#)

Functions

def `func()`
Documentation for a function. [More...](#)

Detailed Description

Documentation for this module.

[More details.](#)

Function Documentation

[◆](#) `func()`

`def pyexample.func ( )`

Documentation for a function.

[More details.](#)

Generated by  1.8.15

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Pourquoi ?](#)

[Installation de paquets](#)

[Débug](#)

[Linter](#)

[Analyse des performances](#)

[Tests](#)

[Documentation](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Résultat HTML de l'exemple précédent

Formatage des docstrings – reST

```
"""
```

```
This is a reST style.
```

```
:param param1: this is a first param
```

```
:param param2: this is a second param
```

```
:returns: this is a description of what is returned
```

```
:raises KeyError: raises an exception
```

```
"""
```

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Formatage des docstrings – Google vs Numpy

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Pourquoi ?](#)

[Installation de paquets](#)

[Débug](#)

[Linter](#)

[Analyse des performances](#)

[Tests](#)

[Documentation](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

"""

This is an example of Google style.

Args:

param1 (array): This is the first param.

param2: This is a second param.

Returns:

*This is a description of what
is returned.*

Raises:

KeyError: Raises an exception.

"""

"""

This is an example of numpydoc style.

Parameters

param1 : array_like

This is the first param.

param2 :

This is a second param.

Returns

string

*This is a description of what
is returned.*

Raises

KeyError

when a key error

"""

Formatage des docstrings – Google vs Numpy

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Pourquoi ?](#)

[Installation de paquets](#)

[Débug](#)

[Linter](#)

[Analyse des performances](#)

[Tests](#)

[Documentation](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

```
exemple_docstring_simple.top_secret(param1, param2)
```

This is an example of Google style.

Parameters:

- **param1** – This is the first param.
- **param2** – This is a second param.

Returns:

This is a description of what is returned.

Raises:

`KeyError` – Raises an exception.

Résultat HTML de l'exemple précédent

Bibliographie

Matthieu Falce

► documentation

- <http://queirozf.com/entries/docstrings-by-example-documenting-python-code-the-right-way>
- <https://stackoverflow.com/questions/3898572/what-is-the-standard-python-docstring-format>
- <https://docs.python-guide.org/writing/documentation/>
- <https://fr.slideshare.net/shimizukawa/sphinx-autodoc-automated-api-documentation-europython-2015-in-bilbao>
- generation / formattage automatique des docstrings : <https://github.com/dadadel/pyment>

► *code formatters*

- sametmax.com/once-you-go-black-you-never-go-back/

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Pourquoi ?

Installation de paquets

Débug

Linter

Analyse des performances

Tests

Documentation

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

**Bibliothèque
standard**

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Bibliothèque standard

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

5- Bibliothèque standard

5.1. Batteries included

“Batteries included”

Python est un langage avec beaucoup de fonctionnalités incluses par défaut

- ▶ gestion de fichiers et des OS (lecture / écriture, compression, diff...)
- ▶ programmation réseau / parallèle / IPC / crypto ...
- ▶ multimédia (images, son, IHM)
- ▶ débogueur, tests unitaires...
- ▶ gestion des dates, traductions...

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

Batteries included

[Module sys](#)

[Module os](#)

[Mathématiques](#)

[Expressions régulières](#)

[Base de données](#)

[XML](#)

[JSON](#)

[Interaction réseau](#)

[Archivage des fichiers](#)

[Aller plus loin](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

“Batteries included”

Matthieu Falce

Python est un langage avec beaucoup de fonctionnalités incluses par défaut

- ▶ gestion de fichiers et des OS (lecture / écriture, compression, diff...)
- ▶ programmation réseau / parallèle / IPC / crypto ...
- ▶ multimédia (images, son, IHM)
- ▶ débogueur, tests unitaires...
- ▶ gestion des dates, traductions...

Il est aussi possible d'installer des modules tiers (très nombreux).

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

Batteries included

[Module sys](#)

[Module os](#)

[Mathématiques](#)

[Expressions régulières](#)

[Base de données](#)

[XML](#)

[JSON](#)

[Interaction réseau](#)

[Archivage des fichiers](#)

[Aller plus loin](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

5- Bibliothèque standard

5.2. Module sys

Manipulation des variables en lien avec l'interpréteur.

```
import sys

# affiche les paramètres passés lors de l'appel du script
# par ex : python gros_calcul.py fichier_entree.mat
print(sys.argv)

# avoir des infos sur les nombres flottants
print(sys.float_info)

# afficher / manipuler le path
print(sys.path)

# afficher l'OS
if sys.platform == "linux":
    print("Ouiiii")
elif sys.platform == "win32":
    print("Oui")

# manipuler les fichiers d'entrée / sortie / erreur
sys.stdin
sys.stdout
sys.stderr

# version de python
# utiliser platform plutôt
if sys.version.startswith("3."):
    print("youpi python3")
else:
    print(":'(')

# fermer le programme (optionnel)
sys.exit()
```

[Vue d'ensemble](#)[Langage Python](#)[Programmation
Orientée objet
\(POO\)](#)[Bonnes pratiques](#)[Bibliothèque
standard](#)[Batteries included](#)[Module sys](#)[Module os](#)[Mathématiques](#)[Expressions régulières](#)[Base de données](#)[XML](#)[JSON](#)[Interaction réseau](#)[Archivage des fichiers](#)[Aller plus loin](#)[Création de
modules](#)[Programmation
concurrente](#)[Python scientifique](#)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

5- Bibliothèque standard

5.3. Module os

Manipulation des variables en lien avec l'OS. Essaie d'avoir la même interface entre les différents OS.

```
import os

# accès aux variables d'environnement
print(os.environ)

# permet de modifier le dossier courant
os.chdir()

# lister un dossier
# utiliser "glob" pour les choses plus complexes
os.listdir(".")

# séparateur de fichiers
print(os.sep)

# créer un dossier (et ceux qui manquent entre)
os.makedirs("/tmp/test_os/super_test/", exist_ok=True)

# exécuter une commande
# pour les choses plus compliquées utiliser "subprocess"
commande = "ls /tmp"
os.system(commande)

# compter le nombre de CPU
print(os.cpu_count())
```

[Vue d'ensemble](#)[Langage Python](#)[Programmation
Orientée objet
\(POO\)](#)[Bonnes pratiques](#)[Bibliothèque
standard](#)[Batteries included](#)[Module sys](#)**Module os**[Mathématiques](#)[Expressions régulières](#)[Base de données](#)[XML](#)[JSON](#)[Interaction réseau](#)[Archivage des fichiers](#)[Aller plus loin](#)[Création de
modules](#)[Programmation
concurrente](#)[Python scientifique](#)

Manipulation des variables en lien avec l'OS. Essaie d'avoir la même interface entre les différents OS.

```
# permet de manipuler les chemins de fichiers
# depuis 3.4 on peut utiliser "pathlib"
# qui est plus haut niveau

from os

# ne pas avoir à manipuler les séparateurs de dossiers
print(os.path.join("/", "tmp", "test_os_path"))

# afficher des parties communes de fichiers
os.path.commonpath(['usr/lib', 'usr/local/lib'])

# normaliser les chemins
os.path.normpath(
    "/tmp/test_os_path/pas_ici/../../autre_test"
)

# avoir le dernier élément d'un chemin (fichier ?)
path, filename = os.path.split("/tmp/test_os_path/data.csv")

# faire l'expansion de l'utilisateur dans les chemins
expansion = os.path.expanduser(
    os.path.join("~", "test_os_path")
)
```

[Vue d'ensemble](#)[Langage Python](#)[Programmation
Orientée objet
\(POO\)](#)[Bonnes pratiques](#)[Bibliothèque
standard](#)[Batteries included](#)[Module sys](#)[Module os](#)[Mathématiques](#)[Expressions régulières](#)[Base de données](#)[XML](#)[JSON](#)[Interaction réseau](#)[Archivage des fichiers](#)[Aller plus loin](#)[Création de
modules](#)[Programmation
concurrente](#)[Python scientifique](#)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

5- Bibliothèque standard

5.4. Mathématiques

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Ne pas forcément utiliser ceux là pour les calculs
scientifiques.

Ils sont plus lents que ceux de numpy / scipy

Outils mathématiques

Matthieu Falce

```
import random, decimal, fractions, statistics

# nbs aléatoires
print(random.randint(1, 20))
print(random.random())

# choisir dans une liste
print("Jean Pierre, la réponse : ", random.choice(["a", "b", "c", "d"]))

# lois aléatoires...
print(random.lognormvariate(mu=10, sigma=2))
data = [random.uniform(1, 10) for _ in range(100)]
print("Moyenne", statistics.mean(data))
print("Ecart type", statistics.stdev(data))

D = decimal.Decimal
F = fractions.Fraction

# calculs exacts
fr = F(16, -10) # simplification
print(fr.numerator) # -8
print(F(1, 3) + F(1, 3) + F(1, 3))

print((1.1 + 2.2 - 3.3) * 1e19) # 4440.89...
print((D("1.1") + D("2.2") - D("3.3")) * int(1e19)) # 0
print((D(1.1) + D(2.2) - D(3.3)) * int(1e19)) # 1776.356839
```

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Batteries included](#)

[Module sys](#)

[Module os](#)

[Mathématiques](#)

[Expressions régulières](#)

[Base de données](#)

[XML](#)

[JSON](#)

[Interaction réseau](#)

[Archivage des fichiers](#)

[Aller plus loin](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

5- Bibliothèque standard

5.5. Expressions régulières

Expressions Régulières ?

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Chaîne de caractères, qui décrit, selon une syntaxe précise,
un ensemble de chaînes de caractères possibles

https://fr.wikipedia.org/wiki/Expression_r%C3%A9guli%C3%A8re

- ▶ tous les caractères sont valides
- ▶ quantificateurs (*, ?, +)
- ▶ opérateur de choix (a|b), listes de caractères [aeiou] et inversion de listes [^aeiou] ...
- ▶ caractères spéciaux (début de ligne : ^, fin de ligne : \$)
- ▶ ...

Vous pouvez les tester sur <https://regex101.com>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Exemples

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Batteries included](#)

[Module sys](#)

[Module os](#)

[Mathématiques](#)

[Expressions régulières](#)

[Base de données](#)

[XML](#)

[JSON](#)

[Interaction réseau](#)

[Archivage des fichiers](#)

[Aller plus loin](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Expression	Chaînes capturées	Chaînes non capturées
ab	ab	a / b / ""
a b	a / b	ab / c / ...
a+	a / aa / aaaa...aa	"" / ab / b
a?	"" / a	aa / aaa..aa / ab / b
a*	"" / a / aa / aaaa...aa	ab / b
a	*a	tout le reste
[aeiou]	a / e / ...	"" / ae / z

Exemples

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Batteries included](#)

[Module sys](#)

[Module os](#)

[Mathématiques](#)

[Expressions régulières](#)

[Base de données](#)

[XML](#)

[JSON](#)

[Interaction réseau](#)

[Archivage des fichiers](#)

[Aller plus loin](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Expression	Chaînes capturées	Chaînes non capturées
[^aeiou]	b / r / ... / 9 / -	"" / a / bc
a{1,3}	a / aa / aaa	tout le reste
[aeiou]	a / e / ...	"" / ae / z
ex - (a?e æ é)quo	ex-equo, ex-aequo, ex-équo et ex-æquo	ex-quo, ex-aiquo, ex-aeko, ex-æéquo
^Section .+	Section 1 / Section a / Section a.a/2	"" / Sectionner / voir Section 1
[1234567890]+ (,[1234567890]+)?	2 / 42 / 2,32 / 0.432	3, / ,643 / ""

Cas d'usages

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Quand les utiliser :

- ▶ traitements complexes
- ▶ tolérance sur des chaînes en entrée
- ▶ si le framework vous y oblige

Quand les utiliser :

- ▶ traitements complexes
- ▶ tolérance sur des chaînes en entrée
- ▶ si le framework vous y oblige

Quand ne pas les utiliser :

- ▶ traitements simples (plutôt outils du langage)
- ▶ *parsing* compliqué (plutôt des outils sur des grammaires)

En python

Matthieu Falce

Python rajoute des caractères spéciaux pour des cas courants :

- ▶ `\w` : tous les caractères alphanumériques et underscore (`[A-Za-z0-9_]`)
- ▶ `\W` : ni caractères alphanumériques ni underscore (`^[A-Za-z0-9_]`)
- ▶ `\d` : chiffres (0-9)
- ▶ `\D` : autre chose qu'un chiffre (`^0-9`)
- ▶ `\s` : séparateur de texte (`[\t \r \n \v \f]`)
- ▶ `\S` : non séparateur de texte (`^[ \t \r \n \v \f]`)
- ▶ `\b` : début ou fin de mot (attention il FAUT utiliser des "rawstrings" pour que ça marche)

<https://regex101.com> permet d'exporter le code python correspondant à vos expressions

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

En python

Matthieu Falce

```
import re
```

```
regex = r"ch?at"
```

```
assert re.search(regex, "chat") is not None
```

```
assert re.search(regex, "cat") is not None
```

```
assert re.search(regex, "chien") is None
```

```
# match vs search
```

```
assert re.match(regex, "le chat") is None
```

```
assert re.search(regex, "le chat") is not None
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

En python

Matthieu Falce

```
import re
```

```
regex = "(?P<bien>\w*) c'est bien, (?P<mieux>\w*) c'est mieux"  
test_string = "Python c'est bien, Perl c'est mieux"
```

```
searched = re.search(regex, test_string)  
assert searched.groupdict() == {"bien": "Python", "mieux": "Perl"}
```

```
# si la regex ne trouve rien, re.search vaut None  
test_string = "Python 2 c'est bien, Python 3 c'est mieux"  
assert re.search(regex, test_string) is None
```

```
# on modifie la regex pour gérer le nouveau cas  
regex = "(?P<bien>[\w\s.]*) c'est bien, (?P<mieux>[\w\s.]*) c'est mieux"  
test_string = "Python 2.7 c'est bien, Python 3.6 c'est mieux"  
searched = re.search(regex, test_string)  
assert searched.groupdict() == {"bien": "Python 2.7", "mieux": "Python 3.6"}
```

```
# comment faire quand il y a plusieurs match dans la chaîne  
multiple = re.findall("ch?at", "chat -- dog -- cat")  
assert multiple == ["chat", "cat"]
```

```
# python_version_pattern = "Python (?P<major>\d*).(P<minor>\d*)" "  
# test_string = "Python 2.4 -- Python 3.5 -- Python 0.11 -- Python 32.34224"  
# searched = re.findall(regex, test_string)  
# assert searched == [('2', '4'), ('3', '5'), ('0', '11'), ('32', '34224')]
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

5- Bibliothèque standard

5.6. Base de données

- ▶ Python permet de se connecter à des bases de données
- ▶ Normalisation avec la DB API (database API) ⁴⁷
 - ▶ comme un pilote d'imprimante $\Rightarrow$ on lui dit ce qu'on veut imprimer, il s'occupe des spécificités
 - ▶ augmente la compréhension du code
 - ▶ facilite le changement de SGBD
 - ▶ inspirée de Open Database Connectivity (ODBC) et Java Database Connectivity (JDBC)

47.<https://www.python.org/dev/peps/pep-0249/>

Présentation DB API

Matthieu Falce

Avec SQLite

```
import sqlite3

print("Paramstyle:", sqlite3.paramstyle) # Paramstyle: qmark

# connexion à la base et récupération du curseur
db = sqlite3.connect(':memory:')
cursor = db.cursor()
cursor.execute("""
    CREATE TABLE IF NOT EXISTS users(
        id INTEGER PRIMARY KEY AUTOINCREMENT UNIQUE,
        name TEXT,
        age INTEGER)
""")

# On applique les modifications avec commit
db.commit()

cursor.execute("""INSERT INTO users(name, age) VALUES(?, ?)""", ("matthieu", 323))
db.commit()

cursor.execute('SELECT * FROM users;')
# récupérer le premier
user1 = cursor.fetchone()
print(user1) # (1, 'matthieu', 323)

# on ferme tout à la fin
cursor.close()
db.close()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Présentation DB API

Avec Mysql

```
# avant d'installer avec pip faire: sudo apt install libmysqlclient-dev
# sur windows, il y a un wheel avec les bons binaires
import MySQLdb

print("Paramstyle:", MySQLdb.paramstyle) # Paramstyle: format

# connexion à la base et récupération du curseur
# pas de mot de passe et compte root de MySQL, ne faites pas ça...
db = MySQLdb.connect(host="127.0.0.1", user="root", db="formation")
cursor=db.cursor()
cursor.execute("""
    CREATE TABLE IF NOT EXISTS users(
        id INTEGER PRIMARY KEY AUTO_INCREMENT UNIQUE,
        name TEXT,
        age INTEGER);
""")

# On applique les modifications avec commit
db.commit()

cursor.execute("""INSERT INTO users(name, age) VALUES(%s, %s);""", ("matthieu", 323))
db.commit()

cursor.execute('''SELECT * FROM users;''')
# récupérer le premier
user1 = cursor.fetchone()
print(user1) # (1, 'matthieu', 323)

# on ferme tout à la fin
cursor.close()
db.close()
```

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Présentation DB API

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

En résumé

- ▶ même structure et méthodes appelées
- ▶ différence de syntaxe des paramètres
- ▶ différences au niveau du SQL supporté...
- ▶ si l'on ne commit pas on ne stocke pas les données en base
 - ▶ curseurs globaux à une connexion $\Rightarrow$ données potentiellement non enregistrées accessibles

Insérer / récupérer des données

Matthieu Falce

```
import sqlite3
```

```
db = sqlite3.connect(':memory:')
cursor = db.cursor()
cursor.execute("""
    CREATE TABLE IF NOT EXISTS users(
        id INTEGER PRIMARY KEY AUTOINCREMENT UNIQUE, name TEXT, age INTEGER)
""")
db.commit()
```

```
# insérer des données en mode batch
```

```
users = [
    ("olivier", 30), ("jean-louis", 90), ("luc", 32),
    ("matthieu", 24), ("pierre", 54), ("françois", 78)
]
cursor.executemany("""
INSERT INTO users(name, age) VALUES(?, ?)""", users)
```

```
# récupérer toutes les données
```

```
print("----- Tous -----")
cursor.execute("""SELECT id, name, age FROM users""")
rows = cursor.fetchall()
for row in rows:
    print('{0} : {1} - {2}'.format(row[0], row[1], row[2]))
```

```
# récupérer une sélection les données
```

```
print("----- Selection -----")
cursor.execute("""SELECT id, name, age FROM users WHERE age > 30""")
for row in cursor.fetchall():
    print('{0} : {1} - {2}'.format(row[0], row[1], row[2]))
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Supprimer / mettre à jour des données

Matthieu Falce

```
import sqlite3
```

```
db = sqlite3.connect(':memory:')
cursor = db.cursor()
cursor.execute("""
    CREATE TABLE IF NOT EXISTS users(
        id INTEGER PRIMARY KEY AUTOINCREMENT UNIQUE, name TEXT, age INTEGER)
""")
db.commit()
```

```
# insérer des données en mode batch
```

```
users = [
    ("olivier", 30), ("jean-louis", 90), ("luc", 32),
    ("matthieu", 24), ("pierre", 54), ("françois", 78)
]
cursor.executemany("""INSERT INTO users(name, age) VALUES(?, ?)""", users)
db.commit()
```

```
# on va modifier les jeunes pour leur rajouter un préfixe
```

```
# || pour concaténer des chaînes en SQLite
```

```
cursor.execute("""UPDATE users SET name = name || ' Jr' WHERE age < 30 ;""")
db.commit()
```

```
# on va supprimer les gens qui ont un nom de plus de 5 caractères
```

```
cursor.execute("""DELETE FROM users WHERE length(name)>6 ;""")
db.commit()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Erreurs et exceptions

Matthieu Falce

Taxonomie des exceptions d'après la PEP 249

StandardError

 |__Warning

 |__Error

 |__InterfaceError

 |__DatabaseError

 |__DataError

 |__OperationalError

 |__IntegrityError

 |__InternalError

 |__ProgrammingError

 |__NotSupportedError

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Batteries included](#)

[Module sys](#)

[Module os](#)

[Mathématiques](#)

[Expressions régulières](#)

[Base de données](#)

[XML](#)

[JSON](#)

[Interaction réseau](#)

[Archivage des fichiers](#)

[Aller plus loin](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Erreurs et exceptions

Matthieu Falce

Quelles données en base à la fin du script ?

```
import sqlite3

db = sqlite3.connect('/tmp/test.db')
cursor = db.cursor()
cursor.execute("""
    CREATE TABLE IF NOT EXISTS users(
        id INTEGER PRIMARY KEY AUTOINCREMENT UNIQUE, name TEXT UNIQUE, age INTEGER)
""")
db.commit()

# utilisateurs avec des noms identiques
users = [("matthieu", 30), ("matthieu", 90)]

try:
    for user in users:
        cursor.execute("""INSERT INTO users(name, age) VALUES(?, ?)""", user)
except sqlite3.IntegrityError as e:
    print("Integrity Error, roll back")
    db.rollback()
finally:
    # Close the db connection
    db.commit()
    db.close()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Erreurs et exceptions

Matthieu Falce

Quelles données en base à la fin du script ?

```
import sqlite3

db = sqlite3.connect('/tmp/test.db')
cursor = db.cursor()
cursor.execute("""
    CREATE TABLE IF NOT EXISTS users(
        id INTEGER PRIMARY KEY AUTOINCREMENT UNIQUE, name TEXT UNIQUE, age INTEGER)
    """)
db.commit()

# utilisateurs avec des noms identiques
users = [("matthieu", 30), ("matthieu", 90)]

try:
    for user in users:
        cursor.execute("""INSERT INTO users(name, age) VALUES(?, ?)""", user)
        db.commit()
except sqlite3.IntegrityError as e:
    print("Integrity Error, roll back")
    db.rollback()
finally:
    # Close the db connection
    db.commit()
    db.close()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Bibliographie / Aller plus loin

Matthieu Falce

- ▶ <https://wiki.python.org/moin/DbApiCheatSheet>
- ▶ <http://sweetohm.net/article/python-dbapi.html>
- ▶ <https://apprendre-python.com/page-database-data-base-donnees-query-sql-mysql-postgre-sqlite>
- ▶ <https://www.sqlitetutorial.net/sqlite-python/>
- ▶ comment gérer le *multithreading* ?
 - ▶ curseurs non *thread safe*
 - ▶ une connexion par thread
- ▶ ORM ⁴⁸ $\Rightarrow$ abstraire les différences entre moteurs
 - ▶ SQLAlchemy
 - ▶ Pewee
 - ▶ PonyORM
 - ▶ ORM Django

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

48.<https://www.fullstackpython.com/object-relational-mappers-orms.html>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

5- Bibliothèque standard

5.7. XML

```
import xml.etree.cElementTree as ET
```

```
# écriture
```

```
root = ET.Element("root")
```

```
doc = ET.SubElement(root, "doc")
```

```
ET.SubElement(doc, "field1", name="blah").text = "some value1"
```

```
ET.SubElement(doc, "field2", name="asdfasd").text = "some value2"
```

```
tree = ET.ElementTree(root)
```

```
tree.write("filename.xml")
```

[Vue d'ensemble](#)[Langage Python](#)[Programmation
Orientée objet
\(POO\)](#)[Bonnes pratiques](#)[Bibliothèque
standard](#)[Batteries included](#)[Module sys](#)[Module os](#)[Mathématiques](#)[Expressions régulières](#)[Base de données](#)[XML](#)[JSON](#)[Interaction réseau](#)[Archivage des fichiers](#)[Aller plus loin](#)[Création de
modules](#)[Programmation
concurrente](#)[Python scientifique](#)

```
import io
from xml.dom import minidom

# lecture d'un XML

data = """
<data> <items>
    <item name="item1"></item> <item name="item2"></item>
    <item name="item3"></item> <item name="item4"></item>
</items></data>"""

# parse attend un fichier, on crée un StringIO pour le duper

file_like_from_str = io.StringIO(data)
xmldoc = minidom.parse(file_like_from_str)
itemlist = xmldoc.getElementsByTagName('item')
print(len(itemlist))
print(itemlist[0].attributes['name'].value)
for s in itemlist:
    print(s.attributes['name'].value)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modulesProgrammation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

5- Bibliothèque standard

5.8. JSON

JSON

Matthieu Falce

```
import json
```

```
# créer un JSON
```

```
donnees_test = {  
    "chaîne": "dictionnaire",  
    "liste": [1, 2, 3]  
}
```

```
# crée le fichier test.json
```

```
json.dump(donnees_test, open("test.json", "w"))
```

```
# stocke le résultat dans une chaîne
```

```
representation_json = json.dumps(donnees_test)
```

```
# lire un json
```

```
# depuis un fichier
```

```
data = json.load(open("test.json"))
```

```
# depuis une chaîne
```

```
data2 = json.loads(representation_json)
```

```
assert data == donnees_test
```

```
assert data2 == donnees_test
```

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Batteries included](#)

[Module sys](#)

[Module os](#)

[Mathématiques](#)

[Expressions régulières](#)

[Base de données](#)

[XML](#)

JSON

[Interaction réseau](#)

[Archivage des fichiers](#)

[Aller plus loin](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)



Certaines données ne sont pas JSON sérialisables. Il faut créer son propre serialiseur JSON dans ce cas. ⁵⁰

```
from json import dumps
from datetime import date, datetime

def json_serial(obj):
    """JSON serializer for objects not serializable
    by default json code"""
    if isinstance(obj, (datetime, date)):
        return obj.isoformat()
    raise TypeError("Type %s not serializable" % type(obj))

print(dumps(datetime.now(), default=json_serial))
```

50.<https://stackoverflow.com/questions/11875770/how-to-overcome-datetime-datetime-not-json-serializable>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modulesProgrammation
concurrente

Python scientifique

CSV – excel

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Batteries included](#)

[Module sys](#)

[Module os](#)

[Mathématiques](#)

[Expressions régulières](#)

[Base de données](#)

[XML](#)

[JSON](#)

[Interaction réseau](#)

[Archivage des fichiers](#)

[Aller plus loin](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

```
#####  
# version quick and dirty  
  
# écrire  
data = [[1, 2], [3, 4, 5]]  
open("eggs.csv", "w").write(  
    "\n".join(["\t".join(map(str, line)) for line in data])  
)  
  
# lire  
data = [line.strip().split("\t") for line in open("eggs.csv", "r")]
```

CSV – excel

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Batteries included](#)

[Module sys](#)

[Module os](#)

[Mathématiques](#)

[Expressions régulières](#)

[Base de données](#)

[XML](#)

[JSON](#)

[Interaction réseau](#)

[Archivage des fichiers](#)

[Aller plus loin](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

```
import csv
```

```
# écrire le fichier
```

```
data = [  
    ["Spam"] * 5 + ["Baked Beans"],  
    ['Spam', 'Lovely Spam', 'Wonderful Spam'],  
    ["Avec des accents éàù", "ça marche"]  
]
```

```
with open('eggs.csv', 'w') as csvfile:  
    spamwriter = csv.writer(  
        csvfile, delimiter=' ',  
        quotechar='|', quoting=csv.QUOTE_MINIMAL  
    )  
    for row in data:  
        spamwriter.writerow(row)
```

```
# lire le fichier
```

```
with open('eggs.csv', 'r') as csvfile:  
    spamreader = csv.reader(csvfile, delimiter=' ', quotechar='|')  
    for row in spamreader:  
        print(' '.join(row))
```


On peut utiliser `xlrd`, `openpyxl` ou `pandas` (qui se base sur ces dernières) ⁵¹

```
# pip install pandas xlrd openpyxl
import pandas as pd

xl = pd.ExcelFile("./fichiers_a_lire/excel_plusieurs_feuilles.xlsx")
names = xl.sheet_names

df = xl.parse(names[0])
df2 = xl.parse(names[1])
print(df.head())
print(df2.head())

df = pd.read_excel("./fichiers_a_lire/excel_une_feuille.xlsx")
print(df.head())

# écrire
df.to_excel(
    'fichiers_a_lire/test.xlsx',
    sheet_name='sheet1',
    index=False
)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modulesProgrammation
concurrente

Python scientifique

⁵¹<http://www.python-excel.org/>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

5- Bibliothèque standard

5.9. Interaction réseau

Avec la lib standard

```
# pip install requests
import urllib.request
import urllib.parse
import pprint, json

urlopen = urllib.request.urlopen

url = 'https://httpbin.org/anything'

values = {
    'name': 'Michael Foord',
    'location': 'Northampton',
    'language': 'Python'
}

# requête get simple
with urlopen(url) as response:
    pprint.pprint(json.loads(response.read()))

# GET avec paramètres
url_values = urllib.parse.urlencode(values)
full_url = url + '?' + url_values
with urlopen(full_url) as response:
    pprint.pprint(json.loads(response.read()))

# requête post avec paramètres
data = urllib.parse.urlencode(values)
data = data.encode('ascii') # data should be bytes
req = urllib.request.Request(url, data)
with urlopen(req) as response:
    pprint.pprint(json.loads(response.read()))
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modulesProgrammation
concurrente

Python scientifique

Avec requests

```
# pip install requests
import requests
import pprint

url = 'https://httpbin.org/anything'

values = {
    'name': 'Michael Foord',
    'location': 'Northampton',
    'language': 'Python'
}

# requête GET simple
r = requests.get(url)
pprint.pprint(r.json())

# requête GET avec paramètres
r = requests.get(url, data=values)
pprint.pprint(r.json())

# requête POST avec paramètres
r = requests.post(url, data=values)
pprint.pprint(r.json())
```

[Vue d'ensemble](#)[Langage Python](#)[Programmation
Orientée objet
\(POO\)](#)[Bonnes pratiques](#)[Bibliothèque
standard](#)[Batteries included](#)[Module sys](#)[Module os](#)[Mathématiques](#)[Expressions régulières](#)[Base de données](#)[XML](#)[JSON](#)[Interaction réseau](#)[Archivage des fichiers](#)[Aller plus loin](#)[Création de
modules](#)[Programmation
concurrente](#)[Python scientifique](#)

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Batteries included](#)

[Module sys](#)

[Module os](#)

[Mathématiques](#)

[Expressions régulières](#)

[Base de données](#)

[XML](#)

[JSON](#)

[Interaction réseau](#)

[Archivage des fichiers](#)

[Aller plus loin](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

On peut aussi lancer un serveur web vite fait sur sa machine :

```
python -m http.server
```

Sockets

Matthieu Falce

```
# Requête HTTP à la main
```

```
# exemple socket client
```

```
import socket
```

```
HOST = 'google.com'      # The remote host
PORT = 80                # The same port as used by the server
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(
        b"GET / HTTP/1.1\r\nHost: google.com\r\n\r\n"
    )
    data = s.recv(1024)
print('Received', repr(data))
```

```
#=====
```

```
# Echo server program
```

```
# test avec `echo -en "l\n2\n" | nc localhost 50007 -q1`
HOST = ''                # Symbolic name meaning all available interfaces
PORT = 50007             # Arbitrary non-privileged port
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen(1)
    conn, addr = s.accept()
    with conn:
        print('Connected by', addr)
        while True:
            data = conn.recv(1024)
            if not data:
                break
            conn.sendall(data)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

5- Bibliothèque standard

5.10. Archivage des fichiers

Manipulation de fichiers (archivage)

Matthieu Falce

```
import zipfile

# créer une archive
filename = "test_zip.py"
with zipfile.ZipFile('example.zip', mode='w') as zf:
    print('adding ', filename)
    zf.write(filename)

# lister les fichiers d'une archive
with zipfile.ZipFile('example.zip', 'r') as zf:
    print(zf.namelist())

# extraire les fichiers d'une archive
with zipfile.ZipFile('example.zip') as zf:
    for filename in [filename, 'notthere.txt']:
        try:
            data = zf.read(filename)
        except KeyError:
            print('ERROR: Did not find {} in zip file'.format(
                filename))
        else:
            print(filename, ':')
            print(data)
    print()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

Manipulation de fichiers (archivage)

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Batteries included](#)

[Module sys](#)

[Module os](#)

[Mathématiques](#)

[Expressions régulières](#)

[Base de données](#)

[XML](#)

[JSON](#)

[Interaction réseau](#)

[Archivage des fichiers](#)

[Aller plus loin](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

```
from shutil import make_archive
import os
```

```
archive_name = os.path.expanduser(os.path.join('~', 'myarchive'))
root_dir = os.path.expanduser(os.path.join('~', '.ssh'))
make_archive(archive_name, 'gztar', root_dir)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Batteries included

Module sys

Module os

Mathématiques

Expressions régulières

Base de données

XML

JSON

Interaction réseau

Archivage des fichiers

Aller plus loin

Création de
modules

Programmation
concurrente

Python scientifique

5- Bibliothèque standard

5.11. Aller plus loin

Autres modules intéressants I

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Batteries included](#)

[Module sys](#)

[Module os](#)

[Mathématiques](#)

[Expressions régulières](#)

[Base de données](#)

[XML](#)

[JSON](#)

[Interaction réseau](#)

[Archivage des fichiers](#)

[Aller plus loin](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

La librairie standard regorge de modules intéressants en plus des précédents (“python is batteries included”).

Autres modules intéressants II

Matthieu Falce

En voici quelques un :

- ▶ **copy** : copie les objets, récursivement (utile pour les conteneurs et objets custom)
- ▶ **logging** : permet d'effectuer le logging des applications. Extrêmement complet
- ▶ **datetime** : permet de gérer les dates (additions, parsing...), des alternatives tierces existent pour les cas complexes
- ▶ **argparse** : permet de gérer les arguments en ligne de commande (des alternatives tierces plus complètes existent)
- ▶ **functools** : permet de manipuler les fonctions d'ordres supérieurs
- ▶ **itertools** : permet de manipuler les itérables et de faciliter les constructions paresseuses

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Batteries included](#)

[Module sys](#)

[Module os](#)

[Mathématiques](#)

[Expressions régulières](#)

[Base de données](#)

[XML](#)

[JSON](#)

[Interaction réseau](#)

[Archivage des fichiers](#)

[Aller plus loin](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

**Création de
modules**

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

Création de modules

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

6- Création de modules

6.1. Setuptools / Distutils

Permettent d'empaqueter un module python.

Coexistence de **Setuptools** et **Distutils** → confusion

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Setuptools / Distutils](#)

[Structure d'un module](#)

[Installation](#)

[Mise en ligne PyPI](#)

[Bibliographie](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Permettent d'emballer un module python.

Coexistence de **Setuptools** et **Distutils** → confusion

On va utiliser setuptools

https:

[//stackoverflow.com/questions/25337706/setuptools-vs-distutils-why-is-distutils-still-a-thing](https://stackoverflow.com/questions/25337706/setuptools-vs-distutils-why-is-distutils-still-a-thing)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

Permettent d'emballer un module python.

Coexistence de **Setuptools** et **Distutils** → confusion

On va utiliser setuptools

https:

[//stackoverflow.com/questions/25337706/setuptools-vs-distutils-why-is-distutils-still-a-thing](https://stackoverflow.com/questions/25337706/setuptools-vs-distutils-why-is-distutils-still-a-thing)

Arborescence et fichiers spécifiques à respecter

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

6- Création de modules

6.2. Structure d'un module

Structure d'un module

Code (`__init__.py`) :

```
def joke():  
    return (  
        'Tu connais la blague du petit dej ? \n'  
        'Pas de bol'  
    )
```

Arborescence :

```
minimal/  
    funniest/  
        __init__.py  
    setup.py
```

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Setuptools / Distutils](#)

[Structure d'un module](#)

[Installation](#)

[Mise en ligne PyPI](#)

[Bibliographie](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Structure d'un module

Code (`__init__.py`) :

```
def joke():  
    return (  
        'Tu connais la blague du petit dej ? \n'  
        'Pas de bol'  
    )
```

Arborescence :

```
minimal/  
    funniest/  
        __init__.py  
    setup.py
```



Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Setuptools / Distutils](#)

[Structure d'un module](#)

[Installation](#)

[Mise en ligne PyPI](#)

[Bibliographie](#)

[Programmation
concurrente](#)

[Python scientifique](#)

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Setuptools / Distutils](#)

[Structure d'un module](#)

[Installation](#)

[Mise en ligne PyPI](#)

[Bibliographie](#)

[Programmation
concurrente](#)

[Python scientifique](#)

dossier du projet		minimal/
dossier du code		funniest/
code du programme		__init__.py
configuration		setup.py

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

6- Création de modules

6.3. Installation

Installation

Matthieu Falce

Contenu de setup.py

```
from setuptools import setup
```

```
setup(  
    name='funniest',  
    version='0.1',  
    description='Super blague',  
    url='http://example.com/ahahah/',  
    author='Matthieu Falce',  
    author_email='clown@example.com',  
    license='MIT',  
    packages=['funniest'],  
    zip_safe=False  
)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

Installation avec pip

PIP va se charger des copies → endroits connus dans le PYTHONPATH

```
# on se place à l'endroit du setup.py
pip install .      # mode normal

# mode ''édition'': évite de réinstaller en continue
# quand on modifie le module
pip install -e .
```

Utilisation du module

Depuis n'importe où sur l'ordinateur

```
import funniest
print(funniest.joke)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

Exemple plus complet

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

- ▶ déclaration des dépendances
- ▶ programme plus gros
- ▶ utilisation de fichiers “autres” (données...)
- ▶ points d'entrées

Exemple plus complet

Arborescence :

```
tree complet --charset=ASCII -I "__pycache__"
```

```
complet
|-- funniest
|   |-- command_line.py
|   |-- data
|   |   `-- blagues.txt
|   |-- __init__.py
|   |-- texput.log
|   `-- text.py
|-- MANIFEST.in
`-- setup.py
```

2 directories, 7 files

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Setuptools / Distutils](#)

[Structure d'un module](#)

[Installation](#)

[Mise en ligne PyPI](#)

[Bibliographie](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Exemple plus complet

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Setuptools / Distutils](#)

[Structure d'un module](#)

[Installation](#)

[Mise en ligne PyPI](#)

[Bibliographie](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Contenu de text.py :

```
from pathlib import Path
from markdown import markdown
import os

# test lecture fichiers de données
module_path = Path(
    os.path.dirname(os.path.abspath(__file__))
)
print(open(module_path / "data/blagues.txt").readlines())

def joke():
    return markdown(
        'Tu connais la *blague du petit dej* ? \n'
        '_Pas de bol_'
    )
```

Exemple plus complet

Matthieu Falce

Contenu de
__init__.py :

```
from .text import joke
```

Contenu de
command_line.py :

```
import funniest

def main():
    print(funniest.joke())
```

Contenu de blagues.txt :

```
"Un jour Dieu dit à Casto de ramer. Et depuis, castorama..."
```

Contenu de MANIFEST.in (pour les fichiers statiques) :

```
# Include the data files
include funniest/data/blagues.txt
```

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Setuptools / Distutils](#)

[Structure d'un module](#)

[Installation](#)

[Mise en ligne PyPI](#)

[Bibliographie](#)

[Programmation
concurrente](#)

[Python scientifique](#)

Exemple plus complet

Matthieu Falce

Contenu de setup.py :

```
from setuptools import setup

setup(
    name='funniest_bLyR4',
    version='0.1',
    description='Super blague / Utilisé dans des formations',
    url='http://example.com/ahahah/',
    author='Matthieu Falce',
    author_email='clown@example.com',
    packages=['funniest'],
    license='MIT',

    install_requires=[
        'markdown',
    ],
    entry_points = { # commandes qui seront installées
        'console_scripts': [
            'funniest-joke=funniest.command_line:main'
        ],
    },
    include_package_data=True,
    zip_safe=False
)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

6- Création de modules

6.4. Mise en ligne PyPI

Qui / Où ?

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

- ▶ tout le monde peut téléverser sur PyPI
- ▶ il y a une plateforme de test : <https://test.pypi.org/>
(que nous allons utiliser)
- ▶ il faut créer un compte et le valider

Qui / Où ?

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

On peut créer un fichier qui va contenir le mot de passe du compte (pour ne pas le retaper).
A placer dans `~/.pypirc`

```
[distutils]
index-servers=
    testpypi
    pypi

[testpypi]
repository = https://test.pypi.org/legacy/
username = name_of_the_user
password = hunter2

[pypi]
repository = https://pypi.python.org/pypi
username = name_of_the_user
password = hunter2
```

Source : <https://blog.jetbrains.com/pycharm/2017/05/how-to-publish-your-package-on-pypi/>

Pas à pas

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

```
# on installe twine qui va servir à faire l'upload  
pip install twine
```

```
# on construit les sdist et bdist_wheel  
python setup.py sdist bdist_wheel
```

```
# on upload tout dist avec twine, en prenant la  
# configuration de testpypi  
twine upload -r testpypi dist/*
```

```
# on peut installer dans un autre venv  
# j'ai dû changer le nom du projet pour pouvoir l'uploader  
pip install \\\n  --index-url https://test.pypi.org/simple/ \\\n  --default-timeout=100  
funniest_bLyR4
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

6- Création de modules

6.5. Bibliographie

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Setuptools / Distutils

Structure d'un module

Installation

Mise en ligne PyPI

Bibliographie

Programmation
concurrente

Python scientifique

► Packaging / installation

- Informations packaging officielles
- Exemple officiel (PyPA) de setup.py
- Exemple de packaging de A à Z
- Exemple d'utilisation de pyinstaller de A à Z

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

**Programmation
concurrente**

Avant propos / définitions

GIL

Parallélisation

Bibliographie

Python scientifique

Programmation concurrente

7- Programmation concurrente

7.1. Avant propos / définitions

Programmation concurrente

Les programmes concurrents permettent d'exécuter plusieurs tâches en même temps.

Programmation concurrente

Les programmes concurrents permettent d'exécuter plusieurs tâches en même temps.

Enfin, plus ou moins, mais à l'échelle humaine ils semblent tous faire plusieurs choses en même temps...

Programmation concurrente

Les programmes concurrents permettent d'exécuter plusieurs tâches en même temps.

Enfin, plus ou moins, mais à l'échelle humaine ils semblent tous faire plusieurs choses en même temps...

- ▶ programmation parallèle : on effectue des opérations en parallèle (sur CPU ou GPU)
- ▶ programmation concurrente : plus générale que le parallélisme, on a plusieurs entités indépendantes (un client et un serveur)

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Avant propos / définitions](#)

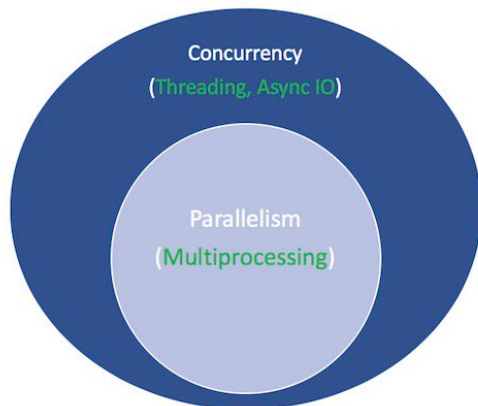
[GIL](#)

[Parallélisation](#)

[Bibliographie](#)

[Python scientifique](#)

Illustration



Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions

GIL

Parallélisation

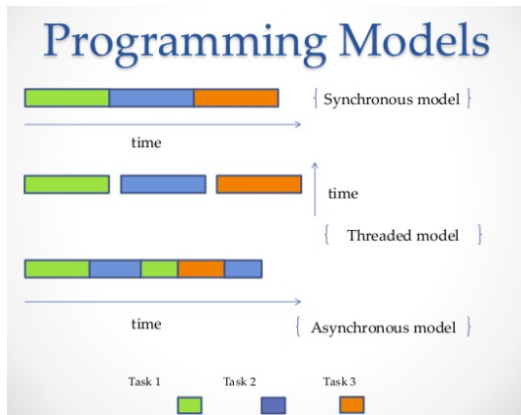
Bibliographie

Python scientifique

Source : <https://realpython.com/async-io-python/>

Illustration

Matthieu Falce



Source :

<https://medium.com/velotio-perspectives/an-introduction-to-asynchronous-programming-in-python-af0189a88bbb>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions

GIL

Parallélisation

Bibliographie

Python scientifique

Probablement l'un des plus gros chantiers de Python récemment :

- ▶ on peut faire les deux : `threding`, `asncio`, `multiprocess`
- ▶ semble complexe à mettre en place
- ▶ problème du GIL
- ▶ introduction en marche forcée de *async* (pour ne pas être à la traîne par rapport aux langages à la mode)

Quand utiliser quoi ?

Matthieu Falce

Dans tous les cas, les programmes devant interagir avec l'extérieur doivent être concurrents d'une façon où d'une autre.

- ▶ si forte utilisation CPU : paralléliser le code sur plusieurs coeurs / processeurs / machines
 - ▶ exemple culinaire : être plusieurs à peler des patates
 - ▶ les processus sont indépendants
 - ▶ nécessite un moyen d'échanger les données entre processus (réseaux, pipes, bdd...)
- ▶ si forte utilisation d'IO : privilégier les threads (processus légers) ou les coroutines (asynchrone)
 - ▶ exemple culinaire : peler les tomates pendant la cuisson des pâtes
 - ▶ très sensible aux performances et aux appels longs
 - ▶ un seul processus donc on peut utiliser des techniques *classiques* de partage de données

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions

GIL

Parallélisation

Bibliographie

Python scientifique

7- Programmation concurrente

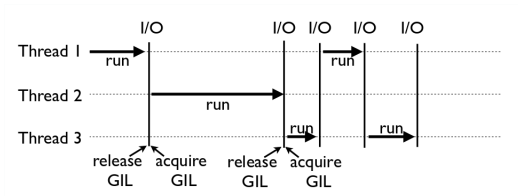
7.2. GIL

Global Interpreter Lock

- ▶ simplicité de l'interpréteur Python
- ▶ bloque l'exécution de plusieurs threads à la fois
 - ▶ la thread qui s'exécute bloque le GIL
 - ▶ le GIL est relâché sur les opérations d'IO / périodiquement

Global Interpreter Lock

- ▶ simplicité de l'interpréteur Python
- ▶ bloque l'exécution de plusieurs threads à la fois
 - ▶ la thread qui s'exécute bloque le GIL
 - ▶ le GIL est relâché sur les opérations d'IO / périodiquement
 - ▶ → multitâche coopératif



<http://www.dabeaz.com/python/UnderstandingGIL.pdf>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions

GIL

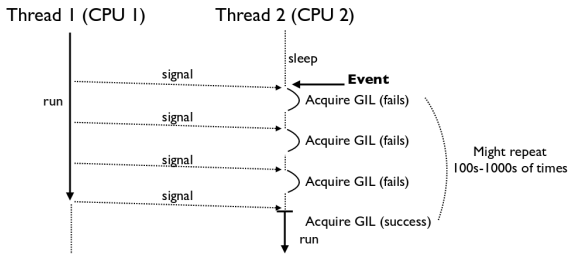
Parallélisation

Bibliographie

Python scientifique

Architectures multi coeurs

Les threads peuvent s'exécuter sur plusieurs coeurs *en même temps*



<http://www.dabeaz.com/python/UnderstandingGIL.pdf>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions

GIL

Parallélisation

Bibliographie

Python scientifique

Gestion des IO

- ▶ les IO ne sont pas forcément bloquants
- ▶ le *buffering* permet à l'OS de compléter des requêtes IO immédiatement
- ▶ le GIL est toujours libéré → trop de changement de contexte si charge importante

7- Programmation concurrente

7.3. Parallélisation

threads – fonction, classe et verrous

Matthieu Falce

```
from threading import Thread
import time

# Define a function for the thread
def print_time(threadName, delay):
    count = 0
    while count < 5:
        time.sleep(delay)
        count += 1
        print("{} {}".format(
            threadName,
            time.ctime(time.time())
        ))

# Create two threads as follows
t1 = Thread(target=print_time, args=("Thread-1", 0.1,))
t2 = Thread(target=print_time, args=("Thread-2", 0.2,))

t1.start(); t2.start()
print("Pas bloqué")
t1.join(); t2.join()
print("Fini")
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions

GIL

Parallélisation

Bibliographie

Python scientifique

threads – fonction, classe et verrous

Matthieu Falce

```
from threading import Thread
import time

class PrintNoBlock(Thread):
    def __init__(self, name, delay):
        super().__init__()
        self.running = False
        self.name = name
        self.delay = delay

    def run(self):
        self.running = True
        count = 0
        while count < 5 and self.running:
            time.sleep(self.delay)
            count += 1
            print("{} {}".format(
                self.name,
                time.ctime(time.time())
            ))

# Create two threads as follows
t1 = PrintNoBlock("Thread-1", 0.1)
t2 = PrintNoBlock("Thread-2", 0.2)

t1.start(); t2.start()
print("Pas bloqué")
t1.join(); t2.join()
print("Fini")
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions

GIL

Parallélisation

Bibliographie

Python scientifique

threads – fonction, classe et verrous

Matthieu Falce

```
from threading import Thread, RLock
import time, random, sys

lock = RLock()

class NoProblemSync(Thread):
    def __init__(self, name):
        super().__init__()
        self.name = name; self.running = True
        self.s = ["tortue", "cheval", "tomate"]

    def run(self):
        count = 0
        while count < 3 and self.running:
            with lock:
                if not self.running: return
                print(self.name, end=" ")
                for char in random.choice(self.s):
                    if not self.running: return
                    print(char, end='')
                    sys.stdout.flush()
                    time.sleep(random.random())
                print()
            time.sleep(random.random() / 5)
            count += 1

# Create two threads as follows
t1 = NoProblemSync("Thread-1")
t2 = NoProblemSync("Thread-2")

try:
    t1.start(); t2.start(); t1.join(); t2.join()
except KeyboardInterrupt:
    t1.running = False; t2.running = False
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions

GIL

Parallélisation

Bibliographie

Python scientifique

Multiprocess

Matthieu Falce

```
import multiprocessing
import time

print("Outside worker")

def worker(num):
    """worker function"""
    print("Worker:", num, time.time())

if __name__ == "__main__":
    jobs = []
    for i in range(10):
        p = multiprocessing.Process(target=worker, args=(i,))
        jobs.append(p)
        p.start()
    for job in jobs:
        job.join()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions

GIL

Parallélisation

Bibliographie

Python scientifique

Performances Threading – Multiprocess

Matthieu Falce

```
import time
```

```
def cpu_bound(nb):
```

```
    res = 0
```

```
    for val in range(nb * 10_000):
```

```
        res += val
```

```
    return res
```

```
def io_bound(nb):
```

```
    for _ in range(nb):
```

```
        res = open("/etc/fstab", "r").readlines()
```

```
    return res
```

```
def timer(f):
```

```
    def wrapper(nb, diff):
```

```
        tstart = time.time()
```

```
        f(nb, diff)
```

```
        res = "workers: {} \tdifficulty: {} \t name: {} \t time: {:.4f}"
```

```
        print(res.format(nb, diff, f.__name__, time.time()-tstart))
```

```
    return wrapper
```

```
def wrapper_parallel(methode, target, nb_workers, difficulty):
```

```
    jobs = []
```

```
    for _ in range(nb_workers):
```

```
        p = methode(target=target, args=(int(difficulty / nb_workers),))
```

```
        jobs.append(p)
```

```
        p.start()
```

```
    for job in jobs:
```

```
        job.join()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions

GIL

Parallélisation

Bibliographie

Python scientifique

Performances Threading – Multiprocess

Matthieu Falce

```
#!/ python
```

```
import threading
import multiprocessing
from workers import cpu_bound, io_bound, timer, wrapper_parallel

@timer
def mp_io(nb, diff):
    return wrapper_parallel(multiprocessing.Process, io_bound, nb, diff)

@timer
def mp_cpu(nb, diff):
    return wrapper_parallel(multiprocessing.Process, cpu_bound, nb, diff)

@timer
def th_io(nb, diff):
    return wrapper_parallel(threading.Thread, io_bound, nb, diff)

@timer
def th_cpu(nb, diff):
    return wrapper_parallel(threading.Thread, cpu_bound, nb, diff)

if __name__ == "__main__":
    difficulties = [10, 1000, 10_000]
    nbs = [1, 2, 4, 8]
    for diff in difficulties:
        for function in [mp_cpu, th_cpu, mp_io, th_io]:
            for nb in nbs:
                function(nb, diff)
        print("\n#####")
    print("#####\n")
```

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Avant propos / définitions](#)

[GIL](#)

[Parallélisation](#)

[Bibliographie](#)

[Python scientifique](#)

Performances Threading – Multiprocess

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Avant propos / définitions
GIL](#)

[Parallélisation
Bibliographie](#)

[Python scientifique](#)

Nb CPU gérés par Ubuntu 18.04 (linux 4.15.0-34-generic)

workers: 1	difficulty: 10	name: mp_cpu	time: 0.0317
workers: 2	difficulty: 10	name: mp_cpu	time: 0.0207
workers: 4	difficulty: 10	name: mp_cpu	time: 0.0313
workers: 8	difficulty: 10	name: mp_cpu	time: 0.0720

workers: 1	difficulty: 10	name: th_cpu	time: 0.0299
workers: 2	difficulty: 10	name: th_cpu	time: 0.0295
workers: 4	difficulty: 10	name: th_cpu	time: 0.0163
workers: 8	difficulty: 10	name: th_cpu	time: 0.0120

#####

workers: 1	difficulty: 10	name: mp_io	time: 0.0091
workers: 2	difficulty: 10	name: mp_io	time: 0.0062
workers: 4	difficulty: 10	name: mp_io	time: 0.0144
workers: 8	difficulty: 10	name: mp_io	time: 0.0318

workers: 1	difficulty: 10	name: th_io	time: 0.0020
workers: 2	difficulty: 10	name: th_io	time: 0.0045
workers: 4	difficulty: 10	name: th_io	time: 0.0038
workers: 8	difficulty: 10	name: th_io	time: 0.0016

Performances Threading – Multiprocess

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Avant propos / définitions](#)

[GIL](#)

[Parallélisation](#)

[Bibliographie](#)

[Python scientifique](#)

Nb CPU gérés par Ubuntu 18.04 (linux 4.15.0-34-generic)

workers: 1	difficulty: 1000	name: mp_cpu	time: 0.8607
workers: 2	difficulty: 1000	name: mp_cpu	time: 0.5459
workers: 4	difficulty: 1000	name: mp_cpu	time: 0.3685
workers: 8	difficulty: 1000	name: mp_cpu	time: 0.4328

workers: 1	difficulty: 1000	name: th_cpu	time: 0.7932
workers: 2	difficulty: 1000	name: th_cpu	time: 1.0545
workers: 4	difficulty: 1000	name: th_cpu	time: 0.9198
workers: 8	difficulty: 1000	name: th_cpu	time: 0.8794

#####

workers: 1	difficulty: 1000	name: mp_io	time: 0.0460
workers: 2	difficulty: 1000	name: mp_io	time: 0.0389
workers: 4	difficulty: 1000	name: mp_io	time: 0.0365
workers: 8	difficulty: 1000	name: mp_io	time: 0.0429

workers: 1	difficulty: 1000	name: th_io	time: 0.0594
workers: 2	difficulty: 1000	name: th_io	time: 0.1138
workers: 4	difficulty: 1000	name: th_io	time: 0.1544
workers: 8	difficulty: 1000	name: th_io	time: 0.1556

Performances Threading – Multiprocess

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Avant propos / définitions](#)

[GIL](#)

[Parallélisation](#)

[Bibliographie](#)

[Python scientifique](#)

Nb CPU gérés par Ubuntu 18.04 (linux 4.15.0-34-generic)

workers: 1	difficulty: 10000	name: mp_cpu	time: 5.4023
workers: 2	difficulty: 10000	name: mp_cpu	time: 3.2131
workers: 4	difficulty: 10000	name: mp_cpu	time: 2.7226
workers: 8	difficulty: 10000	name: mp_cpu	time: 2.8217

workers: 1	difficulty: 10000	name: th_cpu	time: 4.8232
workers: 2	difficulty: 10000	name: th_cpu	time: 5.8625
workers: 4	difficulty: 10000	name: th_cpu	time: 6.3729
workers: 8	difficulty: 10000	name: th_cpu	time: 7.1965

#####

workers: 1	difficulty: 10000	name: mp_io	time: 0.2956
workers: 2	difficulty: 10000	name: mp_io	time: 0.1519
workers: 4	difficulty: 10000	name: mp_io	time: 0.1939
workers: 8	difficulty: 10000	name: mp_io	time: 0.1940

workers: 1	difficulty: 10000	name: th_io	time: 0.3264
workers: 2	difficulty: 10000	name: th_io	time: 0.9546
workers: 4	difficulty: 10000	name: th_io	time: 1.1755
workers: 8	difficulty: 10000	name: th_io	time: 1.3245

Performances Threading – Multiprocess

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions
GIL

Parallélisation
Bibliographie

Python scientifique

Limite à 1 CPU ⁵²

workers: 1	difficulty: 10	name: mp_cpu	time: 0.0194
workers: 2	difficulty: 10	name: mp_cpu	time: 0.0237
workers: 4	difficulty: 10	name: mp_cpu	time: 0.0379
workers: 8	difficulty: 10	name: mp_cpu	time: 0.0496

workers: 1	difficulty: 10	name: th_cpu	time: 0.0347
workers: 2	difficulty: 10	name: th_cpu	time: 0.0162
workers: 4	difficulty: 10	name: th_cpu	time: 0.0088
workers: 8	difficulty: 10	name: th_cpu	time: 0.0133

#####

workers: 1	difficulty: 10	name: mp_io	time: 0.0122
workers: 2	difficulty: 10	name: mp_io	time: 0.0126
workers: 4	difficulty: 10	name: mp_io	time: 0.0318
workers: 8	difficulty: 10	name: mp_io	time: 0.0600

workers: 1	difficulty: 10	name: th_io	time: 0.0022
workers: 2	difficulty: 10	name: th_io	time: 0.0020
workers: 4	difficulty: 10	name: th_io	time: 0.0024
workers: 8	difficulty: 10	name: th_io	time: 0.0037

⁵²<http://xmodulo.com/run-program-process-specific-cpu-cores-linux.html>

Limite à 1 CPU ⁵²

workers: 1	difficulty: 1000	name: mp_cpu	time: 0.7073
workers: 2	difficulty: 1000	name: mp_cpu	time: 0.6151
workers: 4	difficulty: 1000	name: mp_cpu	time: 0.6643
workers: 8	difficulty: 1000	name: mp_cpu	time: 0.8287

workers: 1	difficulty: 1000	name: th_cpu	time: 0.6797
workers: 2	difficulty: 1000	name: th_cpu	time: 0.7070
workers: 4	difficulty: 1000	name: th_cpu	time: 0.7149
workers: 8	difficulty: 1000	name: th_cpu	time: 0.9501

#####

workers: 1	difficulty: 1000	name: mp_io	time: 0.0297
workers: 2	difficulty: 1000	name: mp_io	time: 0.0396
workers: 4	difficulty: 1000	name: mp_io	time: 0.1407
workers: 8	difficulty: 1000	name: mp_io	time: 0.1114

workers: 1	difficulty: 1000	name: th_io	time: 0.0312
workers: 2	difficulty: 1000	name: th_io	time: 0.0992
workers: 4	difficulty: 1000	name: th_io	time: 0.1273
workers: 8	difficulty: 1000	name: th_io	time: 0.1380

⁵²<http://xmodulo.com/run-program-process-specific-cpu-cores-linux.html>

Limite à 1 CPU ⁵²

workers: 1	difficulty: 10000	name: mp_cpu	time: 8.7052
workers: 2	difficulty: 10000	name: mp_cpu	time: 6.0837
workers: 4	difficulty: 10000	name: mp_cpu	time: 5.2096
workers: 8	difficulty: 10000	name: mp_cpu	time: 5.6368

workers: 1	difficulty: 10000	name: th_cpu	time: 5.1897
workers: 2	difficulty: 10000	name: th_cpu	time: 5.6029
workers: 4	difficulty: 10000	name: th_cpu	time: 4.7810
workers: 8	difficulty: 10000	name: th_cpu	time: 4.6669

#####

workers: 1	difficulty: 10000	name: mp_io	time: 0.2579
workers: 2	difficulty: 10000	name: mp_io	time: 0.3199
workers: 4	difficulty: 10000	name: mp_io	time: 0.3941
workers: 8	difficulty: 10000	name: mp_io	time: 0.2785

workers: 1	difficulty: 10000	name: th_io	time: 0.2330
workers: 2	difficulty: 10000	name: th_io	time: 0.2845
workers: 4	difficulty: 10000	name: th_io	time: 0.2648
workers: 8	difficulty: 10000	name: th_io	time: 0.3064

⁵²<http://xmodulo.com/run-program-process-specific-cpu-cores-linux.html>

Map Reduce

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions

GIL

Parallélisation

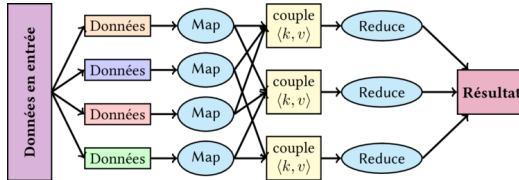
Bibliographie

Python scientifique

- ▶ patron d'architecture pour le calcul parallèle (et distribué)
- ▶ utilisé pour traiter de grande quantités de données

Map Reduce

Matthieu Falce



Source :

<https://fr.wikipedia.org/wiki/MapReduce#/media/File:Mapreduce.png>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions

GIL

Parallélisation

Bibliographie

Python scientifique

7- Programmation concurrente

7.4. Bibliographie

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Avant propos / définitions

GIL

Parallélisation

Bibliographie

Python scientifique

► GIL

- <http://www.dabeaz.com/python/UnderstandingGIL.pdf>
- [https://en.wikipedia.org/wiki/Preemption_\(computing\)](https://en.wikipedia.org/wiki/Preemption_(computing))
- <https://lwn.net/Articles/612483/>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

Python scientifique

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

8- Python scientifique

8.1. Écosystème scientifique

Écosystème

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

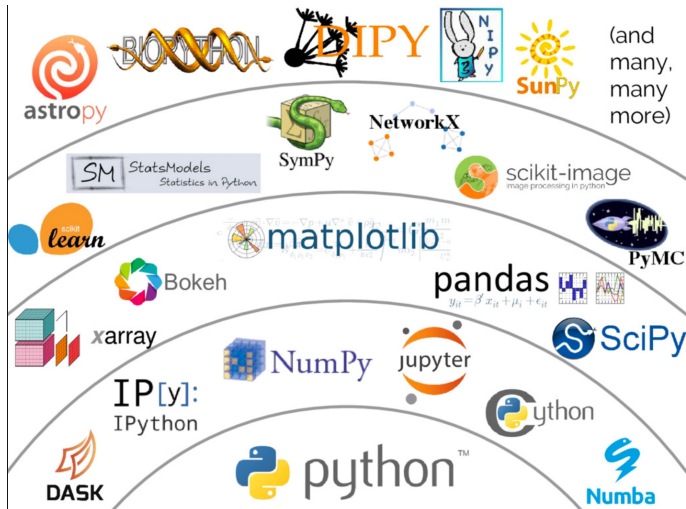
Dask

Bibliographie

- ▶ écosystème très riche
- ▶ utilisé aussi bien par les scientifiques que les entreprises
- ▶ origine : développeurs et scientifiques
- ▶ sponsorisé par de grandes entreprises (google, facebook, Enthought, Anaconda Inc)
- ▶ tout ce que vous cherchez existe probablement déjà

Écosystème

Matthieu Falce



Source : <https://www.datacamp.com/community/blog/python-scientific-computing-case>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

NumPy

SciPy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

Écosystème

Calculs :

- ▶ `numpy` ⁵³
- ▶ `scipy` ⁵⁴
- ▶ `pandas` ⁵⁵
- ▶ ...

Plotting :

- ▶ `matplotlib` ⁵⁶
- ▶ `seaborn` ⁵⁷
- ▶ `bokeh` ⁵⁸
- ▶ ...

53.<http://www.numpy.org/>

54.<https://www.scipy.org/>

55.<https://pandas.pydata.org/>

56.<https://matplotlib.org/>

57.<https://seaborn.pydata.org/>

58.<https://bokeh.pydata.org/en/latest/>

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

Python scientifique

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

```
import numpy as np
```

```
xs = np.arange(-2*np.pi, 2*np.pi, 100)  
ys = np.sin(xs) - 3*xs + 2
```

```
#####
```

```
A = np.array([[1, 2], [3, 4]])  
B = np.array([[5, 6], [7, 8]])  
print(A.dot(B))
```


Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

8- Python scientifique

8.2. Jupyter

IPython – Jupyter

Matthieu Falce

- ▶ shell interactif avec auto-complétion
- ▶ fonctions *magique* (mesure du temps, infos shell...)
- ▶ notebook (et maintenant lab) → programmation littérale, IDE en ligne
- ▶ utilisation d'autres “noyaux” (R, Julia, C, Haskell...)
- ▶ calcul parallèle
- ▶ présentation des résultats
- ▶ ...

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

IPython – Jupyter

Matthieu Falce

- ▶ shell interactif avec auto-complétion
- ▶ fonctions *magique* (mesure du temps, infos shell...)
- ▶ notebook (et maintenant lab) → programmation littérale, IDE en ligne
- ▶ utilisation d'autres “noyaux” (R, Julia, C, Haskell...)
- ▶ calcul parallèle
- ▶ présentation des résultats
- ▶ ...



Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

8- Python scientifique

8.3. Numpy

Paquet fondateur de la stack scientifique Python.

- ▶ propose un type de tableaux multidimensionnels
- ▶ met les performances au premier plan
- ▶ manipulation vectorielles / matricielles faciles
- ▶ outils pour manipuler ces tableaux (algèbre linéaire, traitement du signal, ...)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /

Paquet fondateur de la stack scientifique Python.

- ▶ propose un type de tableaux multidimensionnels
- ▶ met les performances au premier plan
- ▶ manipulation vectorielles / matricielles faciles
- ▶ outils pour manipuler ces tableaux (algèbre linéaire, traitement du signal, ...)

Utilise des bibliothèques Fortran ou C/C++ → bonnes performances

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /

Tableaux numpy

- ▶ tableaux multidimensionnels (NDarray)
- ▶ facilement indexable
- ▶ vectorisation du code (UFunc et broadcasting)

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Multiprocessing /

Tableaux numpy

- ▶ tableaux multidimensionnels (NDarray)
- ▶ facilement indexable
- ▶ vectorisation du code (UFunc et broadcasting)

Même manipulation que Matlab

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning

Tableaux numpy

Matthieu Falce

- ▶ tableaux multidimensionnels (NDarray)
- ▶ facilement indexable
- ▶ vectorisation du code (UFunc et broadcasting)

```
import numpy as np

xs = np.arange(-2*np.pi, 2*np.pi, 100)
ys = np.sin(xs) - 3*xs + 2

#####

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(A.dot(B))
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning

Structure

- ▶ un seul type de données par tableau (ou objet python)
- ▶ métadonnées + données linéaires
- ▶ orientation (C ou Fortran) → important pour les performances

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

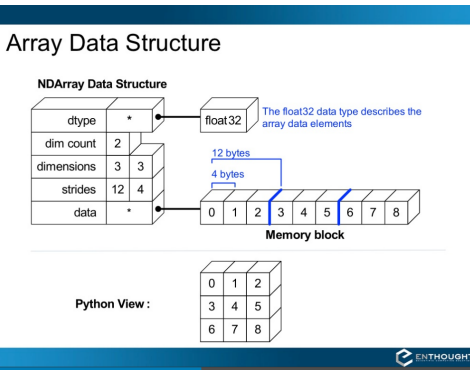
Sympy

Analyse de réseaux

Machine learning

Structure

- ▶ un seul type de données par tableau (ou objet python)
- ▶ métadonnées + données linéaires
- ▶ orientation (C ou Fortran) → important pour les performances



[https:](https://www.slideshare.net/enthought/numpy-talk-at-siam)

[//www.slideshare.net/enthought/numpy-talk-at-siam](https://www.slideshare.net/enthought/numpy-talk-at-siam)

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

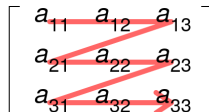
Analyse de réseaux

Machine learning

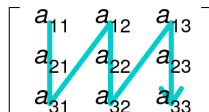
Structure

- ▶ un seul type de données par tableau (ou objet python)
- ▶ métadonnées + données linéaires
- ▶ orientation (C ou Fortran) → important pour les performances

Row-major order



Column-major order



By Cmglee - Own work, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=65107030>

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

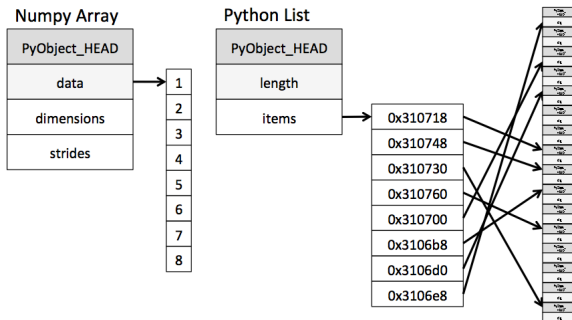
Analyse de réseaux

Machine learning

Structure

Matthieu Falce

- ▶ un seul type de données par tableau (ou objet python)
- ▶ métadonnées + données linéaires
- ▶ orientation (C ou Fortran) → important pour les performances



<https://jakevdp.github.io/PythonDataScienceHandbook/02.01-understanding-data-types.html>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Création des tableaux

Matthieu Falce

```
import numpy as np
```

```
# à partir d'une liste / liste de liste...
```

```
xs = np.array([i for i in range(10)])
```

```
A = np.array([[1, 2], [3, 4]])
```

```
# équivalent de range
```

```
rs = np.arange(10, 50, 2)
```

```
# valeurs régulièrement espacées
```

```
es = np.linspace(-3.65, np.pi, 100)
```

```
# forcer les types
```

```
ts = np.linspace(-3.65, np.pi, 100, dtype=np.int)
```

```
ts2 = np.linspace(-3.65, np.pi, 100, dtype=np.complex) + 2j
```

```
# tous les utilitaires classiques
```

```
ones = np.ones((3, 1))
```

```
diag = np.eye(3)
```

```
full = np.full((3, 2), np.pi)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning

Création des tableaux

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning

Creation

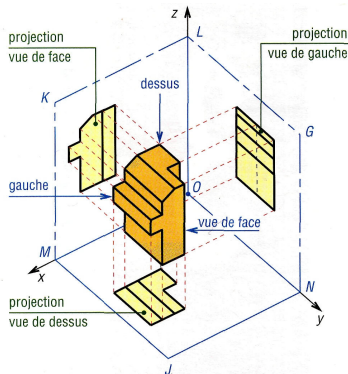
Code	Result	Code	Result
<code>Z = zeros(9)</code>		<code>Z = zeros((5,9))</code>	
<code>Z = ones(9)</code>		<code>Z = ones((5,9))</code>	
<code>Z = array([0,0,0,0,0,0,0,0,0])</code>		<code>Z = array([[0,0,0,0,0,0,0,0,0], [0,0,0,0,0,0,0,0,0], [0,0,0,0,0,0,0,0,0], [0,0,0,0,0,0,0,0,0], [0,0,0,0,0,0,0,0,0]])</code>	
<code>Z = arange(9)</code>		<code>Z = arange(5*9).reshape(5,9)</code>	
<code>Z = random.uniform(0,1,9)</code>		<code>Z = random.uniform(0,1,(5,9))</code>	

<http://www.labri.fr/perso/nrougier/teaching/numpy/numpy.html>

Changements de forme / dimensions

Matthieu Falce

Certaines opérations vont s'opérer sur certaines dimensions.
On peut les visualiser comme étant des projections.



http://www.zpag.net/Tecnologies_Industrielles/projections_orthogonales_normali.htm

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Machine Learning

Changements de forme / dimensions

Matthieu Falce

```
import numpy as np
```

```
# changement du nombre de dimensions
```

```
A = np.arange(1, 3 * 4 + 1).reshape((3, 4))
```

```
print(A)
```

```
# [[ 1  2  3  4]
```

```
#  [ 5  6  7  8]
```

```
#  [ 9 10 11 12]]
```

```
# on peut effectuer des actions selon
```

```
# certaines dimensions
```

```
B = np.arange(1, (3 * 4 * 5) + 1).reshape((3, 4, 5))
```

```
B.mean(axis=0)
```

```
B.mean(axis=1)
```

```
B.mean(axis=2)
```

```
B.sum(axis=1)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /

Changements de forme / dimensions

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /

Reshaping

Code

Result

Code

Result

```
Z[2,2] = 1
```



```
Z = Z.reshape(4,3)
```



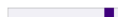
```
Z = Z.reshape(6,2)
```



```
Z = Z.reshape(2,6)
```



```
Z = Z.reshape(1,12)
```



```
Z = Z.reshape(12,1)
```



<http://www.labri.fr/perso/nrougier/teaching/numpy/numpy.html>

Indexing

Matthieu Falco

```
import numpy as np
```

```
# découpage
```

```
D = np.arange((100 * 5)).reshape((100, 5))
```

```
split = 30
```

```
test, train = D[:,split:], D[:,split:, :]
```

```
# indexing booléen
```

```
# on met toutes les valeurs paires à 0
```

```
A = np.arange(1, 3 * 4 + 1).reshape((3, 4))
```

```
pairs = (A % 2 == 0)
```

```
pairs = pairs.astype(bool)
```

```
A[pairs] = 0
```

```
# slicing et réassignation partielle
```

```
B = np.arange(100, dtype=np.uint8).reshape((10, 10))
```

```
B[:,2, ::2] = B[1::2, 1::2] / 2
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning

Indexing

Matthieu Falce

Slicing

Code	Result	Code	Result
<code>Z</code>		<code>Z[...] = 1</code>	
<code>Z[1,1] = 1</code>		<code>Z[:,0] = 1</code>	
<code>Z[0,:] = 1</code>		<code>Z[2:,2:] = 1</code>	
<code>Z[:,::2] = 1</code>		<code>Z[:,2:] = 1</code>	
<code>Z[:-2,:-2] = 1</code>		<code>Z[2:4,2:4] = 1</code>	
<code>Z[::2,::2] = 1</code>		<code>Z[3::2,3::2] = 1</code>	

<http://www.labri.fr/perso/nrougier/teaching/numpy/numpy.html>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

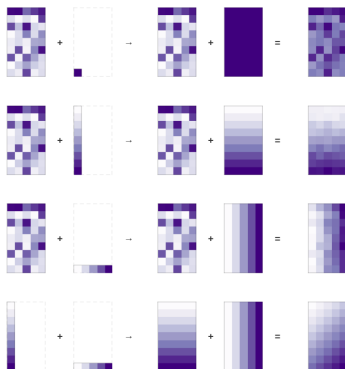
Analyse de réseaux

Machine learning

Broadcasting

Le broadcasting permet de manipuler entre eux des tableaux de tailles différentes.

Broadcasting



<http://www.labri.fr/perso/nrougier/teaching/numpy/numpy.html>

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning

Broadcasting

Le broadcasting permet de manipuler entre eux des tableaux de tailles différentes.



Ce n'est pas magique

```
In [10]: A = np.arange(9).reshape((3, 3))
In [11]: B = np.arange(4)
In [12]: A + B
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-12-151064de832d> in <module>()
----> 1 A + B
ValueError: operands could not be broadcast
together with shapes (3,3) (4,)
```

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Machine Learning

Fonctions universelles

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Fonctions universelles s'appliquent sur les éléments d'un tableau de façon vectorisée (sans boucles apparentes).

Exemple : `y = np.sin(x)`

Fonctions universelles

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Présentation

Tableaux

Structure des tableaux

Création des tableaux

Manipulations usuelles

Broadcasting

Fonctions universelles

Scipy

Pandas

Sympy

Analyse de réseaux

Fonctions universelles s'appliquent sur les éléments d'un tableau de façon vectorisée (sans boucles apparentes).

Exemple : `y = np.sin(x)`

On peut créer ses propres fonctions vectorisées avec `np.frompyfunc` ou `np.vectorize`.



Ce n'est pas pour ça qu'elles seront plus rapides.

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Présentation

Scikits

Fonctionnalités

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

8- Python scientifique

8.4. Scipy

Méthodes additionnelles à Numpy pour le calcul scientifique.

- ▶ fonctions spéciales
- ▶ intégration
- ▶ optimisation
- ▶ équations différentielles
- ▶ traitement du signal
- ▶ algèbre linéaire
- ▶ ...

Si des routines sont partagées avec Numpy → plus complètes
dans Scipy ⁵⁹

Scipy utilise LAPACK, Numpy pas forcément ⁶⁰

⁵⁹<https://docs.scipy.org/doc/numpy/reference/routines.dual.html>

⁶⁰<https://www.scipy.org/scipylib/faq.html#what-is-the-difference-between-numpy-and-scipy>

SciPy Toolkits → extensions pour Scipy indépendantes

Liste non exhaustive ⁶¹

- ▶ scikit-learn
- ▶ scikit-image
- ▶ scikit-bio
- ▶ ...

```
import numpy as np
import matplotlib.pyplot as plt

t = np.arange(256)
sig = np.sin(t)

# sp est un tableau de complexes
sp1 = np.fft.fft(sig)
freq1 = np.fft.fftfreq(t.shape[-1])
plt.plot(freq1, sp1.real, freq1, sp1.imag)
plt.show()

# comparaisons fréquences
sig2 = np.sin(2 * t)
sp2 = np.fft.fft(sig2)
freq2 = np.fft.fftfreq(t.shape[-1])

plt.plot(freq2, sp2.real, label="F2")
plt.plot(freq1, sp1.real, label="F1")
plt.legend()
plt.show()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standardCréation de
modulesProgrammation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Présentation

Scikits

Fonctionnalités

Pandas

SymPy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

```
from scipy.optimize import minimize
```

```
minimize(lambda x: x**2, x0=1000)
```

```
#      fun: 5.713415792109052e-17
# hess_inv: array([[0.50000012]])
#      jac: array([-2.16266884e-10])
# message: 'Optimization terminated successfully.'
#      nfev: 24
#       nit: 3
#      njev: 8
#   status: 0
# success: True
#       x: array([-7.55871404e-09])
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standardCréation de
modulesProgrammation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Présentation

Scikits

Fonctionnalités

Pandas

SymPy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standardCréation de
modulesProgrammation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Présentation

Scikits

Fonctionnalités

Pandas

SymPy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

```
from scipy.optimize import minimize

minimize(lambda x: x[0]**2 + x[1] ** 2, x0=(1000, 33))

#      fun: 1.3011523813214424e-13
# hess_inv: array([[0.54198343, 0.00254437],
#                  [0.00254437, 0.5001542 ]])
#      jac: array([7.35719908e-07, 4.45876263e-08])
# message: 'Optimization terminated successfully.'
#      nfev: 40
#       nit: 5
#      njev: 10
#   status: 0
#  success: True
#         x: array([3.60409374e-07, 1.48432326e-08])
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standardCréation de
modulesProgrammation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Présentation

Scikits

Fonctionnalités

Pandas

SymPy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

```
from scipy.optimize import minimize

minimize(lambda x: x[0]**2 + x[1] ** 2, x0=(1000, 33))

#      fun: 1.3011523813214424e-13
# hess_inv: array([[0.54198343, 0.00254437],
#                  [0.00254437, 0.5001542 ]])
#      jac: array([7.35719908e-07, 4.45876263e-08])
# message: 'Optimization terminated successfully.'
#      nfev: 40
#       nit: 5
#      njev: 10
#   status: 0
#  success: True
#         x: array([3.60409374e-07, 1.48432326e-08])
```

On peut mettre des contraintes sur les paramètres et
changer de méthode d'optimisation aussi.

- ▶ en 2D
- ▶ splines et courbes paramétrées
- ▶ ...

```
import numpy as np
from scipy.interpolate import interp1d
import matplotlib.pyplot as plt

x = np.linspace(0, 10, num=11, endpoint=True)
y = np.cos(-x ** 2 / 9.0)
f = interp1d(x, y)
f2 = interp1d(x, y, kind='cubic')

xnew = np.linspace(0, 10, num=41, endpoint=True)
plt.plot(x, y, 'o', xnew, f(xnew), '-', xnew, f2(xnew), '--')
plt.legend(['data', 'linear', 'cubic'], loc='best')
plt.show()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standardCréation de
modulesProgrammation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Présentation

Scikits

Fonctionnalités

Pandas

SymPy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Intégration – équations différentielles ⁶⁵ ⁶⁶

Matthieu Falce

- ▶ en 1D
- ▶ en 2D
- ▶ ... jusqu'à N dimensions
- ▶ paramétrages complexes
- ▶ ODE non stiff et stiff, pas adaptatif...

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Présentation

Scikits

Fonctionnalités

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

65.<https://docs.scipy.org/doc/scipy/reference/tutorial/integrate.html>

66.<https://docs.scipy.org/doc/scipy/reference/integrate.html>

Intégration – équations différentielles ^{65 66}

Matthieu Falce

```
from scipy.integrate import quad
```

```
def fonction_a_integrer(x, a, b):  
    return a * x ** 3 + b
```

```
a = 1
```

```
b = 0
```

```
surface = quad(  
    fonction_a_integrer,  
    -10,  
    10,  
    args=(a, b)  
)
```

```
print(surface)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Présentation

Scikits

Fonctionnalités

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

65.<https://docs.scipy.org/doc/scipy/reference/tutorial/integrate.html>

66.<https://docs.scipy.org/doc/scipy/reference/integrate.html>

Intégration – équations différentielles ^{65 66}

Matthieu Falce

```
import numpy as np
from scipy.integrate import odeint
import matplotlib.pyplot as plt
```

```
def carre_der(t, x):
    return 2*x
```

```
ts = np.arange(10)
ys = odeint(carre_der, 2, ts)
plt.plot(ts, ys)
plt.show()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Présentation

Scikits

Fonctionnalités

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

65.<https://docs.scipy.org/doc/scipy/reference/tutorial/integrate.html>

66.<https://docs.scipy.org/doc/scipy/reference/integrate.html>

“Tout” pour le traitement du signal :⁶⁷

- ▶ convolutions
- ▶ conceptions / analyses de filtres (FIR, FII) / analyse réponse
- ▶ analyses de spectres
- ▶ détections de pics

67.<https://docs.scipy.org/doc/scipy/reference/signal.html#module-scipy.signal>

68.<https://docs.scipy.org/doc/scipy/reference/tutorial/signal.html>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Présentation

Scikits

Fonctionnalités

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask



Fonctions potentiellement légèrement différentes de celles de
Numpy
Compilées avec BLAS et LAPACK

Numpy supporte les matrices creuses de différents types :

- ▶ `csc_matrix`: Compressed Sparse Column format
- ▶ `csr_matrix`: Compressed Sparse Row format
- ▶ `bsr_matrix`: Block Sparse Row format
- ▶ `lil_matrix`: List of Lists format
- ▶ `dok_matrix`: Dictionary of Keys format
- ▶ `coo_matrix`: COOrdinate format (aka IJV, triplet format)
- ▶ `dia_matrix`: DIAgonal format

70.<https://docs.scipy.org/doc/scipy-0.14.0/reference/sparse.html>

71.<https://docs.scipy.org/doc/scipy/reference/tutorial/arpack.html>

Numpy supporte les matrices creuses de différents types :

- ▶ `csc_matrix`: Compressed Sparse Column format
- ▶ `csr_matrix`: Compressed Sparse Row format
- ▶ `bsr_matrix`: Block Sparse Row format
- ▶ `lil_matrix`: List of Lists format
- ▶ `dok_matrix`: Dictionary of Keys format
- ▶ `coo_matrix`: COOrdinate format (aka IJV, triplet format)
- ▶ `dia_matrix`: DIAGonal format

Utiliser les méthodes de `scipy.sparse.linalg` pour faire des opérations et garder des matrices creuses. Selon le type de matrices utilisées ; différentes possibilités (perte du slicing...)

70.<https://docs.scipy.org/doc/scipy-0.14.0/reference/sparse.html>

71.<https://docs.scipy.org/doc/scipy/reference/tutorial/arpack.html>

- ▶ distributions discrètes et continues
- ▶ analyses de données statistiques (kurtosis, zscores...)
- ▶ tests statistiques
- ▶ ...

Manipulation d'images

Matthieu Falce

Les images (rasterisées ou pixelisées) sont des tableaux *numpy* classiques.

Voici les principales bibliothèques pour en faire :

- ▶ *Pillow* ⁷⁴ : plutôt utilisée pour les manipulations classiques (redimensionnement, rotation, changement de format, ...)
- ▶ *numpy* / *scipy* classique ⁷⁵ : permet de manipuler les tableaux de données de pixels directement
- ▶ *scikit image* ^{76 77} : Propose de nombreux algorithmes pour faire du traitement d'images
- ▶ *openCV* ⁷⁸ : le port de la Bibliothèque d'analyse d'image openCV. Permet de travailler en temps réel (à partir d'une caméra par exemple)

74.<https://pillow.readthedocs.io/en/stable/>

75.Plus d'infos ici : http://scipy-lectures.org/advanced/image_processing/

76.<https://scikit-image.org/>

77.<https://scipy-lectures.org/packages/scikit-image/index.html>

78.https://docs.opencv.org/master/d6/d00/tutorial_py_root.html

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Présentation

Scikits

Fonctionnalités

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Manipulation d'images

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Présentation

Scikits

Fonctionnalités

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

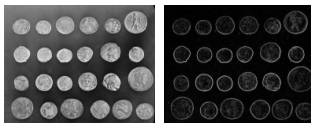
Graphiques

Performances

Dask

```
from skimage import data, io, filters
```

```
image = data.coins()  
edges = filters.sobel(image)  
io.imshow(edges)  
io.show()
```



Manipulation d'images avec scikit image.

Source : <https://scikit-image.org/>

Manipulation d'images

Matthieu Falce

```
import cv2 as cv

# Read image from your local file system
original_image = cv.imread("path/to/your-image.jpg")

# Convert color image to grayscale for Viola-Jones
grayscale_image = cv.cvtColor(original_image, cv.COLOR_BGR2GRAY)

# Load the classifier and create a cascade object for face detection
face_cascade = cv.CascadeClassifier("path/to/haarcascade_frontalface_alt.xml")

detected_faces = face_cascade.detectMultiScale(grayscale_image)

for (column, row, width, height) in detected_faces:
    cv.rectangle(
        original_image, (column, row), (column + width, row + height), (0, 255, 0), 2
    )

cv.imshow("Image", original_image)
cv.waitKey(0)
cv.destroyAllWindows()
```

Détection de visages avec openCV.

Source : <https://realpython.com/traditional-face-detection-python/>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Présentation

Scikits

Fonctionnalités

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Présentation

Dataframes

Manipulations de
dataframes

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

8- Python scientifique

8.5. Pandas

Bibliothèque essentielle à l'analyse de données.

Données structurées (lignes / colonnes à la SQL) et séries temporelles.

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Présentation

Dataframes

Manipulations de
dataframes

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dataframes et séries

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Présentation

Dataframes

Manipulations de
dataframes

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

- ▶ **series** : suite de données à 1 dimension (comme un tableau numpy avec d'autres fonctionnalités)
- ▶ **dataframes** : regroupement de plusieurs séries de même taille (comme un tableur)

Manipulations de dataframes

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Présentation

Dataframes

**Manipulations de
dataframes**

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Création de dataframes et de séries

```
import numpy as np
import pandas as pd

# créer une série
s = pd.Series(np.random.randn(5), index=["a", "b", "c", "d", "e"])

# créer un dataframe
d = {
    "one": pd.Series([1.0, 2.0, 3.0], index=["a", "b", "c"]),
    "two": pd.Series([1.0, 2.0, 3.0, 4.0], index=["a", "b", "c", "d"]),
}
df = pd.DataFrame(d)
df["three"] = s

print("description de df :")
df.describe()
```

Manipulations de dataframes

Matthieu Falce

Indexation et accès aux données

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html

```
import numpy as np
import pandas as pd
```

créer une série

```
s = pd.Series(np.random.randn(5), index=["a", "b", "c", "d", "e"])
df = pd.DataFrame({"one": s, "two": s[1:], "three": s[2:]})
```

accès aux colonnes

```
one, two = df["one"], df.two
df[["one", "two"]]
```

accès aux lignes

```
df[:1] # slicing
a, bcd, adb = df.loc["a"], df.loc["b":], df.loc[["a", "b", "d"]]
df.iloc[2:]
type(df[1:2]) # pandas.core.frame.DataFrame
type(df.iloc[1]) # pandas.core.series.Series
```

accès aux éléments

```
df.loc["a", "one"] # index ligne, colonne
```

échantillonnage

```
seed = 1
df.sample(n=3, replace=True, random_state=seed)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Présentation

Dataframes

**Manipulations de
dataframes**

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Manipulations de dataframes

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Présentation

Dataframes

**Manipulations de
dataframes**

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Aparté sur les performances d'indexation

```
## vitesse
# # bad practice
# In [89]: %timeit df["one"]["a"]
# 8.9  $\mu$ s  $\pm$  107 ns per loop (mean  $\pm$  std. dev. of 7 runs, 100000 loops each)

# # high level loc
# In [90]: %timeit df.loc["a", "one"]
# 6.6  $\mu$ s  $\pm$  230 ns per loop (mean  $\pm$  std. dev. of 7 runs, 100000 loops each)

# # low level at
# In [91]: %timeit df.at["a", "one"]
# 3.88  $\mu$ s  $\pm$  33.3 ns per loop (mean  $\pm$  std. dev. of 7 runs, 100000 loops each)
```

Manipulations de dataframes

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Présentation

Dataframes

**Manipulations de
dataframes**

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Lecture de CSV et nettoyage de données

```
import pandas as pd
import matplotlib.pyplot as plt

url = ("http://facweb.cs.depaul.edu/mobasher/classes"
      "/csc478/Data/titanic-trimmed.csv")
titanic = pd.read_csv(url)
titanic.head(10)

age_mean = titanic.age.mean()
titanic.age.fillna(age_mean, axis=0, inplace=True)
titanic.dropna(axis=0, inplace=True)
titanic.age.describe()

titanic.age.plot.kde()
plt.show()
```

Manipulations de dataframes

Matthieu Falce

Exemples de graphiques possibles

```
import numpy as np
import pandas as pd
from pandas.plotting import lag_plot
from pandas.plotting import autocorrelation_plot
from matplotlib import pyplot as plt
```

```
xs = np.linspace(-1, 1, 100)
ys = np.cos(xs)
ys2 = np.random.random(100)
```

```
df = pd.DataFrame({"cos": ys, "rand": ys2})
```

```
lag_plot(df.cos)
plt.show()
lag_plot(df.rand)
plt.show()
```

```
autocorrelation_plot(df.rand)
plt.show()
autocorrelation_plot(df.cos)
plt.show()
```

```
ax = df.cos.plot()
fig = ax.get_figure()
fig.savefig("cos.png")
```

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

[Écosystème scientifique](#)

[Jupyter](#)

[Numpy](#)

[Scipy](#)

[Pandas](#)

[Présentation](#)

[Dataframes](#)

**[Manipulations de
dataframes](#)**

[SymPy](#)

[Analyse de réseaux](#)

[Machine learning /
statistiques](#)

[Graphiques](#)

[Performances](#)

Manipulations de dataframes

Matthieu Falce

Opération sur des groupes de données

```
import pandas as pd
import numpy as np

df = pd.DataFrame(
    [
        ("bird", "Falconiformes", 389.0),
        ("bird", "Psittaciformes", 24.0),
        ("mammal", "Carnivora", 80.2),
        ("mammal", "Primates", np.nan),
        ("mammal", "Carnivora", 58),
    ],
    index=["falcon", "parrot", "lion", "monkey", "leopard"],
    columns=["class", "order", "max_speed"],
)

grouped1 = df.groupby("class")
grouped2 = df.groupby("order", axis="columns")
grouped3 = df.groupby(["class", "order"])

for n, g in grouped3:
    print(n)
    print(g)
    print(type(g))
    print("")
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Présentation

Dataframes

**Manipulations de
dataframes**

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Manipulations de dataframes

Matthieu Falce

Manipulation de séries temporelles

```
import pandas as pd
import numpy as np
```

```
# time index
```

```
dti = pd.date_range("2018-01-13", periods=3, freq="H")
dti = dti.tz_localize("UTC")
dti.tz_convert("US/Pacific")
```

```
## offsets
```

```
# https://pandas.pydata.org/pandas-docs/stable/user\_guide/timeseries.html#timeseries-offset-aliases
start, end = "2019-01-12", "2019-12-25"
pd.date_range(start, end, freq="BM")
```

```
# conversion
```

```
## https://docs.python.org/3/library/datetime.html#strptime-and-strftime-behavior
pd.to_datetime("12-11-2010 00:00", format="%d-%m-%Y %H:%M")
```

```
# resampling
```

```
idx = pd.date_range("2018-01-01", periods=48, freq="H")
ts = pd.Series(range(len(idx)), index=idx)
ts.resample("2H").mean()
```

```
s = pd.Series(range(len(idx)), index=idx)
for i in s.resample("6H"):
    print(i)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Présentation

Dataframes

**Manipulations de
dataframes**

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Indexation (représentation graphique)

Plus d'informations sur les différents modes d'accès ici :

<https://stackoverflow.com/questions/28757389/pandas-loc-vs-iloc-vs-at-vs-iat>

Python Pandas Selections and Indexing

.iloc selections - position based selection

`data.iloc[<row selection>, <column selection>]`

Integer list of rows: [0,1,2]

Slice of rows: [4:7]

Single values: 1

Integer list of columns: [0,1,2]

Slice of columns: [4:7]

Single column selections: 1

loc selections - position based selection

`data.loc[<row selection>, <column selection>]`

Index/Label value: 'john'

List of labels: ['john', 'sarah']

Logical/Boolean index: data['age'] == 10

Named column: 'first_name'

List of column names: ['first_name', 'age']

Slice of columns: 'first_name':'address'

Source : <https://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/>

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Présentation

Dataframes

**Manipulations de
dataframes**

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

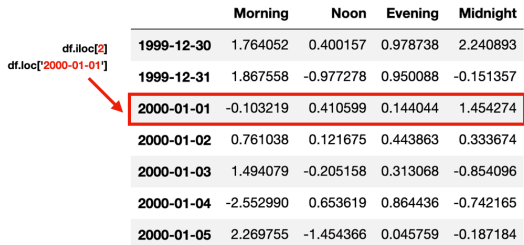
Indexation (représentation graphique)

Matthieu Falce

Plus d'informations sur les différents modes d'accès ici :

<https://stackoverflow.com/questions/28757389/pandas-loc-vs-iloc-vs-at-vs-iat>

Pandas Select Row



The diagram illustrates two ways to select a row in a Pandas DataFrame. On the left, the code `df.iloc[2]` and `df.loc['2000-01-01']` is shown. A red arrow points from the second code line to the row for '2000-01-01' in the table below, which is highlighted with a red border.

	Morning	Noon	Evening	Midnight
1999-12-30	1.764052	0.400157	0.978738	2.240893
1999-12-31	1.867558	-0.977278	0.950088	-0.151357
2000-01-01	-0.103219	0.410599	0.144044	1.454274
2000-01-02	0.761038	0.121675	0.443863	0.333674
2000-01-03	1.494079	-0.205158	0.313068	-0.854096
2000-01-04	-2.552990	0.653619	0.864436	-0.742165
2000-01-05	2.269755	-1.454366	0.045759	-0.187184

© Matt Harasymczuk, 2020, CC-BY-SA-4.0

Source : [https:](https://python.astrotech.io/pandas/dataframe/loc.html)

[//python.astrotech.io/pandas/dataframe/loc.html](https://python.astrotech.io/pandas/dataframe/loc.html)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Présentation

Dataframes

Manipulations de
dataframes

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Indexation (représentation graphique)

Matthieu Falce

Plus d'informations sur les différents modes d'accès ici :

<https://stackoverflow.com/questions/28757389/pandas-loc-vs-iloc-vs-at-vs-iat>

Pandas Select Column



	Morning	Noon	Evening	Midnight
1999-12-30	1.764052	0.400157	0.978738	2.240893
1999-12-31	1.867558	-0.977278	0.950088	-0.151357
2000-01-01	-0.103219	0.410599	0.144044	1.454274
2000-01-02	0.761038	0.121675	0.443863	0.333674
2000-01-03	1.494079	-0.205158	0.313068	-0.854096
2000-01-04	-2.552990	0.653619	0.864436	-0.742165
2000-01-05	2.269755	-1.454366	0.045759	-0.187184

© Matt Harasymczuk, 2020, CC-BY-SA-4.0

Source : [https:](https://python.astrotech.io/pandas/dataframe/loc.html)

[//python.astrotech.io/pandas/dataframe/loc.html](https://python.astrotech.io/pandas/dataframe/loc.html)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Présentation

Dataframes

Manipulations de
dataframes

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

8- Python scientifique

8.6. Sympy

Sympy⁷⁴ est une bibliothèque permettant le calcul symbolique :

- ▶ simplification d'expression
- ▶ analyse (dérivation, intégration)
- ▶ affichage des sorties en $\text{\LaTeX}$

⁷⁴<http://www.sympy.org>

Calcul symbolique

Matthieu Falce

```
from sympy import *
```

```
# permet l'affichage des formules  
init_session()
```

```
# on déclare des variables  
x, a, b = symbols("x a b")
```

```
# on défini une intégrale  
a = Integral(cos(x) * exp(x), x)  
print(a)  
latex(a) # on affiche son code latex
```

```
# on va simplifier des expressions  
simplify(sin(x) ** 2 + cos(x) ** 2)  
simplify(x ** a * x ** b)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

8- Python scientifique

8.7. Analyse de réseaux

Il existe plusieurs bibliothèques pour manipuler des graphes (réseaux) en python :

- ▶ `igraph` ⁷⁵
- ▶ `networkx` ⁷⁶
- ▶ `graph-tool` ⁷⁷

La plus connue est networkX mais peut être lente (codée en pur python), les autres sont potentiellement plus rapide.

⁷⁵<http://igraph.org>

⁷⁶<http://networkx.github.io>

⁷⁷<http://graph-tool.skewed.de>

Manipulation de graphes

Matthieu Falce

```
import networkx as nx
```

```
# on crée un graphe en 2 parties
```

```
G = nx.Graph()  
G.add_edges_from([(1, 2), (1, 3)])  
G.add_node(4)
```

```
# on calcule des propriétés du graphe
```

```
nx.connected_components(G)  
list(nx.connected_components(G))  
sorted(d for n, d in G.degree)  
nx.clustering(G)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

**Machine learning /
statistiques**

Graphiques

Performances

Dask

Bibliographie

8- Python scientifique

8.8. Machine learning / statistiques

Il existe plusieurs bibliothèques dédiées apprentissage :

- ▶ modèles statistiques (régressions, tests statistiques, méthodes sur séries temporelles) : statsmodels ⁷⁸
- ▶ algorithmes de *machine learning* : scikit-learn ⁷⁹
- ▶ *curve feating* : prophet⁸⁰ (analyse de séries temporelles)
- ▶ *deep learning* : tensorflow⁸¹, keras⁸²

78.<https://www.statsmodels.org/stable/index.html>

79.<https://scikit-learn.org/stable/>

80.<https://facebook.github.io/prophet/>

81.<https://www.tensorflow.org/>

82.<https://keras.io/>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

8- Python scientifique

8.9. Graphiques

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

1. amélioration et diversification des outils
2. réalisation de graphiques de qualité
3. écosystème riche (seuls R et Javascript ont de meilleures bibliothèques à mon avis)

Bibliothèques de plot la plus célèbre

API inspirée de Matlab

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

Matplotlib

Bibliothèques de plot la plus célèbre

API inspirée de Matlab

```
from matplotlib import pyplot as plt
import numpy as np

xs = np.linspace(-2*np.pi, 2*np.pi, 100)
ys1 = np.sinc(xs)
ys2 = np.sin(xs)

plt.plot(ys1, c="r", label="Sinc")
plt.plot(ys2, c="b", label="Sin")
plt.xlabel("X")
plt.ylabel("f(x)")
plt.legend()
plt.show()
```

plt agit comme une machine à état

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

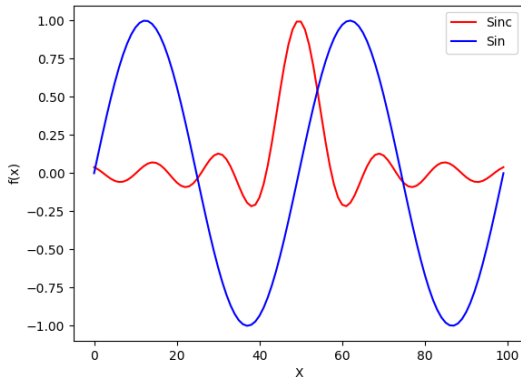
Concurrents 2D

Graphiques 3D

Bibliothèques de plot la plus célèbre

API inspirée de Matlab

Le style aussi



[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

[Écosystème scientifique](#)

[Jupyter](#)

[Numpy](#)

[Scipy](#)

[Pandas](#)

[SymPy](#)

[Analyse de réseaux](#)

[Machine learning /
statistiques](#)

[Graphiques](#)

Matplotlib

[Personnalisation](#)

[Seaborn](#)

[Concurrents 2D](#)

[Graphiques 3D](#)

Matplotlib – types de plots

Matplotlib est une bibliothèque graphique → peut tout faire.

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

Matplotlib – types de plots

Matplotlib est une bibliothèque graphique → peut tout faire.



Si l'on s'en donne la peine

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

Matplotlib – types de plots

Matplotlib est une bibliothèque graphique → peut tout faire.



Si l'on s'en donne la peine

- ▶ lineplot
- ▶ scatterplot
- ▶ cartographie
- ▶ hexbin
- ▶ nuages de mots
- ▶ 3D
- ▶ animations
- ▶ ...

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

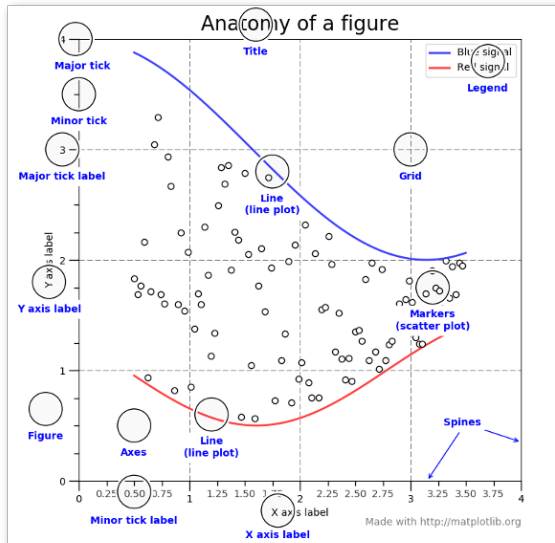
Seaborn

Concurrents 2D

Graphiques 3D

Anatomie d'une figure

Matthieu Falce



https://matplotlib.org/faq/usage_faq.html

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

Anatomie d'une figure

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

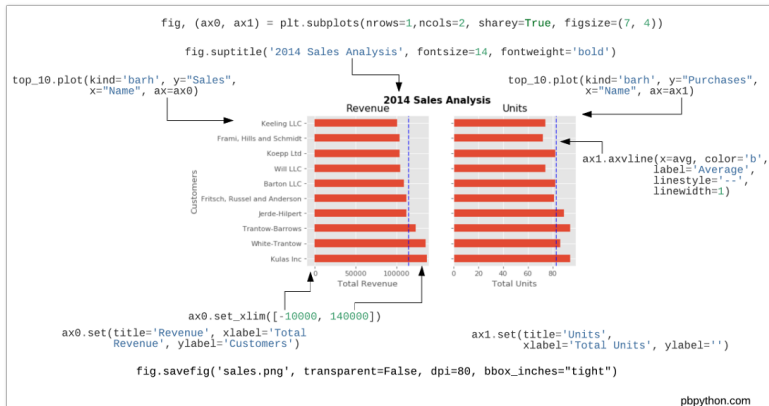
Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

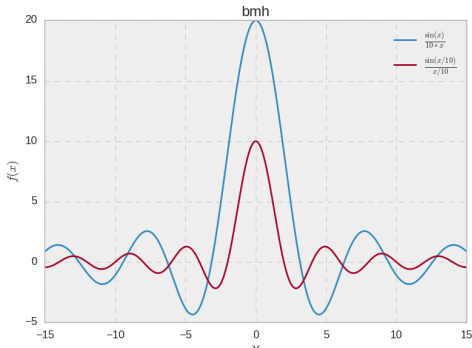


<http://pbpython.com/effective-matplotlib.html>

Personnalisation

Vous pouvez surcharger la configuration de Matplotlib :

- ▶ passer des paramètres à la fonction de dessin
- ▶ changer de style
- ▶ modifier le `pyplot.rcParams`



Thème : **bmh**

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

[Écosystème scientifique](#)

[Jupyter](#)

[Numpy](#)

[Scipy](#)

[Pandas](#)

[SymPy](#)

[Analyse de réseaux](#)

[Machine learning /
statistiques](#)

[Graphiques](#)

[Matplotlib](#)

[Personnalisation](#)

[Seaborn](#)

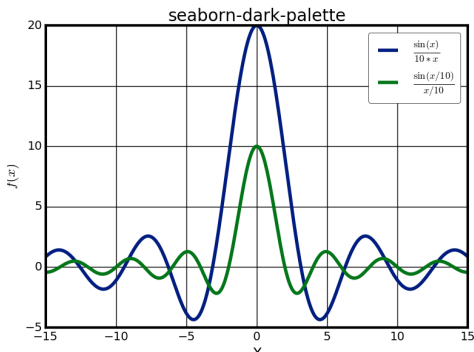
[Concurrents 2D](#)

[Graphiques 3D](#)

Personnalisation

Vous pouvez surcharger la configuration de Matplotlib :

- ▶ passer des paramètres à la fonction de dessin
- ▶ changer de style
- ▶ modifier le `pyplot.rcParams`



Thème : seaborn-dark-palette

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

[Écosystème scientifique](#)

[Jupyter](#)

[Numpy](#)

[Scipy](#)

[Pandas](#)

[SymPy](#)

[Analyse de réseaux](#)

[Machine learning /
statistiques](#)

[Graphiques](#)

[Matplotlib](#)

[Personnalisation](#)

[Seaborn](#)

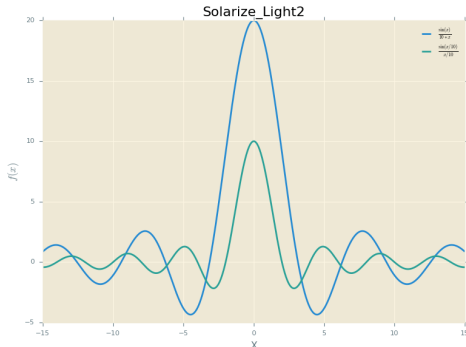
[Concurrents 2D](#)

[Graphiques 3D](#)

Personnalisation

Vous pouvez surcharger la configuration de Matplotlib :

- ▶ passer des paramètres à la fonction de dessin
- ▶ changer de style
- ▶ modifier le `pyplot.rcParams`



Thème : Solarize_Light2

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

[Écosystème scientifique](#)

[Jupyter](#)

[Numpy](#)

[Scipy](#)

[Pandas](#)

[SymPy](#)

[Analyse de réseaux](#)

[Machine learning /
statistiques](#)

[Graphiques](#)

[Matplotlib](#)

[Personnalisation](#)

[Seaborn](#)

[Concurrents 2D](#)

[Graphiques 3D](#)

Personnalisation

Vous pouvez surcharger la configuration de Matplotlib :

- ▶ passer des paramètres à la fonction de dessin
- ▶ changer de style
- ▶ modifier le `pyplot.rcParams`

```
from matplotlib import pyplot as plt
import numpy as np
```

```
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.serif'] = 'Ubuntu'
plt.rcParams['ytick.labelsize'] = 8
plt.rcParams['legend.fontsize'] = 10
plt.rcParams['figure.titlesize'] = 20
plt.rcParams['lines.linewidth'] = 10
```

```
xs = np.linspace(-15, 15, 1000)
ys1 = 20 * np.sin(xs) / xs
ys2 = 10 * np.sinc(xs / 2)
```

```
plt.plot(xs, ys1, label=r"$\frac{\sin(x)}{x}$")
plt.plot(xs, ys2, label=r"$\frac{\sin(x/10)}{x/10}$")
```

```
plt.title("Courbes")
plt.legend()
plt.xlabel("x")
plt.ylabel("$f(x)$")
plt.title("Surcharge rcParams")
plt.savefig("styles/rcParams.png")
plt.show()
```

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

[Écosystème scientifique](#)

[Jupyter](#)

[Numpy](#)

[Scipy](#)

[Pandas](#)

[SymPy](#)

[Analyse de réseaux](#)

[Machine learning /
statistiques](#)

[Graphiques](#)

[Matplotlib](#)

[Personnalisation](#)

[Seaborn](#)

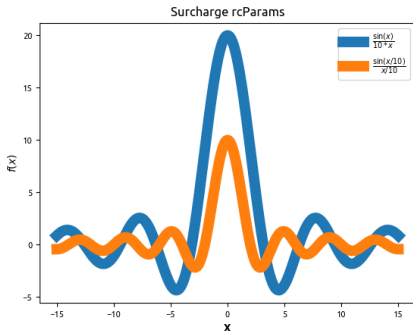
[Concurrents 2D](#)

[Graphiques 3D](#)

Personnalisation

Vous pouvez surcharger la configuration de Matplotlib :

- ▶ passer des paramètres à la fonction de dessin
- ▶ changer de style
- ▶ modifier le `pyplot.rcParams`



Surcharge de rcParams

Matthieu Falce

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

[Écosystème scientifique](#)

[Jupyter](#)

[Numpy](#)

[Scipy](#)

[Pandas](#)

[SymPy](#)

[Analyse de réseaux](#)

[Machine learning /
statistiques](#)

[Graphiques](#)

[Matplotlib](#)

[Personnalisation](#)

[Seaborn](#)

[Concurrents 2D](#)

[Graphiques 3D](#)

Seaborn

Seaborn se base sur matplotlib.

Il rajoute des styles et des fonctionnalités (surtout utile en statistiques).

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

Seaborn

Seaborn se base sur matplotlib.

Il rajoute des styles et des fonctionnalités (surtout utile en statistiques).

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
R = np.random.random((5, 5))
sns.heatmap(R)
plt.savefig("sns_heatmap.png")
```

```
plt.clf()
A = np.random.normal(10, 1, 100)
B = np.random.normal(6, 5, 100)
sns.boxplot(x=["A", "B"], y=[A, B])
plt.savefig("sns_boxplot.png")
```

```
plt.clf()
sns.kdeplot(A, shade=True, label="A")
sns.distplot(B, label="B")
plt.legend()
plt.savefig("sns_distplot.png")
```

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

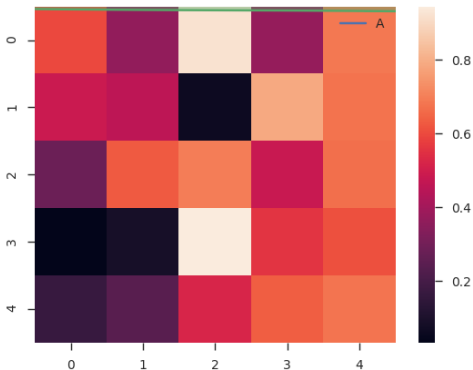
Concurrents 2D

Graphiques 3D

Seaborn

Seaborn se base sur matplotlib.

Il rajoute des styles et des fonctionnalités (surtout utile en statistiques).



Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

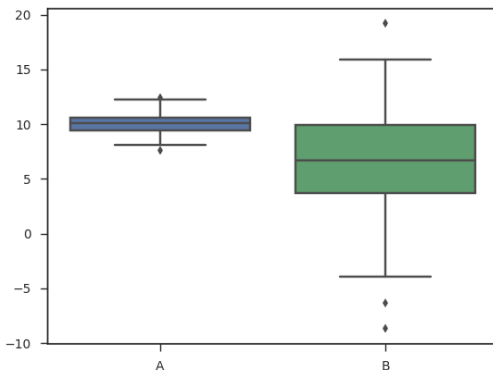
Concurrents 2D

Graphiques 3D

Seaborn

Seaborn se base sur matplotlib.

Il rajoute des styles et des fonctionnalités (surtout utile en statistiques).



Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

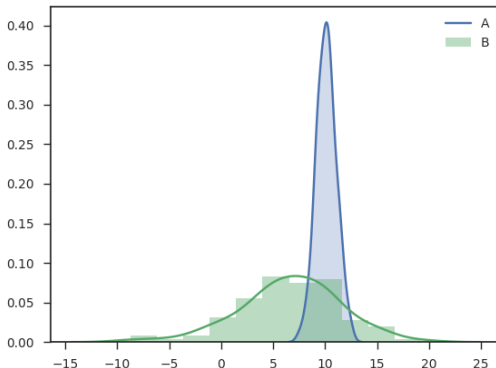
Seaborn

Concurrents 2D

Graphiques 3D

Seaborn se base sur matplotlib.

Il rajoute des styles et des fonctionnalités (surtout utile en statistiques).



Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

Matplotlib n'est pas la seule bibliothèque de graphiques pour Python :

- ▶ (seaborn)
- ▶ plotly (figures web interactives)
- ▶ mpld3 (transforme une figure mpl en Javascript)
- ▶ bokeh (figures web interactive)
- ▶ plotly (figures web interactive)
- ▶ ggplot (port de la bibliothèque ggplot2 de R)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

Concurrent : *plotly*

- ▶ interactive
- ▶ web charts

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

Concurrent : *plotly*

- ▶ interactive
- ▶ web charts

```
import plotly.express as px
```

```
xs = [i for i in range(100)]  
fig = px.scatter(x=xs, y=[i ** 2 for i in xs])  
fig.show()
```

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

Concurrent : *plotly*

Matthieu Falce

```
import plotly.graph_objects as go
```

```
fig = go.Figure(  
    data=go.Scatter(  
        x=[1, 2, 3, 4],  
        y=[10, 11, 12, 13],  
        mode="markers",  
        marker=dict(  
            size=[40, 60, 80, 100],  
            color=[0, 1, 2, 3]),  
    )  
)  
  
fig.show()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

Concurrent : *plotly*

```
import matplotlib.pyplot as plt
import plotly
from plotly.tools import mpl_to_plotly

fig, ax = plt.subplots()
ax.plot([1, 2, 3], [1, 4, 9], "o")

plotly_fig = mpl_to_plotly(fig)
```

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

On peut utiliser Dash ⁸³

- ▶ utilise plotly
- ▶ basé sur le framework web flask
- ▶ permet de créer des dashboards web interactifs sans faire de HTML / JS
- ▶ bindings en R et Python
- ▶ exemples : <https://dash-gallery.plotly.host/Portal/>

83.<https://plot.ly/dash/>

Graphiques 3D

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

```
# source
# https://matplotlib.org/examples/mplot3d/lines3d\_demo.html
```

```
import matplotlib as mpl
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
import matplotlib.pyplot as plt
```

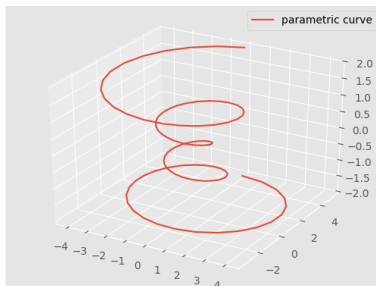
```
mpl.rcParams['legend.fontsize'] = 10
```

```
fig = plt.figure()
ax = fig.gca(projection='3d')
theta = np.linspace(-4 * np.pi, 4 * np.pi, 100)
z = np.linspace(-2, 2, 100)
r = z**2 + 1
x = r * np.sin(theta)
y = r * np.cos(theta)
ax.plot(x, y, z, label='parametric curve')
ax.legend()
```

```
plt.savefig("test_3d.png")
plt.show()
```

Graphiques 3D

Matthieu Falce



Résultat graphique 3D

[Vue d'ensemble](#)

[Langage Python](#)

[Programmation
Orientée objet
\(POO\)](#)

[Bonnes pratiques](#)

[Bibliothèque
standard](#)

[Création de
modules](#)

[Programmation
concurrente](#)

[Python scientifique](#)

[Écosystème scientifique](#)

[Jupyter](#)

[Numpy](#)

[Scipy](#)

[Pandas](#)

[SymPy](#)

[Analyse de réseaux](#)

[Machine learning /
statistiques](#)

[Graphiques](#)

[Matplotlib](#)

[Personnalisation](#)

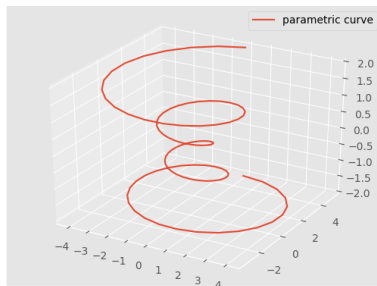
[Seaborn](#)

[Concurrents 2D](#)

[Graphiques 3D](#)

Graphiques 3D

Matthieu Falce



Résultat graphique 3D



Ce n'est pas de la "vraie" 3D... (pas de notion de volumes)

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

Graphiques 3D

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Matplotlib

Personnalisation

Seaborn

Concurrents 2D

Graphiques 3D

Pour faire de la vraie 3d :

- ▶ mayavi
- ▶ <https://lorensen.github.io/VTKExamples/site/Python/>
- ▶ (ParaView)
- ▶ moteurs de jeu 3D

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Numexpr

Dask

Bibliographie

8- Python scientifique

8.10. Performances



N'optimisez que ce qui est nécessaire

- ▶ faites des tests de performances (“profiling”)
- ▶ n'optimisez que ce qui est nécessaire
- ▶ ne commencez que quand tout fonctionne et est testé
- ▶ évitez les copies et les mauvaises structures mémoires
- ▶ utilisez de bons algorithmes
- ▶ préférez les méthodes de Scipy souvent plus rapide que celles de Numpy
- ▶ zen of Numpy
- ▶ ...

Les calculs Numpy se font en générant des tableaux intermédiaires. Numexpr permet de les supprimer en effectuant les calculs directement

```
import numpy as np
import numexpr as ne

a = np.arange(1e6)
b = np.arange(1e6)

c = ne.evaluate("a + 1")
# %timeit c = ne.evaluate("a + 1")
# 866 µs ± 74.6 µs per loop
# (mean ± std. dev. of 7 runs, 1000 loops each)

# %timeit c = a + 1
# 845 µs ± 37.2 µs per loop
# (mean ± std. dev. of 7 runs, 1000 loops each)

d = ne.evaluate("sin(a) + arcsinh(a/b)")
# %timeit np.sin(a) + np.arcsinh(a/b)
# The slowest run took 6.65 times longer than the fastest.
# This could mean that an intermediate result is being cached.
# 154 ms ± 139 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

# %timeit ne.evaluate("sin(a) + arcsinh(a/b)")
# 66.2 ms ± 2.11 ms per loop
# (mean ± std. dev. of 7 runs, 10 loops each)
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Numexpr

Dask

Bibliographie

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Autres

Bibliographie

8- Python scientifique

8.11. Dask

Framework de parallélisme.

Pour les *medium data* (ne tient plus en RAM mais sur un SSD)

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Autres

Bibliographie

Framework de parallélisme.

Pour les *medium data* (ne tient plus en RAM mais sur un SSD)

S'intègre avec (en réutilisant les mêmes API) :

- ▶ numpy
- ▶ pandas
- ▶ scikit learn

[Vue d'ensemble](#)[Langage Python](#)[Programmation
Orientée objet
\(POO\)](#)[Bonnes pratiques](#)[Bibliothèque
standard](#)[Création de
modules](#)[Programmation
concurrente](#)[Python scientifique](#)[Écosystème scientifique](#)[Jupyter](#)[Numpy](#)[Scipy](#)[Pandas](#)[SymPy](#)[Analyse de réseaux](#)[Machine learning /
statistiques](#)[Graphiques](#)[Performances](#)[Dask](#)[Autres](#)[Bibliographie](#)

Framework de parallélisme.

Pour les *medium data* (ne tient plus en RAM mais sur un SSD)

S'intègre avec (en réutilisant les mêmes API) :

- ▶ numpy
- ▶ pandas
- ▶ scikit learn

2 concepts :

- ▶ scheduler : exécute des graphes de calculs (comme make, luigi, celery...)
- ▶ big data collections : partitionnement des données ne tenant pas en RAM

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

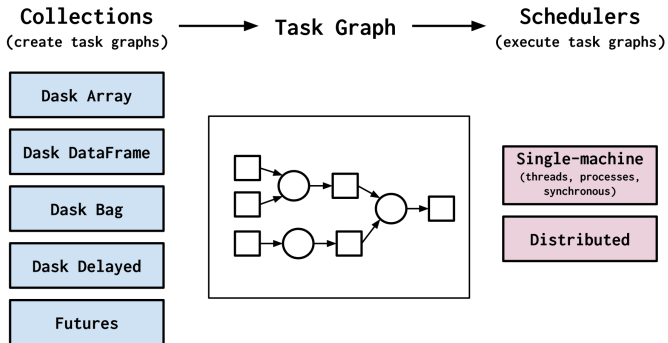
Dask

Autres

Bibliographie

Dask : vue d'ensemble

Source: <https://docs.dask.org/en/latest/>



Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

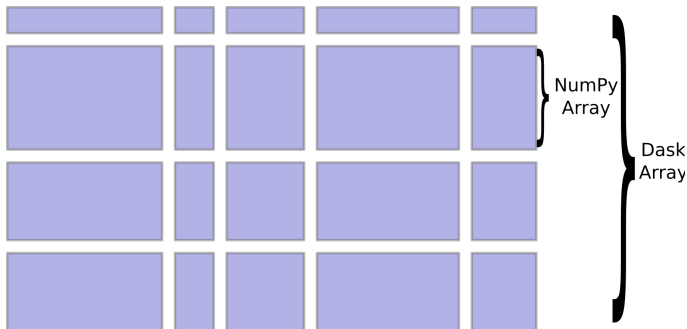
Dask

Autres

Bibliographie

Dask : structure d'un *dask array*

Source: <https://docs.dask.org/en/latest/array.html>



```
from dask.distributed import Client, progress
```

```
client = Client(  
    n_workers=2,  
    threads_per_worker=2,  
    memory_limit="1GB"  
) # workers configuration  
# we can change for process workers  
# to deal with GIL perf issues  
  
# go to : http://127.0.0.1:8787
```



```
# source : https://examples.dask.org/dataframe.html
import dask
import dask.dataframe as dd

# lazy operation, they are only performed when we need the result
df = dask.datasets.timeseries()
df.head()

df2 = df[df.y > 0]
df3 = df2.groupby("name").x.std()

computed_df = df3.compute()
type(computed_df)

df[["x", "y"]].resample("24h").mean().compute().plot()
df[["x", "y"]].rolling(window="24h").mean().head()

# display the call graph
df[["x", "y"]].resample("24h").mean().visualize()

# store in RAM for faster computation
df = df.persist()
```

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Autres

Bibliographie

D'autres techniques, plus ou moins matures permettent d'améliorer les temps de calculs également :

- ▶ Numba
- ▶ Pythran
- ▶ (Theano)
- ▶ distribution python par Intel

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

8- Python scientifique

8.12. Bibliographie

Bibliographie I

Matthieu Falce

► graphiques

- <http://www.labri.fr/perso/nrougier/teaching/matplotlib/matplotlib.html#id8>
- <https://python-graph-gallery.com/matplotlib/>
- <https://matplotlib.org/gallery.html>
- <http://pbpython.com/effective-matplotlib.html>
- https://matplotlib.org/faq/usage_faq.html
- <http://futurile.net/2016/02/27/matplotlib-beautiful-plots-with-style/#id16>

► numpy

- <http://www.scipy-lectures.org/numpy/numpy.html>
- http://www.scipy-lectures.org/advanced/advanced_numpy/#block-of-memory
- <https://docs.scipy.org/doc/numpy/reference/internals.html>

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

Bibliographie II

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

- ▶ <https://jakevdp.github.io/PythonDataScienceHandbook/02.01-understanding-data-types.html>
- ▶ <http://www.labri.fr/perso/nrougier/teaching/numpy/numpy.html>
- ▶ <http://www.labri.fr/perso/nrougier/teaching/numpy.100/index.html>

▶ scipy

- ▶ <https://scipy-cookbook.readthedocs.io/>
- ▶ <https://docs.scipy.org/doc/scipy/reference/tutorial/index.html>
- ▶ <https://docs.scipy.org/doc/scipy/reference/index.html>
- ▶ <https://makina-corpus.com/blog/metier/2017/presentation-de-lecosysteme-python-scientifique>

Bibliographie III

Matthieu Falce

► pandas

- la feuille de triche pandas officielle :
https://github.com/pandas-dev/pandas/blob/master/doc/cheatsheet/Pandas_Cheat_Sheet.pdf
- inline vs copy operations : https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#indexing-view-versus-copy
- les explications sur les différentes méthodes d'indexation : <https://stackoverflow.com/questions/28757389/pandas-loc-vs-iloc-vs-at-vs-iat>
- <https://pandas.pydata.org/pandas-docs/stable/dsintro.html>
- <https://pandas.pydata.org/pandas-docs/stable/visualization.html>
- http://falce.net/presentation/python_pandas_monaco_parking
- https://pandas.pydata.org/pandas-docs/stable/user_guide/cookbook.html#cookbook-resample

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

Bibliographie IV

Matthieu Falce

- ▶ **dask**
 - ▶ tutoriel sur comment charger de grandes quantités de données : <https://blog.dask.org/2019/06/20/load-image-data>
 - ▶ notebooks d'exemples / tutos en lignes : <https://hub-binder.mybinder.ovh/user/dask-dask-examples-irbwzcml/lab>
 - ▶ cas d'usages réels : <https://stories.dask.org/en/latest/>
 - ▶ spark vs dask vs base de données : <https://docs.dask.org/en/latest/spark.html>
 - ▶ mise en place + vidéo : <https://docs.dask.org/en/latest/setup.html>
- ▶ **manipulation d'images**
 - ▶ documentation d'opencv : https://docs.opencv.org/master/d7/d4d/tutorial_py_thresholding.html

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

Bibliographie V

Matthieu Falce

Vue d'ensemble

Langage Python

Programmation
Orientée objet
(POO)

Bonnes pratiques

Bibliothèque
standard

Création de
modules

Programmation
concurrente

Python scientifique

Écosystème scientifique

Jupyter

Numpy

Scipy

Pandas

Sympy

Analyse de réseaux

Machine learning /
statistiques

Graphiques

Performances

Dask

Bibliographie

- ▶ les exemples sur scipy-lectures : <https://scipy-lectures.org/packages/scikit-image/index.html> et http://scipy-lectures.org/advanced/image_processing/